Python for Finance - M1 EF Dauphine

Matéo Molinaro

September 2026

1 What Is a High-Level Programming Language?

Python is considered a **high-level programming language**. This means that it provides a high level of abstraction from the physical operation of the computer.

In other words, Python allows the programmer to focus primarily on **what problem should be solved**, rather than on the low-level details of **how the computer executes the corresponding operations**.

For example, consider the following Python code:

```
returns = prices.pct_change()
mean_return = returns.mean()
```

The programmer directly expresses the desired operations: compute returns and then calculate their average.

At a lower level, these operations ultimately involve many additional details, such as memory allocation, numerical representations, CPU instructions, registers, and memory access.

Programming languages can therefore be viewed as different levels of abstraction from the underlying hardware:

Higher level / More abstraction

Python
↓
C / C++
↓
Assembly
↓
Machine instructions
↓
CPU / Memory

Lower level / Closer to hardware

For example, in Python:

```
x = 42
```

Python automatically handles many operations related to the creation and management of the object representing the value `42`. These include, among others, memory allocation, type management, and memory deallocation.

Lower-level languages such as C provide the programmer with substantially more explicit control over these mechanisms.

Key Idea

> A **high-level programming language** abstracts away many of the low-level details of computer hardware, allowing programmers to express the logic of the problem more directly.

Importantly, **high-level does not mean more advanced or more difficult**. It refers to the **level of abstraction from the hardware**.

2 Compiled vs. Interpreted Languages

Another important distinction concerns how source code is transformed into instructions that can ultimately be executed by the computer.

2.1 Compiled Languages

In a traditionally compiled language such as C or C++, the source code is processed by a **compiler** before execution.

Conceptually:

Source code
↓
Compiler
↓
Machine code
↓
CPU

For example, a C program is generally compiled into machine instructions before it is executed.

The compiler can analyze and optimize the program in advance. The resulting machine code can then be executed directly by the processor.

This is one reason why compiled languages such as C and C++ generally provide very high execution performance.

2.2 Interpreted Languages

In the traditional model of an interpreted language, the source code is executed through an **interpreter** at runtime.

Conceptually:

Source code
↓
Interpreter
↓
Execution

Python is therefore commonly described as an **interpreted language**.

However, this description is a simplification.

2.3 What Actually Happens with Python?

The standard implementation of Python, called **CPython**, first compiles Python source code into an intermediate representation called **bytecode**.

The process can be represented as:

Python source code (`.py`)
↓
Compilation
↓
Python bytecode
↓
Python Virtual Machine
↓
Machine instructions / CPU

The bytecode is then executed by the Python interpreter, more precisely by the Python Virtual Machine.

Python can therefore be viewed as involving **both compilation and interpretation**:

> Python source code is typically **compiled into bytecode**, which is then **executed by the Python interpreter**.

3 Programming Paradigms

A **programming paradigm** is a general approach to structuring and expressing the logic of a computer program.

Python is a **multi-paradigm programming language**, meaning that it supports several programming styles. In particular, Python supports three major paradigms:

- **Imperative programming**
- **Object-Oriented Programming (OOP)**
- **Functional programming**

These paradigms are not mutually exclusive. In practice, Python programs often combine elements from all three.

3.1 Imperative Programming

In the **imperative paradigm**, a program is expressed as a sequence of instructions that modify the **state** of the program.

The programmer explicitly specifies **how** a computation should be performed, step by step.

For example:

```
prices = [100, 105, 103, 110]

returns = []

for i in range(1, len(prices)):
    r = prices[i] / prices[i - 1] - 1
    returns.append(r)
```

The program explicitly describes the sequence of operations:

Initialize variables → Iterate over prices → Compute return → Modify `returns`

The state of the variable `returns` changes progressively during execution.

Typical features of imperative programming include:

- variable assignments;
- loops such as `for` and `while`;
- conditional statements such as `if`;
- explicit modification of program state;
- execution of instructions in a specified order.

Key idea.

> Imperative programming describes **how to perform a computation** through a sequence of instructions that modify the program state.

Procedural programming, where instructions are organized into reusable procedures or functions, can be viewed as a form of imperative programming.

3.2 Object-Oriented Programming

Object-Oriented Programming (OOP) organizes a program around **objects**.

An object combines:

- **data**, called **attributes**;
- **behavior**, represented by **methods**.

Objects are typically created from **classes**, which define their structure and behavior.

For example, a simple portfolio can be represented as:

```
class Portfolio:

    def __init__(self, weights):
        self.weights = weights

    def compute_return(self, asset_returns):
        return sum(
            w * r
            for w, r in zip(self.weights, asset_returns)
        )


portfolio = Portfolio([0.5, 0.3, 0.2])

r = portfolio.compute_return([0.02, -0.01, 0.03])
```

Here, the object `portfolio` contains both:

State	Behavior
`weights`	`compute_return()`

This approach is particularly useful for organizing large applications.

For example, a quantitative backtesting system could contain:

```
DataHandler
SignalModel
Portfolio
RiskModel
Backtester
PerformanceAnalyzer
```

Each class is responsible for a specific part of the system.

Key idea.

> Object-oriented programming organizes programs around **objects that combine data and behavior**.

3.3 Functional Programming

The **functional paradigm** treats computation primarily as the evaluation and composition of **functions**.

A central idea is to avoid unnecessary changes in state and favor functions whose output depends only on their inputs.

For example:

```
def compute_return(p0, p1):
    return p1 / p0 - 1
```

The function takes inputs and produces an output without modifying variables outside the function.

Conceptually:

$$\text{Inputs} \rightarrow \text{Function} \rightarrow \text{Output}$$

For the same inputs, a **pure function** always produces the same output and does not create side effects.

Python also provides several tools inspired by functional programming, including:

- `lambda` expressions;
- `map()`;
- `filter()`;
- `reduce()`;
- comprehensions;
- higher-order functions.

For example:

```
returns = [0.02, -0.01, 0.03, -0.02]

positive_returns = list(
    filter(lambda r: r > 0, returns)
)
```

Functional programming is particularly useful when transformations can be expressed as compositions of independent functions.

Key idea.

> Functional programming describes computations primarily through the **evaluation and composition of functions**, while limiting changes to program state and side effects.

3.4 Comparing the Three Paradigms

The same problem can often be approached using different programming paradigms.

Paradigm	Central Idea	Typical Python Features
Imperative	Execute instructions that progressively modify program state.	Assignments, `for`, `while`, `if`
Object-Oriented	Organize data and behavior into interacting objects.	Classes, objects, attributes, methods
Functional	Express computations as functions and transformations while limiting state changes.	Functions, `lambda`, `map`, `filter`, comprehensions

A useful simplified distinction is:

Imperative	"Which instructions should the computer execute?"
Object-Oriented	"Which objects should represent the system?"
Functional	"Which functions transform the inputs into outputs?"

Python allows these approaches to be combined. For example, a quantitative finance application may use **objects** to represent portfolios and strategies, **imperative code** to control a backtest through time, and **functional transformations** to compute signals and process data.

Therefore, Python is not only a **high-level** and commonly described as an **interpreted** language, but also a **multi-paradigm programming language**.

4 Sets

A **set** is a built-in Python data structure that stores a collection of **unique elements**.

Sets are particularly useful when:

- duplicate values should be removed;
- we want to test whether an element belongs to a collection;
- we want to perform mathematical set operations such as unions, intersections, and differences.

4.1 Creating a Set

A set can be created using curly brackets:

```
stocks = {"AAPL", "MSFT", "GOOG"}
```

Unlike a list, a set contains only **unique values**.

For example:

```
stocks = {"AAPL", "MSFT", "AAPL", "GOOG"}

print(stocks)
```

produces a set containing only:

```
{"AAPL", "MSFT", "GOOG"}
```

The duplicate `"AAPL"` is automatically removed.

A list can also be converted into a set:

```
stocks = ["AAPL", "MSFT", "AAPL", "GOOG"]

unique_stocks = set(stocks)
```

This is a common way to remove duplicates from a collection.

4.2 Set vs. List

Sets and lists represent collections of elements, but they have important differences.

Property	**List**	**Set**
Syntax	`[a, b, c]`	`{a, b, c}`
Duplicates	Allowed	Not allowed
Ordered	Yes	No guaranteed positional order
Indexing	Yes	No
Membership testing	Generally $O(n)$	Average $O(1)$

For example, a list can be indexed:

```
stocks = ["AAPL", "MSFT", "GOOG"]

print(stocks[0])
```

whereas a set cannot:

```
stocks = {"AAPL", "MSFT", "GOOG"}

stocks[0]   # Error
```

A set should therefore be viewed primarily as a collection based on **membership**, rather than position.

4.3 Adding and Removing Elements

Elements can be added using `add()`:

```
stocks = {"AAPL", "MSFT"}

stocks.add("GOOG")
```

Elements can be removed using `remove()`:

```
stocks.remove("MSFT")
```

or `discard()`:

```
stocks.discard("MSFT")
```

The difference is that `remove()` raises an error if the element does not exist, whereas `discard()` does not.

4.4 Membership Testing

One of the main advantages of sets is efficient membership testing.

For example:

```
universe = {"AAPL", "MSFT", "GOOG", "NVDA"}

if "AAPL" in universe:
    print("Stock belongs to the universe")
```

Membership testing in a Python set is, on average,

$$O(1),$$

because sets are implemented using **hash tables**.

By comparison, searching for an element in a list generally requires

$$O(n)$$

operations in the worst case.

4.5 Mathematical Set Operations

Python sets naturally support standard mathematical set operations.

Suppose we have two investment universes:

```
universe_A = {"AAPL", "MSFT", "GOOG", "NVDA"}
universe_B = {"MSFT", "NVDA", "AMZN", "META"}
```

Union

The **union** contains all elements appearing in either set:

$$A \cup B.$$

In Python:

```
universe_A | universe_B
```

which gives:

```
{"AAPL", "MSFT", "GOOG", "NVDA", "AMZN", "META"}
```

Intersection

The **intersection** contains elements appearing in both sets:

$$A \cap B.$$

In Python:

```
universe_A & universe_B
```

which gives:

```
{"MSFT", "NVDA"}
```

Difference

The **difference** contains elements belonging to the first set but not to the second:

$$A \setminus B.$$

In Python:

```
universe_A - universe_B
```

which gives:

```
{"AAPL", "GOOG"}
```

Symmetric Difference

The **symmetric difference** contains elements appearing in exactly one of the two sets:

$$A \triangle B.$$

In Python:

```
universe_A ^ universe_B
```

4.6 An Important Detail: The Empty Set

An empty set must be created using:

```
empty_set = set()
```

and **not**:

```
empty_set = {}
```

Indeed, `{}` creates an empty **dictionary**, not an empty set.

4.7 Summary

> A Python **set** is an unordered collection of unique elements, optimized for membership testing and mathematical set operations.

The most important operations to remember are:

Operation	Python	Mathematics
Membership	`x in A`	$x \in A$
Union	`A | B`	$A \cup B$
Intersection	`A & B`	$A \cap B$
Difference	`A - B`	$A \setminus B$
Symmetric difference	`A ^ B`	$A \triangle B$

5 Hashability in Python

5.1 What Does "Hashable" Mean?

In Python, an object is said to be **hashable** if Python can associate it with a **hash value** that remains stable during the object's lifetime.

A hash value can be obtained using the built-in function:

```
hash(object)
```

For example:

```
hash(42)
hash("Python")
hash((1, 2, 3))
```

Integers, strings, and tuples of hashable objects are generally hashable.

5.2 What Is a Hash Value?

A **hash function** transforms an object into an integer:

$$\boxed{h : \text{Object} \longrightarrow \mathbb{Z}}$$

Conceptually:

$$\texttt{"AAPL"} \xrightarrow{\text{hash function}} \text{integer hash value}$$

Python uses these hash values internally to efficiently locate objects in **hash tables**.

Hash tables are the data structure underlying:

- dictionaries (`dict`);
- sets (`set`).

This is why membership testing in a set or dictionary is very efficient on average:

$$\boxed{O(1)}$$

5.3 Why Is a List Not Hashable?

A Python `list` is **mutable**: its contents can be modified after the list has been created.

For example:

```
x = [1, 2, 3]

x.append(4)

print(x)
# [1, 2, 3, 4]
```

If lists were hashable, changing the contents of a list could change the information from which its hash would naturally be computed.

Conceptually:

$$[1, 2, 3] \xrightarrow{\text{hash}} h_1$$

but after modifying the list:

$$[1, 2, 3, 4] \xrightarrow{\text{hash}} h_2.$$

This would violate the requirement that the hash of an object used as a hash-table key must remain stable.

Python therefore deliberately makes lists **unhashable**.

For example:

```
x = [1, 2, 3]

hash(x)
```

raises:

```
TypeError: unhashable type: 'list'
```

5.4 Practical Consequences

Because lists are not hashable, they cannot be used as:

- elements of a `set`;
- keys of a `dict`.

For example, the following is invalid:

```
my_set = {[1, 2], [3, 4]}
```

Similarly:

```
my_dict = {
    [1, 2]: "A"
}
```

Both raise a `TypeError`.

5.5 List vs. Tuple

This explains an important difference between lists and tuples.

A list is mutable:

```
x = [1, 2, 3]
x[0] = 10
```

and is therefore not hashable.

A tuple is immutable:

```
x = (1, 2, 3)
```

and, provided all its elements are themselves hashable, it can be hashable:

```
hash((1, 2, 3))
```

Therefore, a tuple can be used as a dictionary key:

```
prices = {
    ("AAPL", "2026-09-07"): 220.5,
    ("MSFT", "2026-09-07"): 410.2
}
```

5.6 Summary

Type	Mutable?	Hashable?
`int`	No	Yes
`float`	No	Yes
`str`	No	Yes
`tuple`	No	Usually yes*
`list`	Yes	No
`set`	Yes	No

**A tuple is hashable only if all of its elements are hashable.*

> **Key idea:** A hashable object has a stable hash value and can therefore be used in hash-based structures such as dictionary keys or set elements. A list is mutable, so Python deliberately makes it unhashable.

6 Correction: `for` and `while` Loops

6.1 `for` Loops

Consider the following list:

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
```

Loop 1

```
for stock in my_stocks:
    print(stock)
```

The `for` loop successively assigns each element of `my_stocks` to the variable `stock`.

The output is therefore:

```
AMZN
TSLA
AAPL
META
```

Conceptually:

$$\texttt{stock} = \texttt{AMZN} \rightarrow \texttt{TSLA} \rightarrow \texttt{AAPL} \rightarrow \texttt{META}.$$

Loop 2

Consider:

```
for i in range(1, 10):
    print(i)
```

The function

```
range(start, stop)
```

generates integers from `start` up to, but **excluding**, `stop`.

Therefore,

$$\texttt{range(1,10)} = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}.$$

The output is:

```
1
2
3
4
5
6
7
8
9
```

> **Important:** the upper bound of `range()` is excluded.

Loop 3: Iterating over a Dictionary

Consider:

```
my_prices = [133.62, 889.36, 167.23, 161.11]

my_ptf = dict(zip(my_stocks, my_prices))

for stock, price in my_ptf.items():
    print(f"Le prix de {stock} est {price}")
```

First, `zip()` associates the elements having the same positions:

$$\begin{aligned}
\texttt{AMZN} &\longleftrightarrow 133.62,\\
\texttt{TSLA} &\longleftrightarrow 889.36,\\
\texttt{AAPL} &\longleftrightarrow 167.23,\\
\texttt{META} &\longleftrightarrow 161.11.
\end{aligned}$$

Then,

```
dict(zip(my_stocks, my_prices))
```

creates the dictionary:

```
{
    'AMZN': 133.62,
    'TSLA': 889.36,
    'AAPL': 167.23,
    'META': 161.11
}
```

The method `items()` provides the key-value pairs of the dictionary. At each iteration, Python performs tuple unpacking into:

```
stock, price
```

The output is:

```
Le prix de AMZN est 133.62
Le prix de TSLA est 889.36
Le prix de AAPL est 167.23
Le prix de META est 161.11
```

6.2 `while` Loops

Consider:

```
portfolio1 = ['AMZN', 'TSLA', 'AAPL', 'META']
portfolio2 = ['V', 'TTE', 'FTKE', 'F']

while True:
```

```
    portfolio1.append(portfolio2[-1])
    portfolio2.pop()

    if not bool(portfolio2):
        break

print(portfolio1)
```

Why `while True`?

A `while` loop continues executing as long as its condition evaluates to `True`.

For example:

```
while condition:
    ...
```

Using:

```
while True:
```

therefore creates an **infinite loop**, because the condition `True` can never become `False`.

Conceptually:

$$\texttt{True} \rightarrow \text{execute} \rightarrow \texttt{True} \rightarrow \text{execute} \rightarrow \cdots$$

The loop must consequently be terminated explicitly using:

```
break
```

In this example, the stopping condition is:

```
if not bool(portfolio2):
    break
```

Understanding the Stopping Condition

For a list, Python follows the convention:

$$\texttt{bool(non-empty list)} = \texttt{True},$$
$$\texttt{bool(empty list)} = \texttt{False}.$$

Therefore, when:

```
portfolio2 = []
```

we have:

```
bool(portfolio2)
# False
```

and consequently:

```
not bool(portfolio2)
# True
```

The instruction `break` is then executed and the loop stops.

Execution Step by Step

Initially:

```
portfolio1 = ['AMZN', 'TSLA', 'AAPL', 'META']
portfolio2 = ['V', 'TTE', 'FTKE', 'F']
```

At each iteration, the last element of `portfolio2` is first copied into `portfolio1` and then removed from `portfolio2`.

Iteration	Element transferred	portfolio2 after pop()
1	F	[V, TTE, FTKE]
2	FTKE	[V, TTE]
3	TTE	[V]
4	V	[]

Notice that the elements are transferred in **reverse order**, because index `-1` and `pop()` operate on the last element.

The final output is:

```
['AMZN', 'TSLA', 'AAPL', 'META', 'F', 'FTKE', 'TTE', 'V']
```

6.3 A More Concise Variant

The previous loop can be written as:

```
portfolio1 = ['AMZN', 'TSLA', 'AAPL', 'META']
portfolio2 = ['V', 'TTE', 'FTKE', 'F']

while True:
    portfolio1.append(portfolio2.pop())

    if not bool(portfolio2):
        break

print(portfolio1)
```

Why Does This Work?

The important point is that `pop()` does **two things**:

1. it removes an element from the list;
2. it **returns the removed element**.

For example:

```
x = ['A', 'B', 'C']

value = x.pop()
```

After execution:

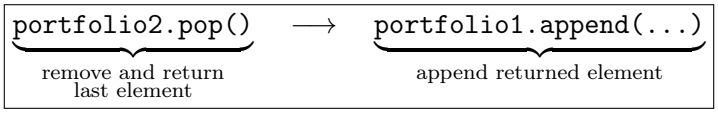
```
value
# 'C'

x
# ['A', 'B']
```

Therefore:

```
portfolio1.append(portfolio2.pop())
```

can be understood as:



Thus the following two instructions:

```
portfolio1.append(portfolio2[-1])
portfolio2.pop()
```

can be replaced by the single instruction:

```
portfolio1.append(portfolio2.pop())
```

Why Is It More Efficient / Parsimonious?

The first version performs two logically separate operations:

```
portfolio1.append(portfolio2[-1])  # access last element
portfolio2.pop()                   # remove last element
```

The second version directly uses the value returned by `pop()`:

```
portfolio1.append(portfolio2.pop())
```

It is therefore:

- **shorter**: one instruction instead of two;
- **less redundant**: the last element does not need to be explicitly accessed before being removed;
- **clearer**: the code directly expresses the operation "move the last element from one list to the other".

Both versions have the same overall time complexity:

$$\boxed{O(n)}$$

for transferring n elements, since `pop()` from the end of a Python list and `append()` are both $O(1)$ amortized operations.

Thus, "more efficient" here should mainly be understood as **more concise and less redundant**, rather than as a change in asymptotic computational complexity.

Additional Simplification

The condition:

```
if not bool(portfolio2):
```

can itself be simplified.

Python automatically interprets an empty list as `False` and a non-empty list as `True`. Therefore, the more idiomatic expression is:

```
if not portfolio2:
    break
```

The loop can therefore be written as:

```
while True:
    portfolio1.append(portfolio2.pop())

    if not portfolio2:
        break
```

An even simpler solution is to put the condition directly in the `while` statement:

```
while portfolio2:
    portfolio1.append(portfolio2.pop())
```

This means:

> "Continue executing the loop while `portfolio2` is not empty."

For this particular problem, this last version is the most concise and idiomatic Python implementation.

7 Correction: Conditional Statements

Consider the following code:

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
stocks2 = ['V', 'TTE', 'AAPL', 'MS', 'AMZN']

for stocks in stocks2:
    if stocks in my_stocks:
        print(f'{stocks} is all ready in the first portfolio')
    else:
        my_stocks.append(stocks)

print(my_stocks)
```

7.1 Understanding the Condition

The instruction

```
if stocks in my_stocks:
```

tests whether the current stock from `stocks2` is already present in `my_stocks`.

The operator `in` is a **membership operator**:

$$\texttt{x in collection} = \begin{cases} \texttt{True}, & \text{if } x \text{ belongs to the collection,} \\ \texttt{False}, & \text{otherwise.} \end{cases}$$

If the stock already exists, a message is printed.

Otherwise, the `else` block is executed and the stock is added to `my_stocks` using:

```
my_stocks.append(stocks)
```

7.2 Step-by-Step Execution

Initially:

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
```

The loop then examines each element of `stocks2`.

Stock	Already present?	Condition	Action
`V`	No	`False`	Append `V`
`TTE`	No	`False`	Append `TTE`
`AAPL`	Yes	`True`	Print message
`MS`	No	`False`	Append `MS`
`AMZN`	Yes	`True`	Print message

Thus, the evolution of `my_stocks` is:

$$
\begin{aligned}
&[\texttt{AMZN}, \texttt{TSLA}, \texttt{AAPL}, \texttt{META}] \\
&\quad \downarrow + \texttt{V} \\
&[\texttt{AMZN}, \texttt{TSLA}, \texttt{AAPL}, \texttt{META}, \texttt{V}] \\
&\quad \downarrow + \texttt{TTE} \\
&[\texttt{AMZN}, \texttt{TSLA}, \texttt{AAPL}, \texttt{META}, \texttt{V}, \texttt{TTE}] \\
&\quad \downarrow + \texttt{MS} \\
&[\texttt{AMZN}, \texttt{TSLA}, \texttt{AAPL}, \texttt{META}, \texttt{V}, \texttt{TTE}, \texttt{MS}]
\end{aligned}
$$

7.3 Output

The two stocks already present in the first portfolio generate a message:

```
AAPL is all ready in the first portfolio
AMZN is all ready in the first portfolio
```

Finally, the instruction

```
print(my_stocks)
```

produces:

```
['AMZN', 'TSLA', 'AAPL', 'META', 'V', 'TTE', 'MS']
```

Therefore, the complete output is:

```
AAPL is all ready in the first portfolio
AMZN is all ready in the first portfolio
['AMZN', 'TSLA', 'AAPL', 'META', 'V', 'TTE', 'MS']
```

7.4 Important Detail

The condition is evaluated against `my_stocks`, which is **modified during the execution of the loop**.

For example, after the first iteration:

```
my_stocks.append('V')
```

the list becomes:

```
['AMZN', 'TSLA', 'AAPL', 'META', 'V']
```

Consequently, subsequent membership tests use the **updated list**, not the original one.

> **Key idea:** The `if` statement checks a condition. If the condition is `True`, the `if` block is executed; otherwise, the `else` block is executed.

8 Methods: Referencing vs. Calling a Method

Consider the following class:

```
class Voiture:

    def __init__(self, marque: str, couleur: str):
        self.marque = marque
        self.couleur = couleur

    def open_door(self):
        return "Door opened"
```

We can create an instance of the class:

```
my_car = Voiture("Renault", "noire")
```

The object `my_car` is an **instance** of the class `Voiture`.

8.1 Accessing a Method

Consider:

```
method_open = my_car.open_door
```

Notice that there are **no parentheses** after `open_door`.

Therefore, the method is **not executed**.

Instead, Python retrieves the method itself and stores a reference to it inside the variable `method_open`.

Conceptually:

`method_open` $\longrightarrow$ `my_car.open_door`

Thus:

```
type(method_open)
```

returns:

```
<class 'method'>
```

The variable `method_open` therefore contains a **method object**, not the result produced by the method.

8.2 Calling a Method

To **execute** the method, parentheses must be added:

```
my_car.open_door()
```

or, since the method has been stored in another variable:

```
method_open()
```

The parentheses `()` represent the **function/method call**.

The execution can be represented as:

$$\texttt{method_open} \xrightarrow{()} \text{execute method} \xrightarrow{\texttt{return}} \texttt{"Door opened"}$$

Therefore:

```
print(method_open())
```

produces:

```
Door opened
```

8.3 Method vs. Return Value

This distinction explains the following result:

```
print(type(method_open), type(method_open()))
```

which produces:

```
<class 'method'> <class 'str'>
```

Indeed:

$$\underbrace{\texttt{method_open}}_{\text{method itself}} \neq \underbrace{\texttt{method_open()}}_{\text{result of executing the method}}$$

More precisely:

`method_open` ⇒ `method`

whereas:

`method_open()` ⇒ `"Door opened"` ⇒ `str`.

The type of the result depends on what the method returns.

In this example:

```
def open_door(self):
    return "Door opened"
```

returns a string, so:

```
type(method_open())
```

returns:

```
<class 'str'>
```

8.4 Storing the Method vs. Storing its Result

It is important to distinguish between:

```
method_open = my_car.open_door
```

and:

```
result = my_car.open_door()
```

In the first case:

`method_open` = the method itself

The method can therefore be executed later:

```
method_open()
method_open()
method_open()
```

Each instruction calls the method again.

In the second case:

`result` = the value returned by the method

Hence:

```
result = my_car.open_door()

print(result)
```

produces:

```
Door opened
```

Here, `result` is a string, not a method.

8.5 What Happens to `self`?

The method is defined as:

```
def open_door(self):
    return "Door opened"
```

but we call it using:

```
my_car.open_door()
```

and not:

```
my_car.open_door(my_car)
```

This is because Python automatically passes the instance `my_car` as the first argument of the method.

Conceptually:

`my_car.open_door()`

is approximately equivalent to:

`Voiture.open_door(my_car).`

Thus, inside the method:

$\boxed{\texttt{self} = \texttt{my_car}}$

This allows the method to access the attributes of the specific instance, for example:

```
def description(self):
    return f"{self.marque} - {self.couleur}"
```

8.6 Summary

Expression	Meaning	Type
`my_car`	Instance of `Voiture`	`Voiture`
`my_car.open_door`	Reference to the method	`method`
`my_car.open_door()`	Execution of the method	`str`
`method_open`	Reference to the method	`method`
`method_open()`	Execution of the stored method	`str`

Key idea:

`my_car.open_door` refers to the **method itself**, whereas `my_car.open_door()` **calls the method and returns its result**.

In Python, functions and methods are objects themselves. They can therefore be assigned to variables and called later.

9 Correction: Importing Data from an HTML Page

Consider the following code:

```
html = 'https://en.wikipedia.org/wiki/'
html = html + 'List_of_largest_oil_and_gas_companies_by_revenue'

stock_data = pd.read_html(html)

df = pd.DataFrame(data=stock_data[0])
```

9.1 What Does `pd.read_html()` Return?

The function

```
pd.read_html(html)
```

searches the HTML page for **HTML tables** and converts them into Pandas DataFrames.

An important point is that `read_html()` does not return a single DataFrame. It returns a **list of DataFrames**:

$$\boxed{\texttt{pd.read_html(url)} \longrightarrow [\texttt{DataFrame}_0, \texttt{DataFrame}_1, \ldots, \texttt{DataFrame}_n]}$$

Each element of the list corresponds to one table detected on the web page.

We can therefore inspect the number of tables using:

```
print(len(stock_data))
```

9.2 Selecting the First Table

The expression

```
stock_data[0]
```

selects the **first table** found on the page.

Since this object is already a DataFrame, writing

```
pd.DataFrame(data=stock_data[0])
```

is actually redundant.

We can simply write:

```
df = stock_data[0]
```

9.3 What Does the Extracted Data Look Like?

The extracted table contains information about large oil and gas companies and their revenues across different years.

Conceptually, the DataFrame has a structure similar to:

Company	**2019**	**2020**	**2021**	**2022**
Company A	250.1	180.2	290.4	410.3
Company B	190.3	140.5	260.1	380.7
Company C	150.8	100.2	210.4	300.1
⋮	⋮	⋮	⋮	⋮

Each:

- **row** represents a company;
- **column** represents a variable or year;
- **cell** contains the corresponding value.

9.4 What Should We Do First?

When importing external data, we should **never immediately assume that the dataset is ready for analysis**.

The first step is to inspect its structure.

A useful initial diagnostic is:

```
df.head()
```

which displays the first five rows.

We should then examine:

```
df.shape
df.columns
df.index
df.dtypes
df.info()
```

These commands answer different questions:

Command	Purpose
`df.head()`	Inspect the first observations
`df.shape`	Number of rows and columns
`df.columns`	Inspect column names
`df.index`	Inspect the row index
`df.dtypes`	Check the data type of each column
`df.info()`	General structure and missing values

9.5 What Problems Should We Look For?

Data extracted from an HTML page is **raw external data**. Before using it for quantitative analysis, several potential problems should be checked.

1. Missing Values

We can inspect missing observations using:

```
df.isna().sum()
```

Missing observations may require deletion, imputation, or another treatment depending on the application.

2. Incorrect Data Types

A column containing numbers may have been imported as strings:

```
df.dtypes
```

For example, values containing symbols, footnotes, or additional text may prevent Pandas from recognizing a column as numeric.

A numerical conversion may therefore be necessary:

```
df["2022"] = pd.to_numeric(df["2022"], errors="coerce")
```

3. Unnecessary Columns

HTML tables may contain columns that are not useful for our analysis.

They can be removed using:

```
df = df.drop(columns=["Column"])
```

4. Column Names

Column names may need to be simplified or standardized:

```
df.columns
```

For example:

```
df = df.rename(columns={"Company name": "company"})
```

5. Duplicates

Duplicated observations can be detected using:

```
df.duplicated().sum()
```

and removed, if appropriate, using:

```
df = df.drop_duplicates()
```

9.6 From Raw HTML Data to Usable Data

The general workflow is therefore:

HTML page → `read_html()` → Raw DataFrame → Inspection → Cleaning → Analysis

In practice:

```
# 1. Import
tables = pd.read_html(html)

# 2. Select the relevant table
df = tables[0]

# 3. Inspect
print(df.head())
print(df.shape)
print(df.dtypes)
```

```
print(df.isna().sum())

# 4. Clean
# ...

# 5. Analyze
# ...
```

> **Key idea:** Importing data is only the first step. External data should always be **inspected, validated, and cleaned** before being used for statistical or financial analysis.

10 Replacing Words in a String Using a Dictionary

Consider the following Python code:

```
# initializing string
my_str = 'I have Apple, Nvidia, Ford and BoFa in my portfolio'

# printing original string
print("The original string is : " + str(my_str))

# lookup Dictionary
MaJ_portfolio = {"Nvidia": "AMD", "BoFa": "Morgan Stanley"}

# performing split()
temp = my_str.split()
res = []

for wrd in temp:
    comma = False if wrd.strip(",") == wrd else True
    res.append(
        MaJ_portfolio.get(wrd.strip(','), wrd.strip(',')) + ','
        if comma
        else MaJ_portfolio.get(wrd, wrd)
    )

res = ' '.join(res)

# printing result
print("Replaced Strings : " + str(res))
```

The objective is to replace some words in a string according to a **lookup dictionary**, while keeping the other words and punctuation unchanged.

10.1 Step 1: Initializing the String

The initial string is:

```
my_str = 'I have Apple, Nvidia, Ford and BoFa in my portfolio'
```

The variable `my_str` is therefore an object of type `str`.

The instruction:

```
print("The original string is : " + str(my_str))
```

produces:

```
The original string is : I have Apple, Nvidia, Ford and BoFa in my portfolio
```

Since `my_str` is already a string, the use of `str(my_str)` is actually unnecessary. We could simply write:

```
print("The original string is : " + my_str)
```

10.2 Step 2: Creating the Lookup Dictionary

The dictionary:

```
MaJ_portfolio = {
    "Nvidia": "AMD",
    "BoFa": "Morgan Stanley"
}
```

defines the replacements that should be performed.

Mathematically, we can think of it as a mapping:

$$\begin{aligned} \texttt{Nvidia} &\longrightarrow \texttt{AMD}, \\ \texttt{BoFa} &\longrightarrow \texttt{Morgan Stanley}. \end{aligned}$$

The keys are the words we want to detect, while the values are their replacements.

Words that are not keys of the dictionary should remain unchanged.

10.3 Step 3: Splitting the String

The instruction:

```
temp = my_str.split()
```

splits the string using whitespace as the default separator.

Therefore, `temp` becomes:

```
[
    'I',
    'have',
    'Apple,',
    'Nvidia,',
    'Ford',
    'and',
    'BoFa',
    'in',
    'my',
    'portfolio'
]
```

Notice an important detail:

```
'Apple,'
'Nvidia,'
```

still contain their commas.

The method `split()` separates the string at spaces, but it does **not** automatically remove punctuation.

10.4 Step 4: Creating the Result List

The instruction:

```
res = []
```

creates an empty list.

This list will progressively contain the transformed words:

$$\texttt{res} = [\text{word}_1, \text{word}_2, \ldots].$$

10.5 Step 5: Iterating over the Words

The loop:

```
for wrd in temp:
```

iterates over every element of `temp`.

For example, successive values of `wrd` are:

$$\texttt{'I'} \to \texttt{'have'} \to \texttt{'Apple,'} \to \texttt{'Nvidia,'} \to \cdots$$

10.6 Step 6: Detecting a Comma

For each word, the following instruction is executed:

```
comma = False if wrd.strip(",") == wrd else True
```

This is a **conditional expression**, whose general syntax is:

```
value_if_true if condition else value_if_false
```

The method:

```
wrd.strip(",")
```

removes commas from the beginning and the end of the string.

For example:

```
"Nvidia".strip(",")
# 'Nvidia'

"Nvidia,".strip(",")
# 'Nvidia'
```

Therefore, if:

```
wrd.strip(",") == wrd
```

then no comma has been removed, and:

```
comma = False
```

Otherwise:

```
comma = True
```

For example:

wrd	wrd.strip(",")	comma
"Ford"	"Ford"	False
"Apple,"	"Apple"	True
"Nvidia,"	"Nvidia"	True

10.7 Step 7: Looking Up and Replacing the Word

The main instruction is:

```
res.append(
    MaJ_portfolio.get(wrd.strip(','), wrd.strip(',')) + ','
    if comma
    else MaJ_portfolio.get(wrd, wrd)
)
```

It combines three operations:

1. dictionary lookup using `get()`;
2. a conditional expression using `if ... else`;
3. insertion into `res` using `append()`.

Understanding `dict.get()`

The general syntax is:

```
dictionary.get(key, default)
```

It means:

$$\texttt{dictionary.get(key, default)} = \begin{cases} \text{value associated with key}, & \text{if key exists}, \\ \text{default}, & \text{otherwise}. \end{cases}$$

For example:

```
MaJ_portfolio.get("Nvidia", "Nvidia")
```

returns:

```
'AMD'
```

because `"Nvidia"` is a key of the dictionary.

However:

```
MaJ_portfolio.get("Ford", "Ford")
```

returns:

```
'Ford'
```

because `"Ford"` is not in the dictionary.

The second argument therefore allows words that are not in the dictionary to remain unchanged.

Case 1: The Word Has a Comma

Consider:

```
wrd = "Nvidia,"
```

We have:

```
comma = True
```

so Python evaluates:

```
MaJ_portfolio.get(
    wrd.strip(','),
    wrd.strip(',')
) + ','
```

First:

```
wrd.strip(',')
# 'Nvidia'
```

Then:

```
MaJ_portfolio.get("Nvidia", "Nvidia")
# 'AMD'
```

Finally, the comma is added back:

```
'AMD' + ','
# 'AMD,'
```

and:

```
res.append('AMD,')
```

is executed.

Thus:

$$\boxed{\texttt{"Nvidia,"} \longrightarrow \texttt{"Nvidia"} \longrightarrow \texttt{"AMD"} \longrightarrow \texttt{"AMD,"}}$$

Case 2: The Word Has No Comma

Consider:

```
wrd = "BoFa"
```

Since there is no comma:

```
comma = False
```

Python therefore executes:

```
MaJ_portfolio.get(wrd, wrd)
```

which becomes:

```
MaJ_portfolio.get("BoFa", "BoFa")
```

Since `"BoFa"` exists in the dictionary, Python returns:

```
'Morgan Stanley'
```

Hence:

$$\boxed{\texttt{"BoFa"} \longrightarrow \texttt{"Morgan Stanley"}}$$

Case 3: Comma but No Replacement

Consider:

```
wrd = "Apple,"
```

A comma is detected, so it is temporarily removed:

```
wrd.strip(",")
# 'Apple'
```

However, `"Apple"` is not a key in `MaJ_portfolio`.

Therefore:

```
MaJ_portfolio.get("Apple", "Apple")
# 'Apple'
```

The comma is then restored:

```
'Apple' + ','
# 'Apple,'
```

Thus the original word remains unchanged:

$$\boxed{\texttt{"Apple,"} \longrightarrow \texttt{"Apple,"}}$$

10.8 Step-by-Step Result

The transformation performed by the loop is:

Original	Dictionary lookup	Result
`I`	Not found	`I`
`have`	Not found	`have`
`Apple,`	Not found	`Apple,`
`Nvidia,`	Nvidia → AMD	`AMD,`
`Ford`	Not found	`Ford`
`and`	Not found	`and`
`BoFa`	BoFa → Morgan Stanley	`Morgan Stanley`
`in`	Not found	`in`
`my`	Not found	`my`
`portfolio`	Not found	`portfolio`

At the end of the loop, `res` is therefore:

```
[
    'I',
    'have',
    'Apple,',
    'AMD,',
    'Ford',
    'and',
    'Morgan Stanley',
    'in',
    'my',
    'portfolio'
]
```

10.9 Step 8: Reconstructing the String with `join()`

At this stage, `res` is still a list.

The instruction:

```
res = ' '.join(res)
```

combines all elements of the list into a single string, inserting one space between consecutive elements.

For example:

```
' '.join(['I', 'have', 'AMD'])
```

returns:

```
'I have AMD'
```

Therefore:

```
res = ' '.join(res)
```

produces:

```
I have Apple, AMD, Ford and Morgan Stanley in my portfolio
```

Conceptually:

$$\texttt{list of strings} \xrightarrow{\texttt{' '.join()}} \texttt{single string}$$

10.10 Final Output

The final instruction:

```
print("Replaced Strings : " + str(res))
```

prints:

```
Replaced Strings : I have Apple, AMD, Ford and Morgan Stanley in my portfolio
```

The complete output of the program is therefore:

```
The original string is : I have Apple, Nvidia, Ford and BoFa in my portfolio
Replaced Strings : I have Apple, AMD, Ford and Morgan Stanley in my portfolio
```

10.11 Summary

The complete transformation can be represented as:

Original string
↓ `split()`
List of words
↓ `for`
Process each word
↓
Detect and temporarily remove comma
↓ `dict.get()`
Replace word if present in dictionary
↓
Restore comma if necessary
↓ `append()`
Build result list
↓ `' '.join()`
Reconstructed string

Key idea: The program uses a dictionary as a **lookup table**. Each word is searched in the dictionary: if it exists, it is replaced by the corresponding value; otherwise, it remains unchanged. Punctuation is temporarily removed for the lookup and subsequently restored.

11 Function Example: Euclidean Distance

Consider the following Python function:

```
from math import sqrt

def euclidian_norm(point1: list, point2: list = [0, 0]) -> float:
    vector = [
        point1[i] - point2[i]
        for i in range(len(point1))
    ]

    return sqrt(
        sum(
            list(
                map(lambda x: x**2, vector)
            )
        )
    )
```

The objective of this function is to compute the **Euclidean distance** between two points.

11.1 Mathematical Definition

Consider two points in $\mathbb{R}^n$:

$$P_1 = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \qquad P_2 = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}.$$

The vector connecting P_2 to P_1 is:

$$P_1 - P_2 = \begin{pmatrix} x_1 - y_1 \\ x_2 - y_2 \\ \vdots \\ x_n - y_n \end{pmatrix}.$$

The Euclidean distance between the two points is the norm of this vector:

$$\boxed{d(P_1, P_2) = \|P_1 - P_2\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}}$$

For example, if:

$$P_1 = (3, 4), \qquad P_2 = (0, 0),$$

then:

$$d(P_1, P_2) = \sqrt{(3-0)^2 + (4-0)^2} = \sqrt{9+16} = 5.$$

11.2 Understanding the Function Signature

The function is defined as:

```
def euclidian_norm(
    point1: list,
    point2: list = [0, 0]
) -> float:
```

There are several important elements.

`point1: list`

The notation:

`point1: list`

is a **type hint** indicating that `point1` is expected to be a list.

For example:

`point1 = [3, 4]`

`point2: list = [0, 0]`

The second argument also has the type hint `list`, but it additionally has a **default value**:

`point2 = [0, 0]`

Therefore, calling:

`euclidian_norm([3, 4])`

is equivalent to:

`euclidian_norm([3, 4], [0, 0])`

In this case, the function computes the distance between the point and the origin:

$$\|(3, 4)\|_2 = 5.$$

```
-> float
```

The notation:

```
-> float
```

is a **return type hint**.

It indicates that the function is expected to return a floating-point number.

Type hints provide information to programmers, IDEs, and static type checkers, but Python does not normally enforce them at runtime.

11.3 Step 1: Constructing the Difference Vector

The first instruction is:

```
vector = [
    point1[i] - point2[i]
    for i in range(len(point1))
]
```

This is a **list comprehension**.

Its general structure is:

```
[expression for element in iterable]
```

Here:

```
range(len(point1))
```

generates all valid indices of `point1`.

For example, if:

```
point1 = [3, 4]
```

then:

```
len(point1)
# 2

range(len(point1))
# 0, 1
```

The expression:

```
point1[i] - point2[i]
```

is therefore evaluated for $i = 0$ and $i = 1$.

For:

```
point1 = [3, 4]
point2 = [0, 0]
```

we obtain:

$$i = 0 : 3 - 0 = 3,$$
$$i = 1 : 4 - 0 = 4.$$

Hence:

```
vector = [3, 4]
```

More generally:

$$\boxed{\texttt{vector} = [x_1 - y_1,\, x_2 - y_2,\, \ldots,\, x_n - y_n]}$$

11.4 Step 2: Squaring Every Component

The next operation is:

```
map(lambda x: x**2, vector)
```

The Lambda Function

The expression:

```
lambda x: x**2
```

defines a small **anonymous function**.

It is equivalent to:

```
def square(x):
    return x**2
```

Mathematically:

$$\boxed{f(x) = x^2}$$

The `map()` Function

The function:

```
map(function, iterable)
```

applies a function to every element of an iterable.
Therefore:

```
map(lambda x: x**2, vector)
```

applies:

$$x \mapsto x^2$$

to every component of `vector`.
If:

```
vector = [3, 4]
```

then conceptually:

$$[3, 4] \longrightarrow [3^2, 4^2] \longrightarrow [9, 16].$$

The `map()` function returns a `map` object rather than directly returning a list.

11.5 Step 3: Converting the `map` Object into a List

The instruction:

```
list(map(lambda x: x**2, vector))
```

converts the `map` object into a list.
For example:

```
vector = [3, 4]

list(map(lambda x: x**2, vector))
```

returns:

```
[9, 16]
```

Thus:

$$\boxed{[x_1 - y_1, \ldots, x_n - y_n] \longrightarrow [(x_1 - y_1)^2, \ldots, (x_n - y_n)^2]}$$

11.6 Step 4: Summing the Squared Components

The function:

```
sum(...)
```

adds all the elements.

Therefore:

```
sum([9, 16])
```

returns:

```
25
```

Mathematically:

$$\boxed{\sum_{i=1}^{n}(x_i - y_i)^2}$$

11.7 Step 5: Taking the Square Root

Finally:

```
sqrt(...)
```

computes the square root of the sum.

For our example:

$$\sqrt{25} = 5.$$

Thus:

```
euclidian_norm([3, 4])
```

returns:

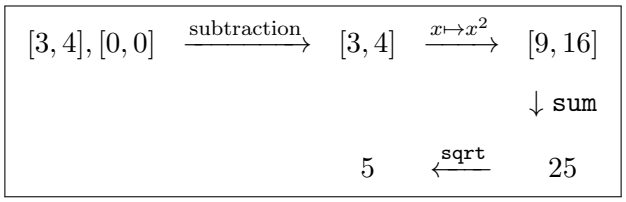
```
5.0
```

11.8 Complete Computation

For:

```
euclidian_norm([3, 4], [0, 0])
```

the complete computation is:



which corresponds exactly to:

$$\sqrt{(3-0)^2+(4-0)^2}=5.$$

11.9 Reading the Return Statement from the Inside Out

The original instruction is:

```
return sqrt(
    sum(
        list(
            map(lambda x: x**2, vector)
        )
    )
)
```

Because functions are nested, it should be read **from the inside out**:

`vector` → `map()` → `list()` → `sum()` → `sqrt()`

For `vector = [3,4]`:

$$[3,4] \to [9,16] \to 25 \to 5.$$

11.10 A Simpler Python Implementation

The conversion to `list` is actually unnecessary because `sum()` can directly consume the iterator returned by `map()`.

The function can therefore be simplified to:

```
def euclidian_norm(point1: list,
                   point2: list = [0, 0]) -> float:

    vector = [
        point1[i] - point2[i]
        for i in range(len(point1))
    ]

    return sqrt(
```

```
        sum(map(lambda x: x**2, vector))
    )
```

An even more readable implementation uses a generator expression:

```
def euclidian_norm(point1: list,
                   point2: list = [0, 0]) -> float:

    return sqrt(
        sum(
            (point1[i] - point2[i])**2
            for i in range(len(point1))
        )
    )
```

11.11 Important Remark: Dimensions

The function implicitly assumes that `point1` and `point2` have the same dimension:

$$\boxed{\text{len}(\texttt{point1}) = \text{len}(\texttt{point2})}$$

For example:

```
euclidian_norm([1, 2, 3], [0, 0])
```

will fail because the function eventually tries to access:

```
point2[2]
```

which does not exist.

Moreover, the default value `[0, 0]` means that the default behavior is specifically adapted to points in $\mathbb{R}^2$.

> **Key idea:** The function computes the difference between two points, squares each component of the resulting vector, sums the squared components, and takes the square root. This is exactly the definition of the Euclidean distance:

$$\boxed{\|P_1 - P_2\|_2 = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}}$$

12 Finding the Closest Point in a Matrix

Consider the following code:

```
vector = [1, 1]

print(
    sorted(M, key=lambda x: euclidian_norm(x, vector))
)
```

The objective is to order the rows of the matrix `M` according to their **Euclidean distance** from the vector:

$$\texttt{vector} = [1, 1].$$

12.1 Rows of the Matrix as Points

Suppose that:

```
M = [
    [3, 4],
    [1, 2],
    [5, 5]
]
```

Each row of the matrix represents a point in $\mathbb{R}^2$:

$$P_1 = (3, 4), \qquad P_2 = (1, 2), \qquad P_3 = (5, 5).$$

The reference vector is:

$$V = (1, 1).$$

The objective is therefore to compare the distances:

$$d(P_1, V), \qquad d(P_2, V), \qquad d(P_3, V).$$

12.2 The `sorted()` Function

The general syntax is:

```
sorted(iterable, key=function)
```

The function `sorted()` sorts the elements of an iterable.

The optional argument:

```
key=function
```

specifies the quantity that Python should use to compare the elements.

Therefore:

`sorted(M, key=...)`

means:

> Sort the rows of `M` according to the value returned by the `key` function.

12.3 Understanding the `lambda` Function

The sorting key is:

```
lambda x: euclidian_norm(x, vector)
```

Here, `x` represents one row of the matrix.

For example, Python will successively evaluate the function with:

```
x = [3, 4]
x = [1, 2]
x = [5, 5]
```

The expression:

```
lambda x: euclidian_norm(x, vector)
```

is an anonymous function equivalent to:

```
def distance_to_vector(x):
    return euclidian_norm(x, vector)
```

Thus, the following two expressions are equivalent:

```
sorted(M, key=lambda x: euclidian_norm(x, vector))
```

and:

```
def distance_to_vector(x):
    return euclidian_norm(x, vector)

sorted(M, key=distance_to_vector)
```

12.4 The Sorting Criterion

The function `euclidian_norm()` computes:

$$d(x, V) = \|x - V\|_2.$$

For a point:

$$x = (x_1, x_2)$$

and:

$$V = (1, 1),$$

the sorting criterion is:

$$\boxed{d(x, V) = \sqrt{(x_1 - 1)^2 + (x_2 - 1)^2}}$$

Python therefore associates each row of the matrix with a numerical **sorting key**.

12.5 Example

For:

$$M = \begin{pmatrix} 3 & 4 \\ 1 & 2 \\ 5 & 5 \end{pmatrix} \qquad \text{and} \qquad V = \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

Python computes:

$$d((3, 4), V) = \sqrt{(3 - 1)^2 + (4 - 1)^2} = \sqrt{13} \approx 3.61,$$

$$d((1, 2), V) = \sqrt{(1 - 1)^2 + (2 - 1)^2} = 1,$$

$$d((5, 5), V) = \sqrt{(5 - 1)^2 + (5 - 1)^2} = \sqrt{32} \approx 5.66.$$

We can represent the values used by `sorted()` as:

Row	**Sorting Key: Distance to** $(1, 1)$
`[3, 4]`	3.61
`[1, 2]`	1.00
`[5, 5]`	5.66

Since `sorted()` uses ascending order by default:

$$1 < 3.61 < 5.66,$$

the result is:

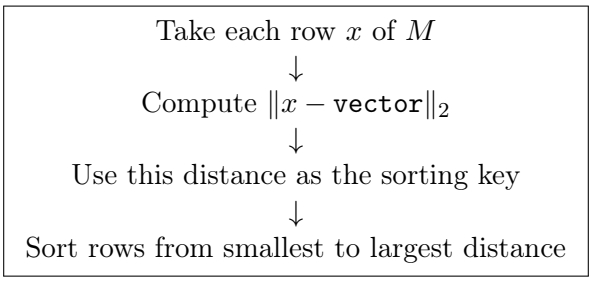
```
[
    [1, 2],
    [3, 4],
    [5, 5]
]
```

12.6 Complete Logic

The instruction:

```
sorted(M, key=lambda x: euclidian_norm(x, vector))
```

can therefore be understood as:



12.7 Returning Only the Closest Point

The original exercise asks for the **row corresponding to the point closest to the given vector**.

The closest point is the first element of the sorted result.

Therefore, we could write:

```
def closest_point(M, vector):
    return sorted(
        M,
        key=lambda x: euclidian_norm(x, vector)
    )[0]
```

Mathematically, we are looking for:

$$x^* = \arg\min_{x \in M} \|x - \texttt{vector}\|_2$$

where arg min returns the value of x that minimizes the distance.

12.8 A Better Solution: Using `min()`

If we only want the closest point, sorting the entire matrix is unnecessary.

Python's `min()` function also accepts a `key` argument:

```
def closest_point(M, vector):
    return min(
        M,
        key=lambda x: euclidian_norm(x, vector)
    )
```

This directly means:

> Return the row `x` of `M` for which `euclidian_norm(x, vector)` is the smallest.

Hence:

`min( M, key=lambda x:  euclidian_norm(x, vector) )` $\iff$ $\arg\min_{x \in M} \|x - \texttt{vector}\|_2$

12.9 Why Is `min()` Preferable?

If the matrix contains n rows, sorting all rows requires approximately:

$$O(n \log n)$$

operations.

Finding only the minimum requires:

$$O(n)$$

operations, since each row only needs to be examined once.

Therefore:

`min(..., key=...)`

is both more explicit and computationally more efficient when we only need the closest point.

> **Key idea:** The `key` argument does not change the elements being returned. It tells Python **which value should be used to compare them**.
>
> Here, each row of the matrix is compared according to its Euclidean distance from the reference vector.

13 String Method Example: `startswith()`

Consider the following function:

```
def find_prefixe(s: str, prefix: str):
    mots = s.split()

    mots_debutant_par_prefixe = [
        mot for mot in mots
        if mot.startswith(prefix)
    ]

    return len(mots_debutant_par_prefixe)
```

The objective of this function is to count how many words in a string **start with a given prefix**.

13.1 The `startswith()` Method

The method:

```
string.startswith(prefix)
```

checks whether a string starts with a specified sequence of characters.

It returns a Boolean:

$$\texttt{string.startswith(prefix)} = \begin{cases} \texttt{True}, & \text{if the string starts with the prefix,} \\ \texttt{False}, & \text{otherwise.} \end{cases}$$

For example:

```
"Apple".startswith("App")
# True

"Apple".startswith("A")
# True

"Apple".startswith("Apple")
# True

"Apple".startswith("ple")
# False

"Microsoft".startswith("Micro")
# True
```

Therefore, a **prefix** is simply a sequence of characters located at the beginning of a string.

For example:

```
"App"
```

is a prefix of:

```
"Apple"
```

because:

$$\texttt{"Apple"} = \texttt{"App"} + \texttt{"le"}$$

13.2 Important: `startswith()` Is Case-Sensitive

String comparisons in Python are case-sensitive.

For example:

```
"Apple".startswith("App")
# True

"Apple".startswith("app")
# False
```

because:

$$\texttt{"A"} \neq \texttt{"a"}.$$

If we want a case-insensitive comparison, we could convert both strings to lowercase:

```
"Apple".lower().startswith("app".lower())
# True
```

13.3 Function Parameters

The function starts with:

```
def find_prefixe(s: str, prefix: str):
```

It takes two arguments:

- `s`: the complete string in which we want to search;
- `prefix`: the sequence of characters that words must start with.

For example:

```
s = "Apple Amazon Microsoft AMD"
prefix = "A"
```

The objective is to count the words beginning with `"A"`.

13.4 Step 1: Splitting the String

The first instruction is:

```
mots = s.split()
```

The method `split()` separates the string at whitespace and returns a list of words.

For:

```
s = "Apple Amazon Microsoft AMD"
```

we obtain:

```
mots = [
    'Apple',
    'Amazon',
    'Microsoft',
    'AMD'
]
```

Thus:

$$\boxed{\texttt{str} \xrightarrow{\texttt{split()}} \texttt{list of strings}}$$

13.5 Step 2: Selecting Words Starting with the Prefix

The next instruction is:

```
mots_debutant_par_prefixe = [
    mot for mot in mots
    if mot.startswith(prefix)
]
```

This is a **list comprehension with a condition**.

Its general syntax is:

```
[expression for element in iterable if condition]
```

Here:

- `mot` is successively each word in `mots`;
- `mot.startswith(prefix)` is the condition;
- the word is included in the new list only if the condition evaluates to `True`.

The code is equivalent to:

```
mots_debutant_par_prefixe = []

for mot in mots:
    if mot.startswith(prefix):
        mots_debutant_par_prefixe.append(mot)
```

13.6 Step-by-Step Example

Consider:

```
s = "Apple Amazon Microsoft AMD"
prefix = "A"
```

After:

```
mots = s.split()
```

we have:

```
['Apple', 'Amazon', 'Microsoft', 'AMD']
```

The list comprehension then evaluates:

mot	mot.startswith("A")	**Selected?**
"Apple"	True	Yes
"Amazon"	True	Yes
"Microsoft"	False	No
"AMD"	True	Yes

Therefore:

```
mots_debutant_par_prefixe
# ['Apple', 'Amazon', 'AMD']
```

13.7 Step 3: Counting the Selected Words

Finally:

```
return len(mots_debutant_par_prefixe)
```

returns the number of elements contained in the filtered list.

In our example:

```
len(['Apple', 'Amazon', 'AMD'])
# 3
```

Hence:

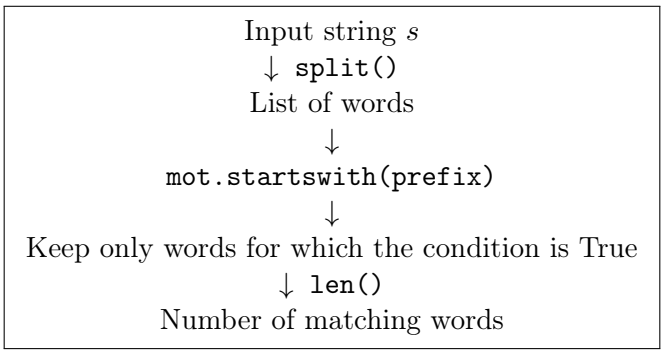
```
find_prefixe(
    "Apple Amazon Microsoft AMD",
    "A"
)
```

returns:

```
3
```

13.8 Complete Logic of the Function

The complete process can be represented as:



For example:

$$
\begin{aligned}
s &= \texttt{"Apple Amazon Microsoft AMD"} \\
&\xrightarrow{\texttt{split()}} [\texttt{Apple}, \texttt{Amazon}, \texttt{Microsoft}, \texttt{AMD}] \\
&\xrightarrow{\texttt{startswith("A")}} [\texttt{Apple}, \texttt{Amazon}, \texttt{AMD}] \\
&\xrightarrow{\texttt{len()}} 3.
\end{aligned}
$$

13.9 Alternative Without Creating an Intermediate List

If the objective is only to count the words, it is not strictly necessary to store all matching words in a list.

For example:

```
def find_prefixe(s: str, prefix: str):
    return sum(
        mot.startswith(prefix)
        for mot in s.split()
    )
```

This works because in Python:

$$\texttt{True} = 1, \qquad \texttt{False} = 0.$$

For example:

$$\texttt{True} + \texttt{True} + \texttt{False} + \texttt{True} = 1 + 1 + 0 + 1 = 3.$$

Key idea: `startswith(prefix)` tests whether a string begins with `prefix`. In this function, it is used as the filtering condition inside a list comprehension, and `len()` then counts how many words satisfy this condition.

14 The `extend()` Method

Consider the following instruction:

```
tous_rendements.extend(rendements)
```

The method `extend()` is used to add **all the elements of an iterable** to an existing list.
For example:

```
tous_rendements = []
rendements = [0.02, -0.01, 0.04]

tous_rendements.extend(rendements)

print(tous_rendements)
```

produces:

```
[0.02, -0.01, 0.04]
```

The three elements of `rendements` have been added individually to `tous_rendements`.

14.1 `append()` vs. `extend()`

It is important to distinguish between:

```
list.append(object)
```

and:

```
list.extend(iterable)
```

The method `append()` adds its argument as **one new element** of the list.
For example:

```
L = [1, 2]

L.append([3, 4])

print(L)
```

returns:

```
[1, 2, [3, 4]]
```

The list `[3, 4]` has been inserted as a **single object**.

We can represent this as:

$$\boxed{[1,2] \xrightarrow{\texttt{append([3,4])}} [1,2,[3,4]]}$$

By contrast, `extend()` iterates over its argument and adds **each element individually**:

```
L = [1, 2]

L.extend([3, 4])

print(L)
```

returns:

```
[1, 2, 3, 4]
```

Thus:

$$\boxed{[1,2] \xrightarrow{\texttt{extend([3,4])}} [1,2,3,4]}$$

14.2 General Rule

If:

```
L = [a, b]
```

then:

```
L.append([c, d])
```

gives:

```
[a, b, [c, d]]
```

whereas:

```
L.extend([c, d])
```

gives:

```
[a, b, c, d]
```

Therefore:

`append()` = add one object

whereas:

`extend()` = add all elements of an iterable

14.3 Application to the Returns Example

Consider the following data:

```
donnees_actions = [
    ("Apple", [0.02, 0.03, -0.01]),
    ("Nvidia", [0.05, 0.04, 0.08]),
    ("Microsoft", [0.01, -0.02, 0.03])
]
```

Inside the loop:

```
for nom, rendements in donnees_actions:
    tous_rendements.extend(rendements)
```

the variable `rendements` successively contains the list of returns of each stock.

First Iteration

For Apple:

```
rendements = [0.02, 0.03, -0.01]
```

so:

```
tous_rendements.extend(rendements)
```

gives:

```
tous_rendements = [0.02, 0.03, -0.01]
```

Second Iteration

For Nvidia:

```
rendements = [0.05, 0.04, 0.08]
```

and the list becomes:

```
tous_rendements = [
    0.02, 0.03, -0.01,
    0.05, 0.04, 0.08
]
```

Third Iteration

For Microsoft:

```
rendements = [0.01, -0.02, 0.03]
```

the final list is:

```
tous_rendements = [
    0.02, 0.03, -0.01,
    0.05, 0.04, 0.08,
    0.01, -0.02, 0.03
]
```

Thus, `extend()` allows us to construct one **flat list** containing all individual return observations:

$$[0.02, 0.03, -0.01] \quad [0.05, 0.04, 0.08] \quad [0.01, -0.02, 0.03]$$
$$\downarrow \texttt{extend()}$$
$$[0.02, 0.03, -0.01, 0.05, 0.04, 0.08, 0.01, -0.02, 0.03]$$

14.4 Why Is `extend()` Used Here?

At the end of the function, we calculate:

```
rendement_median = statistics.median(tous_rendements)
```

We therefore need one flat list containing **all return observations from all stocks**.

Using `extend()` gives:

```
[
    0.02, 0.03, -0.01,
    0.05, 0.04, 0.08,
    0.01, -0.02, 0.03
]
```

If we had instead used:

```
tous_rendements.append(rendements)
```

we would obtain a nested list:

```
[
    [0.02, 0.03, -0.01],
    [0.05, 0.04, 0.08],
    [0.01, -0.02, 0.03]
]
```

which is not the desired structure for directly computing the median of all individual observations.

14.5 Summary

Method	Operation	Example result
`append(x)`	Adds `x` as one element	`[1,2,[3,4]]`
`extend(x)`	Adds each element of `x`	`[1,2,3,4]`

Key idea:

`append()` ⇒ add one object

`extend()` ⇒ add the elements of an iterable

In this example, `extend()` is used to combine the return lists of all stocks into one flat list so that the median return can subsequently be calculated across all observations.

15 Derivation of the Black–Scholes Greeks

We consider the Black–Scholes prices of a European call and put:

$$C(S,t) = S\,N(d_1) - Ke^{-rT}N(d_2),$$

$$P(S,t) = Ke^{-rT}N(-d_2) - S\,N(-d_1),$$

with

$$d_1 = \frac{\ln(S/K) + \left(r + \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}},$$

and

$$d_2 = d_1 - \sigma\sqrt{T}.$$

We denote by

$$N(x) = \int_{-\infty}^{x} \phi(u)\,du$$

the cumulative distribution function of the standard normal distribution, and by

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

its probability density function.

A fundamental identity is:

$$\boxed{\frac{d}{dx} N(x) = \phi(x)}$$

since the derivative of the cumulative distribution function is the probability density function.

15.1 Useful Intermediate Derivatives

Before deriving the Greeks, we first compute several useful derivatives.

Since

$$d_1 = \frac{\ln(S/K) + \left(r + \frac{1}{2}\sigma^2\right) T}{\sigma\sqrt{T}},$$

we have, with respect to S,

$$\frac{\partial d_1}{\partial S} = \frac{1}{S\sigma\sqrt{T}}.$$

Because

$$d_2 = d_1 - \sigma\sqrt{T},$$

we also have

$$\boxed{\frac{\partial d_2}{\partial S} = \frac{1}{S\sigma\sqrt{T}}}$$

For later use, we also need the derivatives with respect to σ:

$$d_1 = \frac{\ln(S/K) + rT}{\sigma\sqrt{T}} + \frac{1}{2}\sigma\sqrt{T},$$

so

$$\frac{\partial d_1}{\partial \sigma} = -\frac{\ln(S/K) + rT}{\sigma^2\sqrt{T}} + \frac{1}{2}\sqrt{T}.$$

Using

$$d_2 = d_1 - \sigma\sqrt{T},$$

we obtain

$$\frac{\partial d_2}{\partial \sigma} = \frac{\partial d_1}{\partial \sigma} - \sqrt{T}.$$

15.2 A Key Identity

A very useful identity in Black–Scholes calculations is

$$\boxed{S\phi(d_1) = Ke^{-rT}\phi(d_2)}$$

Let us derive it.

We know that

$$\phi(d_1) = \frac{1}{\sqrt{2\pi}} e^{-d_1^2/2},$$

and

$$\phi(d_2) = \frac{1}{\sqrt{2\pi}} e^{-d_2^2/2}.$$

Hence

$$\frac{\phi(d_1)}{\phi(d_2)} = \exp\left(-\frac{1}{2}d_1^2 + \frac{1}{2}d_2^2\right).$$

Since

$$d_2 = d_1 - \sigma\sqrt{T},$$

we have

$$d_2^2 - d_1^2 = -2d_1\sigma\sqrt{T} + \sigma^2 T.$$

Therefore,

$$\frac{\phi(d_1)}{\phi(d_2)} = \exp\left(-d_1\sigma\sqrt{T} + \frac{1}{2}\sigma^2 T\right).$$

Now,

$$d_1\sigma\sqrt{T} = \ln(S/K) + \left(r + \frac{1}{2}\sigma^2\right)T,$$

so

$$-d_1\sigma\sqrt{T} + \frac{1}{2}\sigma^2 T = -\ln(S/K) - rT.$$

Thus

$$\frac{\phi(d_1)}{\phi(d_2)} = e^{-\ln(S/K)-rT} = \frac{K}{S}e^{-rT}.$$

Therefore,

$$\boxed{S\phi(d_1) = Ke^{-rT}\phi(d_2)}$$

This identity is what makes several derivative terms cancel.

16 Delta

Delta is defined as

$$\boxed{\Delta = \frac{\partial V}{\partial S}}$$

It measures the sensitivity of the option price to the underlying price.

16.1 Call Delta

For a call,

$$C = SN(d_1) - Ke^{-rT}N(d_2).$$

Differentiate with respect to S:

$$\frac{\partial C}{\partial S} = \frac{\partial}{\partial S}\left[SN(d_1)\right] - Ke^{-rT}\frac{\partial}{\partial S}N(d_2).$$

Using the product rule,

$$\frac{\partial}{\partial S}\left[SN(d_1)\right] = N(d_1) + S\phi(d_1)\frac{\partial d_1}{\partial S}.$$

Also,

$$\frac{\partial}{\partial S}N(d_2) = \phi(d_2)\frac{\partial d_2}{\partial S}.$$

Thus

$$\frac{\partial C}{\partial S} = N(d_1) + S\phi(d_1)\frac{1}{S\sigma\sqrt{T}} - Ke^{-rT}\phi(d_2)\frac{1}{S\sigma\sqrt{T}}.$$

Using

$$S\phi(d_1) = Ke^{-rT}\phi(d_2),$$

the last two terms cancel:

$$\boxed{\Delta_C = N(d_1)}$$

16.2 Put Delta

Using put–call parity,

$$P = C - S + Ke^{-rT},$$

differentiate with respect to S:

$$\frac{\partial P}{\partial S} = \frac{\partial C}{\partial S} - 1.$$

Therefore,

$$\boxed{\Delta_P = N(d_1) - 1}$$

Equivalently,

$$\Delta_P = -N(-d_1).$$

17 Gamma

Gamma is defined as

$$\boxed{\Gamma = \frac{\partial^2 V}{\partial S^2} = \frac{\partial \Delta}{\partial S}}$$

For the call,

$$\Delta_C = N(d_1).$$

Hence

$$\Gamma_C = \phi(d_1)\frac{\partial d_1}{\partial S}.$$

Since

$$\frac{\partial d_1}{\partial S} = \frac{1}{S\sigma\sqrt{T}},$$

we obtain

$$\boxed{\Gamma_C = \frac{\phi(d_1)}{S\sigma\sqrt{T}}}$$

For the put,

$$\Delta_P = N(d_1) - 1.$$

The derivative of the constant -1 is zero, therefore

$$\boxed{\Gamma_P = \Gamma_C = \frac{\phi(d_1)}{S\sigma\sqrt{T}}}$$

Thus calls and puts with the same parameters have the same Gamma.

18 Vega

Vega is defined as

$$\boxed{\nu = \frac{\partial V}{\partial \sigma}}$$

We derive it for the call.

The call price is

$$C = SN(d_1) - Ke^{-rT}N(d_2).$$

Differentiating with respect to σ,

$$\frac{\partial C}{\partial \sigma} = S\phi(d_1)\frac{\partial d_1}{\partial \sigma} - Ke^{-rT}\phi(d_2)\frac{\partial d_2}{\partial \sigma}.$$

Since

$$\frac{\partial d_2}{\partial \sigma} = \frac{\partial d_1}{\partial \sigma} - \sqrt{T},$$

we get

$$\frac{\partial C}{\partial \sigma} = S\phi(d_1)\frac{\partial d_1}{\partial \sigma} - Ke^{-rT}\phi(d_2)\left(\frac{\partial d_1}{\partial \sigma} - \sqrt{T}\right).$$

Expanding,

$$\frac{\partial C}{\partial \sigma} = \left[S\phi(d_1) - Ke^{-rT}\phi(d_2)\right]\frac{\partial d_1}{\partial \sigma} + Ke^{-rT}\phi(d_2)\sqrt{T}.$$

Using the identity

$$S\phi(d_1) = Ke^{-rT}\phi(d_2),$$

the first term vanishes:

$$\frac{\partial C}{\partial \sigma} = Ke^{-rT}\phi(d_2)\sqrt{T}.$$

Using the same identity again,

$$\boxed{\nu_C = S\phi(d_1)\sqrt{T}}$$

The put has the same Vega:

$$\boxed{\nu_P = S\phi(d_1)\sqrt{T}}$$

Hence

$$\boxed{\nu_C = \nu_P}$$

If Vega is reported per one percentage point of volatility, then

$$\boxed{\nu_{1\%} = \frac{S\phi(d_1)\sqrt{T}}{100}}$$

19 Theta

Theta is the sensitivity of the option price to time.

There are two common conventions.

If t denotes calendar time,

$$\boxed{\Theta = \frac{\partial V}{\partial t}}$$

Since

$$T = T_{\text{maturity}} - t,$$

we have

$$\frac{\partial T}{\partial t} = -1.$$

Therefore,

$$\boxed{\frac{\partial V}{\partial t} = -\frac{\partial V}{\partial T}}$$

Below we derive the usual market Theta, which corresponds to

$$\Theta = \frac{\partial V}{\partial t} = -\frac{\partial V}{\partial T}.$$

19.1 Call Theta

The call price is

$$C = SN(d_1) - Ke^{-rT}N(d_2).$$

Differentiate first with respect to T:

$$\frac{\partial C}{\partial T} = S\phi(d_1)\frac{\partial d_1}{\partial T} - \frac{\partial}{\partial T}\left[Ke^{-rT}N(d_2)\right].$$

Using the product rule,

$$\frac{\partial}{\partial T}\left[Ke^{-rT}N(d_2)\right] = -rKe^{-rT}N(d_2) + Ke^{-rT}\phi(d_2)\frac{\partial d_2}{\partial T}.$$

Therefore,

$$\frac{\partial C}{\partial T} = S\phi(d_1)\frac{\partial d_1}{\partial T} + rKe^{-rT}N(d_2) - Ke^{-rT}\phi(d_2)\frac{\partial d_2}{\partial T}.$$

Using

$$S\phi(d_1) = Ke^{-rT}\phi(d_2),$$

we can combine the first and third terms:

$$\frac{\partial C}{\partial T} = S\phi(d_1)\left(\frac{\partial d_1}{\partial T} - \frac{\partial d_2}{\partial T}\right) + rKe^{-rT}N(d_2).$$

Since

$$d_2 = d_1 - \sigma\sqrt{T},$$

we have

$$\frac{\partial d_2}{\partial T} = \frac{\partial d_1}{\partial T} - \frac{\sigma}{2\sqrt{T}}.$$

Thus

$$\frac{\partial d_1}{\partial T} - \frac{\partial d_2}{\partial T} = \frac{\sigma}{2\sqrt{T}}.$$

Hence

$$\frac{\partial C}{\partial T} = \frac{S\phi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}N(d_2).$$

Since market Theta is

$$\Theta_C = -\frac{\partial C}{\partial T},$$

we obtain

$$\boxed{\Theta_C = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2)}$$

19.2 Put Theta

Using put–call parity,

$$P = C - S + Ke^{-rT}.$$

Differentiating with respect to calendar time t,

$$\Theta_P = \Theta_C + rKe^{-rT}.$$

Substituting the call Theta,

$$\Theta_P = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2) + rKe^{-rT}.$$

Since

$$1 - N(d_2) = N(-d_2),$$

we obtain

$$\boxed{\Theta_P = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}N(-d_2)}$$

If Theta is reported per day, a common convention is

$$\boxed{\Theta_{\text{daily}} = \frac{\Theta_{\text{annual}}}{365}}$$

20 Rho

Rho is defined as

$$\boxed{\rho = \frac{\partial V}{\partial r}}$$

20.1 Call Rho

For the call,

$$C = SN(d_1) - Ke^{-rT}N(d_2).$$

Differentiate with respect to r:

$$\frac{\partial C}{\partial r} = S\phi(d_1)\frac{\partial d_1}{\partial r} - \frac{\partial}{\partial r}\left[Ke^{-rT}N(d_2)\right].$$

Since

$$\frac{\partial d_1}{\partial r} = \frac{T}{\sigma\sqrt{T}} = \frac{\sqrt{T}}{\sigma},$$

and similarly,

$$\frac{\partial d_2}{\partial r} = \frac{\sqrt{T}}{\sigma},$$

we obtain

$$\frac{\partial C}{\partial r} = S\phi(d_1)\frac{\sqrt{T}}{\sigma} + KTe^{-rT}N(d_2) - Ke^{-rT}\phi(d_2)\frac{\sqrt{T}}{\sigma}.$$

Using

$$S\phi(d_1) = Ke^{-rT}\phi(d_2),$$

the first and third terms cancel.

Therefore,

$$\boxed{\rho_C = KTe^{-rT}N(d_2)}$$

20.2 Put Rho

Using put–call parity,

$$P = C - S + Ke^{-rT},$$

differentiate with respect to r:

$$\rho_P = \rho_C - KTe^{-rT}.$$

Thus,

$$\rho_P = KTe^{-rT}\left[N(d_2) - 1\right].$$

Using

$$N(d_2) - 1 = -N(-d_2),$$

we obtain

$$\boxed{\rho_P = -KTe^{-rT}N(-d_2)}$$

If Rho is reported per one percentage point change in rates,

$$\boxed{\rho_{1\%} = \frac{\rho}{100}}$$

21 Summary of the Greeks

Greek	**Call**	**Put**
Delta	$N(d_1)$	$N(d_1) - 1$
Gamma	$\frac{\phi(d_1)}{S\sigma\sqrt{T}}$	$\frac{\phi(d_1)}{S\sigma\sqrt{T}}$
Vega	$S\phi(d_1)\sqrt{T}$	$S\phi(d_1)\sqrt{T}$
Theta	$-\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2)$	$-\frac{S\phi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}N(-d_2)$
Rho	$KTe^{-rT}N(d_2)$	$-KTe^{-rT}N(-d_2)$

22 Economic Interpretation

The Greeks can be summarized as local sensitivities of the option price:

$$\boxed{\Delta = \frac{\partial V}{\partial S}}$$

measures the first-order exposure to the underlying price.

$$\boxed{\Gamma = \frac{\partial^2 V}{\partial S^2}}$$

measures the curvature of the option price with respect to the underlying.

$$\boxed{\text{Vega} = \frac{\partial V}{\partial \sigma}}$$

measures the exposure to volatility.

$$\boxed{\Theta = \frac{\partial V}{\partial t}}$$

measures the sensitivity to the passage of time.

$$\boxed{\rho = \frac{\partial V}{\partial r}}$$

measures the sensitivity to the risk-free interest rate.

For small changes in the market variables, a first-order approximation is

$$\Delta V \approx \Delta\, \Delta S + \text{Vega}\, \Delta\sigma + \Theta\, \Delta t + \rho\, \Delta r.$$

Including the second-order effect with respect to the underlying gives

$$\boxed{\Delta V \approx \Delta\, \Delta S + \frac{1}{2}\Gamma(\Delta S)^2 + \text{Vega}\, \Delta\sigma + \Theta\, \Delta t + \rho\, \Delta r}$$

which is the local Taylor expansion underlying the practical interpretation of the Greeks.

23 Computation of Put Theta from Call Theta

Consider the following Python code:

```
theta = (
    -S * norm.pdf(d1) * sigma / (2 * math.sqrt(T))
    - r * K * math.exp(-r * T) * norm.cdf(d2)
)

if option_type == 'put':
    theta += r * K * math.exp(-r * T)
```

The first expression computes the standard Black–Scholes Theta of a European call:

$$\boxed{\Theta_C = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2)}$$

where:

$$\phi(d_1) = \texttt{norm.pdf(d1)}$$

and

$$N(d_2) = \texttt{norm.cdf(d2)}.$$

Thus, the Python line

```
theta = -S * norm.pdf(d1) * sigma / (2 * math.sqrt(T)) \
        - r * K * math.exp(-r * T) * norm.cdf(d2)
```

corresponds exactly to:

$$\Theta_C = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2).$$

23.1 Using Put–Call Parity

The put Theta can be obtained from the call Theta using put–call parity.

For European options:

$$C - P = S - Ke^{-rT}.$$

Equivalently,

$$P = C - S + Ke^{-rT}.$$

Now recall that Theta measures the sensitivity of the option price to the passage of calendar time:

$$\boxed{\Theta = \frac{\partial V}{\partial t}}$$

where t denotes the current calendar time.

In our notation, T denotes the **remaining time to maturity**:

$$T = T_{\text{maturity}} - t,$$

where T_{maturity} is the fixed maturity date.

Therefore,

$$\frac{\partial T}{\partial t} = \frac{\partial}{\partial t}\left(T_{\text{maturity}} - t\right) = -1.$$

Hence,

$$\boxed{\frac{\partial T}{\partial t} = -1}$$

Since the option value V depends on calendar time t through the remaining time to maturity T, we apply the chain rule:

$$\frac{\partial V}{\partial t} = \frac{\partial V}{\partial T}\frac{\partial T}{\partial t}.$$

Using

$$\frac{\partial T}{\partial t} = -1,$$

we obtain

$$\frac{\partial V}{\partial t} = -\frac{\partial V}{\partial T}.$$

Therefore,

$$\boxed{\Theta = \frac{\partial V}{\partial t} = -\frac{\partial V}{\partial T}}$$

The minus sign appears because calendar time t and remaining time to maturity T move in opposite directions:

$$\boxed{t \uparrow \implies T \downarrow}$$

Thus, when differentiating the Black–Scholes formula with respect to the variable T used in the Python implementation, we must take the negative of this derivative to obtain the conventional Theta:

$$\boxed{\Theta = -\frac{\partial V}{\partial T}}$$

Differentiate the put–call parity relation with respect to calendar time t:

$$\Theta_P = \Theta_C + \frac{\partial}{\partial t}\left(Ke^{-rT}\right).$$

Since

$$\frac{\partial}{\partial T}\left(Ke^{-rT}\right) = -rKe^{-rT},$$

and

$$\frac{\partial T}{\partial t} = -1,$$

we obtain

$$\frac{\partial}{\partial t}\left(Ke^{-rT}\right) = rKe^{-rT}.$$

Hence,

$$\boxed{\Theta_P = \Theta_C + rKe^{-rT}}$$

This is precisely what the code does:

```
if option_type == 'put':
    theta += r * K * math.exp(-r * T)
```

The operator

```
+=
```

means:

```
theta = theta + ...
```

Therefore, for a put:

$$\Theta_P = \left[-\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2) \right] + rKe^{-rT}.$$

23.2 Simplifying the Put Theta Formula

Starting from

$$\Theta_P = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2) + rKe^{-rT},$$

factor the last two terms:

$$\Theta_P = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}\left[1 - N(d_2)\right].$$

For the standard normal distribution,

$$N(-x) = 1 - N(x).$$

Therefore,

$$1 - N(d_2) = N(-d_2).$$

Hence,

$$\boxed{\Theta_P = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}N(-d_2)}$$

which is the standard Black–Scholes formula for the Theta of a European put.

23.3 Code-to-Mathematics Correspondence

The first part of the code:

```
-S * norm.pdf(d1) * sigma / (2 * math.sqrt(T))
```

corresponds to:

$$-\frac{S\phi(d_1)\sigma}{2\sqrt{T}}.$$

This is the **time-decay / volatility component**, common to both calls and puts.

The second part:

```
-r * K * math.exp(-r * T) * norm.cdf(d2)
```

corresponds to:

$$-rKe^{-rT}N(d_2).$$

Together, these two terms give the call Theta:

$$\Theta_C = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2).$$

Then, for a put, the code adds:

```
r * K * math.exp(-r * T)
```

which corresponds to:

$$rKe^{-rT}.$$

Therefore:

$$\boxed{\Theta_P = \Theta_C + rKe^{-rT}}$$

and thus:

$$\boxed{\Theta_P = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} + rKe^{-rT}N(-d_2)}$$

24 Expectation of the Exponential of a Normal Random Variable

Suppose that

$$\boxed{X \sim \mathcal{N}(\mu, \sigma^2)}$$

where

$$\mathbb{E}[X] = \mu$$

and

$$\mathrm{Var}(X) = \sigma^2.$$

A fundamental result is

$$\boxed{\mathbb{E}\left[e^X\right] = e^{\mu + \frac{1}{2}\sigma^2}}$$

This result can be obtained directly from the **moment-generating function** (MGF) of the normal distribution.

24.1 Recovering Moments from the MGF

The moment-generating function is defined as

$$M_X(t) = \mathbb{E}\left[e^{tX}\right].$$

The derivatives of the MGF are taken with respect to its argument t.

First Derivative: Recovering $\mathbb{E}[X]$

We first differentiate $M_X(t)$ with respect to t:

$$M_X'(t) = \frac{d}{dt}\mathbb{E}\left[e^{tX}\right].$$

Provided that the moment-generating function exists on an open interval containing $t = 0$, differentiation can be interchanged with expectation:

$$\frac{d}{dt}\mathbb{E}\left[e^{tX}\right] = \mathbb{E}\left[\frac{d}{dt}e^{tX}\right].$$

For a normally distributed random variable, this condition is automatically satisfied because its MGF is finite for every $t \in \mathbb{R}$.

$$M_X'(t) = \mathbb{E}\left[\frac{d}{dt}e^{tX}\right].$$

Since X is treated as constant with respect to t,

$$\frac{d}{dt}e^{tX} = Xe^{tX}.$$

Therefore,

$$\boxed{M_X'(t) = \mathbb{E}\left[Xe^{tX}\right]}$$

This is the **first step**: differentiate with respect to t.

We now perform the **second step**: evaluate the derivative at $t = 0$:

$$M_X'(0) = \mathbb{E}\left[Xe^{0X}\right].$$

Since

$$e^{0X} = 1,$$

we obtain

$$M_X'(0) = \mathbb{E}[X].$$

Hence,

$$\boxed{M_X'(0) = \mathbb{E}[X]}$$

Second Derivative: Recovering $\mathbb{E}[X^2]$

Starting from

$$M_X'(t) = \mathbb{E}\left[Xe^{tX}\right],$$

differentiate once again with respect to t:

$$M_X''(t) = \frac{d}{dt}\mathbb{E}\left[Xe^{tX}\right].$$

Moving the derivative inside the expectation gives

$$M_X''(t) = \mathbb{E}\left[\frac{d}{dt}\left(Xe^{tX}\right)\right].$$

Since X does not depend on t,

$$\frac{d}{dt}\left(Xe^{tX}\right) = X\frac{d}{dt}e^{tX}.$$

Using

$$\frac{d}{dt}e^{tX} = Xe^{tX},$$

we obtain

$$\frac{d}{dt}\left(Xe^{tX}\right) = X^2e^{tX}.$$

Therefore,

$$\boxed{M_X''(t) = \mathbb{E}\left[X^2e^{tX}\right]}$$

This is the first step: compute the second derivative with respect to t.

We then evaluate it at $t = 0$:

$$M_X''(0) = \mathbb{E}\left[X^2e^{0X}\right].$$

Again,

$$e^{0X} = 1,$$

so

$$M_X''(0) = \mathbb{E}[X^2].$$

Hence,

$$\boxed{M_X''(0) = \mathbb{E}[X^2]}$$

General Result

More generally, differentiating n times with respect to t gives

$$\boxed{M_X^{(n)}(t) = \mathbb{E}\left[X^ne^{tX}\right]}$$

and evaluating at $t = 0$ gives

$$M_X^{(n)}(0) = \mathbb{E}\left[X^ne^{0X}\right].$$

Since $e^{0X} = 1$,

$$\boxed{M_X^{(n)}(0) = \mathbb{E}[X^n]}$$

Thus, the derivatives of the moment-generating function evaluated at $t = 0$ recover the raw moments of X:

$$\boxed{\begin{aligned} &M_X(0) = 1, \\ &M_X'(0) = \mathbb{E}[X], \\ &M_X''(0) = \mathbb{E}[X^2], \\ &M_X'''(0) = \mathbb{E}[X^3], \\ &\quad\vdots \\ &M_X^{(n)}(0) = \mathbb{E}[X^n]. \end{aligned}}$$

The key mechanism is therefore

$$\boxed{e^{tX} \xrightarrow{\frac{d}{dt}} Xe^{tX} \xrightarrow{\frac{d}{dt}} X^2e^{tX} \xrightarrow{\frac{d}{dt}} \cdots}$$

followed by setting $t = 0$, which makes the exponential term disappear:

$$e^{0X} = 1.$$

For a normally distributed random variable

$$X \sim \mathcal{N}(\mu, \sigma^2),$$

the moment-generating function is

$$\boxed{M_X(t) = e^{\mu t + \frac{1}{2}\sigma^2 t^2}}$$

Let us derive this expression explicitly.

24.2 Derivation of the Normal MGF

By definition,

$$M_X(t) = \mathbb{E}[e^{tX}].$$

Since X is continuous, an expectation can be written as an integral:

$$\mathbb{E}[g(X)] = \int_{-\infty}^{+\infty} g(x) f_X(x)\, dx.$$

Taking

$$g(x) = e^{tx},$$

we obtain

$$M_X(t) = \int_{-\infty}^{+\infty} e^{tx} f_X(x)\, dx.$$

The probability density function of

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

is

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Therefore,

$$M_X(t) = \int_{-\infty}^{+\infty} e^{tx} \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) dx.$$

Combining the two exponential terms gives

$$M_X(t) = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{+\infty} \exp\left(tx - \frac{(x-\mu)^2}{2\sigma^2}\right) dx.$$

24.3 Completing the Square

The key step is to simplify the exponent:

$$tx - \frac{(x-\mu)^2}{2\sigma^2}.$$

First expand the square:

$$(x-\mu)^2 = x^2 - 2\mu x + \mu^2.$$

Hence,

$$tx - \frac{x^2 - 2\mu x + \mu^2}{2\sigma^2}.$$

Putting everything over the same denominator,

$$= -\frac{x^2 - 2\mu x - 2\sigma^2 tx + \mu^2}{2\sigma^2}.$$

Combining the terms in x,

$$= -\frac{x^2 - 2(\mu + \sigma^2 t)x + \mu^2}{2\sigma^2}.$$

We now complete the square.

Recall that

$$x^2 - 2ax = (x-a)^2 - a^2.$$

Taking

$$a = \mu + \sigma^2 t,$$

we obtain

$$x^2 - 2(\mu + \sigma^2 t)x = \left(x - \mu - \sigma^2 t\right)^2 - \left(\mu + \sigma^2 t\right)^2.$$

Therefore,

$$tx - \frac{(x-\mu)^2}{2\sigma^2} = -\frac{\left(x - \mu - \sigma^2 t\right)^2}{2\sigma^2} + \frac{(\mu + \sigma^2 t)^2 - \mu^2}{2\sigma^2}.$$

Now expand

$$(\mu + \sigma^2 t)^2 - \mu^2.$$

We have

$$(\mu + \sigma^2 t)^2 = \mu^2 + 2\mu\sigma^2 t + \sigma^4 t^2.$$

Thus,

$$(\mu + \sigma^2 t)^2 - \mu^2 = 2\mu\sigma^2 t + \sigma^4 t^2.$$

Dividing by $2\sigma^2$,

$$\frac{2\mu\sigma^2 t + \sigma^4 t^2}{2\sigma^2} = \mu t + \frac{1}{2}\sigma^2 t^2.$$

Hence,

$$\boxed{tx - \frac{(x-\mu)^2}{2\sigma^2} = -\frac{(x - \mu - \sigma^2 t)^2}{2\sigma^2} + \mu t + \frac{1}{2}\sigma^2 t^2}$$

24.4 Returning to the Integral

Substituting this expression into the MGF gives

$$M_X(t) = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{+\infty} \exp\left(-\frac{(x - \mu - \sigma^2 t)^2}{2\sigma^2} + \mu t + \frac{1}{2}\sigma^2 t^2\right) dx.$$

The terms

$$\mu t + \frac{1}{2}\sigma^2 t^2$$

do not depend on x, so they can be taken outside the integral:

$$M_X(t) = e^{\mu t + \frac{1}{2}\sigma^2 t^2} \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{+\infty} \exp\left(-\frac{(x-\mu-\sigma^2 t)^2}{2\sigma^2}\right) dx.$$

Now notice that

$$\frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu-\sigma^2 t)^2}{2\sigma^2}\right)$$

is itself the density of a normal distribution with mean

$$\mu + \sigma^2 t$$

and variance

$$\sigma^2.$$

Since the integral of any probability density over its entire support is equal to 1,

$$\frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{+\infty} \exp\left(-\frac{(x-\mu-\sigma^2 t)^2}{2\sigma^2}\right) dx = 1.$$

Therefore,

$$\boxed{M_X(t) = e^{\mu t + \frac{1}{2}\sigma^2 t^2}}$$

24.5 Obtaining $\mathbb{E}[e^X]$

Recall the definition of the moment-generating function:

$$M_X(t) = \mathbb{E}[e^{tX}].$$

We are interested in

$$\mathbb{E}[e^X].$$

This is simply the MGF evaluated at

$$t = 1,$$

because

$$e^{tX}\Big|_{t=1} = e^X.$$

Therefore,

$$\mathbb{E}[e^X] = M_X(1).$$

Using

$$M_X(t) = e^{\mu t + \frac{1}{2}\sigma^2 t^2},$$

we obtain

$$M_X(1) = e^{\mu(1) + \frac{1}{2}\sigma^2 (1)^2}.$$

Hence,

$$\boxed{\mathbb{E}[e^X] = e^{\mu + \frac{1}{2}\sigma^2}}$$

24.6 Application to Log Returns

Suppose that the annual log return X follows

$$X \sim \mathcal{N}(0, 0.2^2).$$

Thus,

$$\mu = 0$$

and

$$\sigma = 0.2.$$

The expected log return is therefore

$$\mathbb{E}[X] = 0.$$

However, the gross simple return is

$$\frac{S_T}{S_0} = e^X.$$

Therefore,

$$\mathbb{E}\left[\frac{S_T}{S_0}\right] = \mathbb{E}[e^X].$$

Using the previous result,

$$\mathbb{E}[e^X] = e^{\mu + \frac{1}{2}\sigma^2}.$$

Since $\mu = 0$ and $\sigma = 0.2$,

$$\mathbb{E}[e^X] = e^{\frac{1}{2}(0.2)^2}.$$

Since

$$(0.2)^2 = 0.04,$$

we obtain

$$\mathbb{E}[e^X] = e^{0.02} \approx 1.0202.$$

This corresponds exactly to the Python expression

```
math.exp(0.2**2 / 2)
```

Thus,

$$\boxed{\mathbb{E}\left[\frac{S_T}{S_0}\right] \approx 1.0202}$$

This is the expected **gross return**.

The simple net return is

$$R = \frac{S_T - S_0}{S_0} = e^X - 1.$$

Therefore,

$$\mathbb{E}[R] = \mathbb{E}[e^X] - 1.$$

Hence,

$$\boxed{\mathbb{E}[R] = e^{\mu + \frac{1}{2}\sigma^2} - 1}$$

and in this example,

$$\mathbb{E}[R] = e^{0.02} - 1 \approx 0.0202.$$

Thus,

$$\boxed{\mathbb{E}[R] \approx 2.02\%}$$

24.7 Key Takeaway

If

$$X \sim \mathcal{N}(\mu, \sigma^2),$$

then

$$\boxed{M_X(t) = \mathbb{E}[e^{tX}] = e^{\mu t + \frac{1}{2}\sigma^2 t^2}}$$

and evaluating the MGF at $t = 1$ gives

$$\boxed{\mathbb{E}[e^X] = M_X(1) = e^{\mu + \frac{1}{2}\sigma^2}.}$$

Therefore, if X represents a normally distributed log return,

$$\boxed{\mathbb{E}[R_{\text{simple}}] = e^{\mu + \frac{1}{2}\sigma^2} - 1.}$$

In particular, when

$$\mu = 0, \qquad \sigma = 20\%,$$

we obtain

$$\boxed{\mathbb{E}[R_{\text{simple}}] = e^{0.2^2/2} - 1 \approx 2.02\%.}$$

25 Monte Carlo Simulation of a Stock Price

The function simulates many possible future stock-price paths by drawing random daily returns from a normal distribution.

The simulation can be summarized as follows:

$$\boxed{S_0 \longrightarrow S_1 \longrightarrow S_2 \longrightarrow \cdots \longrightarrow S_N}$$

where each transition from S_t to S_{t+1} depends on a newly generated random return.

Repeating this procedure M times produces

$$S_N^{(1)}, S_N^{(2)}, \ldots, S_N^{(M)},$$

which form an empirical Monte Carlo approximation of the distribution of the future stock price.

25.1 Model Inputs

Let

$$S_0$$

denote the initial stock price,

$$\sigma$$

the annualized volatility,

$$N$$

the number of trading days simulated, and

$$M$$

the number of independent Monte Carlo simulations.
In the example,

$$S_0 = 100, \qquad \sigma = 20\% = 0.20, \qquad N = 252, \qquad M = 10{,}000.$$

25.2 From Annual to Daily Volatility

The code first converts annual volatility into daily volatility:

```
volatilite_quotidienne = volatilite / math.sqrt(252)
```

The square-root-of-time rule follows from the additivity of variances.
Suppose daily returns are independent and identically distributed with

$$\mathrm{Var}(R_t) = \sigma_d^2.$$

The annual return, under an additive return approximation, is

$$R_{\text{year}} = \sum_{t=1}^{252} R_t.$$

If daily returns are independent,

$$\mathrm{Var}\left(\sum_{t=1}^{252} R_t\right) = \sum_{t=1}^{252} \mathrm{Var}(R_t).$$

Therefore,

$$\sigma^2 = 252\sigma_d^2.$$

Taking square roots,

$$\sigma = \sqrt{252}\sigma_d.$$

Hence,

$$\boxed{\sigma_d = \frac{\sigma}{\sqrt{252}}}$$

For an annual volatility of 20%,

$$\sigma_d = \frac{0.20}{\sqrt{252}} \approx 0.0126.$$

Thus, daily volatility is approximately

$$\boxed{\sigma_d \approx 1.26\%.}$$

25.3 Generating Random Daily Returns

At every trading day, the code executes

```
rendement = random.gauss(0, volatilite_quotidienne)
```

Mathematically, this means that a new random variable is drawn:

$$\boxed{R_t \sim \mathcal{N}(0, \sigma_d^2)}$$

or equivalently,

$$R_t = \sigma_d Z_t, \qquad Z_t \sim \mathcal{N}(0,1).$$

Therefore,

$$\mathbb{E}[R_t] = 0$$

and

$$\mathrm{Var}(R_t) = \sigma_d^2.$$

The simulation assumes that the random shocks are independent across time:

$$Z_1, Z_2, \ldots, Z_N \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,1).$$

25.4 Discrete Return Simulation

When

```
simul_type == "discrete"
```

the stock price is updated according to

```
prix *= (1 + rendement)
```

which corresponds mathematically to

$$\boxed{S_{t+1} = S_t(1 + R_{t+1})}$$

with

$$R_{t+1} \sim \mathcal{N}(0, \sigma_d^2).$$

After one day,

$$S_1 = S_0(1 + R_1).$$

After two days,

$$S_2 = S_1(1 + R_2) = S_0(1 + R_1)(1 + R_2).$$

After N days,

$$\boxed{S_N = S_0 \prod_{t=1}^{N} (1 + R_t)}$$

Thus, returns are compounded multiplicatively.

Expected Price in the Discrete Simulation

Because

$$\mathbb{E}[R_t] = 0,$$

we have

$$\mathbb{E}[1 + R_t] = 1.$$

Assuming independence,

$$\mathbb{E}[S_N] = S_0 \mathbb{E}\left[\prod_{t=1}^{N} (1 + R_t)\right]$$

becomes

$$\mathbb{E}[S_N] = S_0 \prod_{t=1}^{N} \mathbb{E}[1 + R_t].$$

Therefore,

$$\mathbb{E}[S_N] = S_0 \prod_{t=1}^{N} 1,$$

so

$$\boxed{\mathbb{E}[S_N] = S_0.}$$

Consequently, with

$$S_0 = 100,$$

the average terminal price across a sufficiently large number of simulations should be close to

$$\boxed{100.}$$

25.5 Continuous Return Simulation

When

```
simul_type == "continuous"
```

the code instead performs

```
prix *= math.exp(rendement)
```

which corresponds to

$$\boxed{S_{t+1} = S_t e^{R_{t+1}}.}$$

Here R_t should be interpreted as a continuously compounded (or log) return:

$$\boxed{R_{t+1} = \ln\left(\frac{S_{t+1}}{S_t}\right).}$$

Indeed,

$$S_{t+1} = S_t e^{R_{t+1}}$$

implies

$$\frac{S_{t+1}}{S_t} = e^{R_{t+1}},$$

and therefore

$$\ln\left(\frac{S_{t+1}}{S_t}\right) = R_{t+1}.$$

Compounding Over Multiple Days

After N days,

$$S_N = S_0 e^{R_1} e^{R_2} \cdots e^{R_N}.$$

Using

$$e^a e^b = e^{a+b},$$

we obtain

$$\boxed{S_N = S_0 \exp\left(\sum_{t=1}^{N} R_t\right).}$$

Since

$$R_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma_d^2),$$

the sum of the daily log returns is also normally distributed:

$$\sum_{t=1}^{N} R_t \sim \mathcal{N}\left(0, N\sigma_d^2\right).$$

For one year,

$$N = 252$$

and

$$\sigma_d^2 = \frac{\sigma^2}{252}.$$

Therefore,

$$N\sigma_d^2 = 252\frac{\sigma^2}{252} = \sigma^2.$$

Hence,

$$\boxed{\sum_{t=1}^{252} R_t \sim \mathcal{N}(0, \sigma^2).}$$

Consequently,

$$\boxed{\ln\left(\frac{S_{252}}{S_0}\right) \sim \mathcal{N}(0, \sigma^2).}$$

Equivalently,

$$\boxed{S_{252} = S_0 e^X, \qquad X \sim \mathcal{N}(0, \sigma^2).}$$

Thus, the terminal stock price follows a lognormal distribution.

25.6 Expected Price in the Continuous Simulation

This case has an important difference from the discrete simulation.

Recall that if

$$X \sim \mathcal{N}(\mu, v),$$

then

$$\boxed{\mathbb{E}[e^X] = e^{\mu + \frac{1}{2}v}.}$$

Here,

$$X = \sum_{t=1}^{N} R_t$$

has

$$\mu = 0$$

and

$$v = N\sigma_d^2.$$

Therefore,

$$\mathbb{E}[S_N] = S_0 \mathbb{E}[e^X]$$

gives

$$\boxed{\mathbb{E}[S_N] = S_0 e^{\frac{1}{2}N\sigma_d^2}.}$$

For $N = 252$,

$$N\sigma_d^2 = \sigma^2,$$

so

$$\boxed{\mathbb{E}[S_{252}] = S_0 e^{\frac{1}{2}\sigma^2}.}$$

With

$$S_0 = 100, \qquad \sigma = 0.20,$$

we obtain

$$\mathbb{E}[S_{252}] = 100e^{0.2^2/2}.$$

Thus,

$$\mathbb{E}[S_{252}] = 100e^{0.02} \approx 102.02.$$

Therefore,

$$\boxed{\mathbb{E}[S_{252}] \approx 102.02.}$$

25.7 Why Do the Two Simulations Have Different Expected Prices?

The discrete simulation uses

$$S_{t+1} = S_t(1 + R_{t+1}),$$

where R_t is interpreted as a **simple return**.

Since

$$\mathbb{E}[R_t] = 0,$$

we have

$$\mathbb{E}\left[\frac{S_{t+1}}{S_t}\right] = \mathbb{E}[1 + R_{t+1}] = 1.$$

The continuous simulation instead uses

$$S_{t+1} = S_t e^{R_{t+1}},$$

where R_t is interpreted as a **log return**.

Although

$$\mathbb{E}[R_t] = 0,$$

we have

$$\mathbb{E}[e^{R_t}] = e^{\frac{1}{2}\sigma_d^2} > 1.$$

Therefore,

Simulation	Price update	Expected annual terminal price
Discrete	$S_{t+1} = S_t(1 + R_{t+1})$	S_0
Continuous	$S_{t+1} = S_t e^{R_{t+1}}$	$S_0 e^{\sigma^2/2}$

25.8 Monte Carlo Estimation

The outer loop repeats the complete price-path simulation M times.

Let

$$S_N^{(m)}$$

denote the terminal price obtained in simulation m, where

$$m = 1, \ldots, M.$$

The code therefore produces the sample

$$\boxed{S_N^{(1)}, S_N^{(2)}, \ldots, S_N^{(M)}.}$$

The theoretical expected terminal price

$$\mathbb{E}[S_N]$$

can then be estimated using the sample average

$$\boxed{\widehat{\mathbb{E}[S_N]} = \frac{1}{M} \sum_{m=1}^{M} S_N^{(m)}.}$$

This corresponds to

```
sum(resultats) / len(resultats)
```

in the Python code.

By the Law of Large Numbers,

$$\boxed{\frac{1}{M} \sum_{m=1}^{M} S_N^{(m)} \longrightarrow \mathbb{E}[S_N] \qquad \text{as } M \to \infty.}$$

Thus, increasing the number of simulations makes the Monte Carlo estimate increasingly close to the theoretical expectation.

25.9 Empirical Distribution of Terminal Prices

Finally,

```
plt.hist(resultats, bins=40)
```

constructs a histogram of

$$S_N^{(1)}, \ldots, S_N^{(M)}.$$

The histogram is an empirical approximation of the probability distribution of the terminal stock price:

$$\boxed{\left\{S_N^{(m)}\right\}_{m=1}^{M} \quad \longrightarrow \quad \text{distribution of } S_N.}$$

Monte Carlo simulation therefore does more than estimate the expected price. It approximates the **entire distribution** of possible future prices, from which quantities such as

$$\mathbb{E}[S_N], \qquad \mathrm{Var}(S_N), \qquad \mathbb{P}(S_N < K),$$

or quantiles of S_N can be estimated.

25.10 Summary of the Algorithm

For each Monte Carlo simulation $m = 1, \ldots, M$:

$$\boxed{S_0^{(m)} = S_0}$$

Then, for each day $t = 0, \ldots, N-1$, generate

$$Z_{t+1}^{(m)} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,1)$$

and define

$$R_{t+1}^{(m)} = \frac{\sigma}{\sqrt{252}} Z_{t+1}^{(m)}.$$

For the discrete simulation,

$$\boxed{S_{t+1}^{(m)} = S_t^{(m)} \left(1 + R_{t+1}^{(m)}\right).}$$

For the continuous simulation implemented in the code,

$$\boxed{S_{t+1}^{(m)} = S_t^{(m)} e^{R_{t+1}^{(m)}}.}$$

After N days, store

$$S_N^{(m)}.$$

After M independent simulations, estimate the expected terminal price using

$$\boxed{\widehat{\mathbb{E}[S_N]} = \frac{1}{M}\sum_{m=1}^{M} S_N^{(m)}.}$$

For the parameters used in the example,

$$S_0 = 100, \qquad \sigma = 20\%, \qquad N = 252,$$

the theoretical expectations are

Discrete simulation:	$\mathbb{E}[S_{252}] = 100,$
Continuous simulation in the code:	$\mathbb{E}[S_{252}] = 100e^{0.2^2/2} \approx 102.02.$

25.11 Useful Python Methods and Operators

The Vigenère implementation uses several Python methods and operators that are worth understanding in detail.

25.11.1 `.isalpha()`

The string method

```
char.isalpha()
```

checks whether all characters in a string are **alphabetic characters**.

It returns a Boolean value:

`True` or `False`.

For example:

```
print("a".isalpha())   # True
print("Z".isalpha())   # True
print("7".isalpha())   # False
print(" ".isalpha())   # False
print("!".isalpha())   # False
```

Therefore, in

```
if char.isalpha():
```

the code asks:

"Is char a letter?"

If the answer is `True`, the character is encrypted.

Otherwise, it is left unchanged.

For example, for

```
text = "Hello World!"
```

the characters

$$H, e, l, l, o, W, o, r, l, d$$

satisfy `isalpha()`, whereas

space and !

do not.

Important remark. `isalpha()` is not restricted to the 26 English letters.

For example,

```
"é".isalpha()   # True
"ç".isalpha()   # True
```

because Python recognizes Unicode alphabetic characters.

However, our Vigenère implementation uses arithmetic based on the 26-letter Latin alphabet:

$$A, \ldots, Z \qquad \text{or} \qquad a, \ldots, z.$$

Therefore, if accented or non-Latin characters are possible, a stricter test may be preferable.

25.11.2 `.isupper()`

The method

```
char.isupper()
```

checks whether a character is uppercase.

For example,

```
"A".isupper()   # True
"Z".isupper()   # True
"a".isupper()   # False
"z".isupper()   # False
```

In the Vigenère cipher, it allows us to preserve the capitalization of the original text.

For example,

```
if char.isupper():
    base = ord("A")
else:
    base = ord("a")
```

If

$$\texttt{char = "B"},$$

then

$$\texttt{char.isupper()} = \texttt{True},$$

so

$$\texttt{base} = \text{ord}(\texttt{"A"}) = 65.$$

If instead

$$\texttt{char = "b"},$$

then

$$\texttt{char.isupper()} = \texttt{False},$$

so

$$\texttt{base} = \text{ord}(\texttt{"a"}) = 97.$$

This allows uppercase letters to remain uppercase and lowercase letters to remain lowercase after encryption.

25.11.3 The Modulo Operator %

The operator

```
a % b
```

is called the **modulo operator**.

It returns the remainder of the Euclidean division of a by b.

Mathematically, if

$$a = qb + r,$$

where q is the quotient and r is the remainder, then

$$\boxed{a \bmod b = r.}$$

For example,

$$7 = 2 \times 3 + 1,$$

so

$$\boxed{7 \bmod 3 = 1.}$$

In Python:

```
7 % 3
```

returns

```
1
```

Other examples are

$$\begin{aligned} 5 \bmod 4 &= 1, \\ 6 \bmod 4 &= 2, \\ 7 \bmod 4 &= 3, \\ 8 \bmod 4 &= 0, \\ 9 \bmod 4 &= 1. \end{aligned}$$

Notice that the values repeat periodically:

$$\boxed{0, 1, 2, 3, 0, 1, 2, 3, \ldots}$$

when taking modulo 4.

25.11.4 Using Modulo to Repeat the Key

Suppose the key is

```
key = "clef"
```

It contains four letters, so

$$m = 4.$$

The key index can be computed as

`key_index % len(key)`

Since

$$\text{len}(\texttt{"clef"}) = 4,$$

we obtain

key_index	key_index mod 4	Key letter
0	0	c
1	1	l
2	2	e
3	3	f
4	0	c
5	1	l
6	2	e
7	3	f
8	0	c

Thus,

$$\boxed{\texttt{key_index \% len(key)}}$$

automatically cycles through the positions

$$0, 1, 2, 3, 0, 1, 2, 3, \ldots$$

and therefore repeats the key:

$$\boxed{c, l, e, f, c, l, e, f, c, l, e, f, \ldots}$$

25.11.5 Using Modulo to Wrap Around the Alphabet

Modulo is also used in the encryption formula:

`(ord(char) - base + shift) % 26`

The number 26 appears because the English alphabet contains 26 letters.

We represent letters by

$$A \to 0, \quad B \to 1, \quad \ldots, \quad Z \to 25.$$

Suppose we want to shift the letter Y by 5 positions.

Since

$$Y \to 24,$$

we first calculate

$$24 + 5 = 29.$$

But 29 is outside the range

$$\{0, \ldots, 25\}.$$

Using modulo 26,

$$29 \bmod 26 = 3.$$

Since

$$3 \to D,$$

we obtain

$$\boxed{Y \xrightarrow{+5} D.}$$

Thus, modulo makes the alphabet **cyclic**:

$$\boxed{\cdots \to X \to Y \to Z \to A \to B \to C \to \cdots}$$

The Vigenère transformation can therefore be written mathematically as

$$\boxed{y_i = (x_i + k_i) \bmod 26,}$$

where

$$x_i \in \{0, \ldots, 25\}$$

is the numerical representation of the original letter and

$$k_i \in \{0, \ldots, 25\}$$

is the shift associated with the corresponding key letter.

25.11.6 Summary

Python	Purpose	Example
`char.isalpha()`	Checks whether char is alphabetic	`"A"` → `True`
`char.isupper()`	Checks whether char is uppercase	`"A"` → `True`
`a % b`	Returns the remainder of a divided by b	$7 \bmod 3 = 1$

In the Vigenère cipher, modulo has two particularly important roles:

`key_index % len(key)`	$\rightarrow$	repeat the key cyclically,
`(... + shift) % 26`	$\rightarrow$	wrap around the alphabet.

26 Gradient Descent

26.1 Definition

Gradient descent is an iterative first-order numerical optimization algorithm used to minimize a differentiable objective function. At each iteration, it moves the parameters in the direction opposite to the gradient.

Let

$$f : \mathbb{R}^p \to \mathbb{R}$$

be a differentiable objective function that we want to minimize. The optimization problem is

$$\boxed{\beta^\star = \arg\min_{\beta \in \mathbb{R}^p} f(\beta)}$$

where

$$\beta = \begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix} \in \mathbb{R}^p$$

is the vector of parameters to optimize.

The objective of gradient descent is therefore to construct a sequence

$$\beta_0, \beta_1, \beta_2, \ldots$$

that progressively reduces the value of the objective function.

Importantly, whether this sequence converges to a local minimum, a global minimum, or merely toward a stationary point depends on additional properties of the objective function f.

26.2 The Gradient

The gradient of f at β is the vector containing all its first-order partial derivatives:

$$\boxed{\nabla f(\beta) = \begin{pmatrix} \frac{\partial f(\beta)}{\partial \beta_1} \\ \frac{\partial f(\beta)}{\partial \beta_2} \\ \vdots \\ \frac{\partial f(\beta)}{\partial \beta_p} \end{pmatrix} \in \mathbb{R}^p.}$$

The gradient indicates the direction of the **steepest local increase** of the function with respect to the Euclidean norm.

Therefore,

$$\boxed{-\nabla f(\beta)}$$

indicates the direction of the **steepest local decrease**.

This is the fundamental idea behind gradient descent: at each iteration, we move the parameters in the direction opposite to the gradient.

26.3 Why Does the Negative Gradient Decrease the Function?

Consider a small change

$$\Delta\beta \in \mathbb{R}^p.$$

Reminder: Taylor Expansion

Let

$$f : \mathbb{R} \to \mathbb{R}$$

be a sufficiently differentiable function. Around a point a, the Taylor expansion of f is

$$\boxed{f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f^{(3)}(a)}{3!}(x-a)^3 + \cdots}$$

or, more generally,

$$\boxed{f(x) = \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k + R_n(x),}$$

where $R_n(x)$ is the remainder term.

The first terms have a simple interpretation:

Term	Information captured
$f(a)$	Value of the function at a
$f'(a)(x-a)$	Local slope
$\frac{1}{2}f''(a)(x-a)^2$	Local curvature
Higher-order terms	Finer local behavior

If x is sufficiently close to a, the higher-order terms become small. Keeping only the first-order term gives the **first-order Taylor approximation**:

$$\boxed{f(x) \approx f(a) + f'(a)(x-a).}$$

If we write

$$x = a + \Delta x,$$

then

$$x - a = \Delta x,$$

and therefore

$$\boxed{f(a + \Delta x) \approx f(a) + f'(a)\Delta x.}$$

This says that, locally, a differentiable function behaves approximately like a straight line.

Multivariate case. For gradient descent, we generally consider

$$f : \mathbb{R}^p \to \mathbb{R},$$

with

$$\beta = \begin{pmatrix} \beta_1 \\ \vdots \\ \beta_p \end{pmatrix}.$$

We consider a small displacement

$$\Delta\beta = \begin{pmatrix} \Delta\beta_1 \\ \vdots \\ \Delta\beta_p \end{pmatrix}.$$

The multivariate Taylor expansion around β is

$$\boxed{f(\beta + \Delta\beta) = f(\beta) + \nabla f(\beta)^\top \Delta\beta + \frac{1}{2}\Delta\beta^\top H_f(\beta)\Delta\beta + \cdots}$$

where

$$\nabla f(\beta) = \begin{pmatrix} \frac{\partial f}{\partial \beta_1} \\ \vdots \\ \frac{\partial f}{\partial \beta_p} \end{pmatrix}$$

is the gradient, and

$$H_f(\beta) = \nabla^2 f(\beta) = \begin{pmatrix} \frac{\partial^2 f}{\partial \beta_1^2} & \cdots & \frac{\partial^2 f}{\partial \beta_1 \partial \beta_p} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial \beta_p \partial \beta_1} & \cdots & \frac{\partial^2 f}{\partial \beta_p^2} \end{pmatrix}$$

is the Hessian matrix.

The dimensions are

$$\underbrace{\nabla f(\beta)^\top}_{1\times p}\underbrace{\Delta\beta}_{p\times 1}\in\mathbb{R},$$

and

$$\underbrace{\Delta\beta^\top}_{1\times p}\underbrace{H_f(\beta)}_{p\times p}\underbrace{\Delta\beta}_{p\times 1}\in\mathbb{R}.$$

If $\Delta\beta$ is small, we can neglect the second- and higher-order terms:

$$\boxed{f(\beta+\Delta\beta)\approx f(\beta)+\nabla f(\beta)^\top\Delta\beta.}$$

Application to gradient descent. Gradient descent chooses

$$\boxed{\Delta\beta=-\gamma\nabla f(\beta),\qquad \gamma>0.}$$

Substituting this displacement into the first-order Taylor approximation,

$$f(\beta+\Delta\beta)\approx f(\beta)+\nabla f(\beta)^\top\Delta\beta,$$

gives

$$f\left(\beta-\gamma\nabla f(\beta)\right)\approx f(\beta)+\nabla f(\beta)^\top\left(-\gamma\nabla f(\beta)\right).$$

Therefore,

$$f\left(\beta-\gamma\nabla f(\beta)\right)\approx f(\beta)-\gamma\nabla f(\beta)^\top\nabla f(\beta).$$

Euclidean Norm of the Gradient The notation

$$\|\nabla f(\beta)\|_2$$

denotes the **Euclidean norm**, also called the L^2-norm, of the gradient vector.

More generally, for a vector

$$x=\begin{pmatrix}x_1\\x_2\\\vdots\\x_p\end{pmatrix}\in\mathbb{R}^p,$$

its Euclidean norm is defined as

$$\boxed{\|x\|_2 = \sqrt{\sum_{j=1}^{p} x_j^2}}$$

Therefore, the Euclidean norm of the gradient is

$$\boxed{\|\nabla f(\beta)\|_2 = \sqrt{\sum_{j=1}^{p} \left(\frac{\partial f}{\partial \beta_j}\right)^2}}$$

and its squared norm is

$$\boxed{\|\nabla f(\beta)\|_2^2 = \sum_{j=1}^{p} \left(\frac{\partial f}{\partial \beta_j}\right)^2.}$$

Now, the inner product of a vector with itself is equal to the sum of the squares of its components. Hence,

$$\nabla f(\beta)^\top \nabla f(\beta) = \begin{pmatrix} \frac{\partial f}{\partial \beta_1} & \frac{\partial f}{\partial \beta_2} & \cdots & \frac{\partial f}{\partial \beta_p} \end{pmatrix} \begin{pmatrix} \frac{\partial f}{\partial \beta_1} \\ \frac{\partial f}{\partial \beta_2} \\ \vdots \\ \frac{\partial f}{\partial \beta_p} \end{pmatrix}.$$

Carrying out the matrix multiplication gives

$$\nabla f(\beta)^\top \nabla f(\beta) = \left(\frac{\partial f}{\partial \beta_1}\right)^2 + \left(\frac{\partial f}{\partial \beta_2}\right)^2 + \cdots + \left(\frac{\partial f}{\partial \beta_p}\right)^2.$$

Therefore,

$$\boxed{\nabla f(\beta)^\top \nabla f(\beta) = \|\nabla f(\beta)\|_2^2.}$$

The quantity

$$\|\nabla f(\beta)\|_2$$

can be interpreted geometrically as the **length or magnitude of the gradient vector**. It measures how strongly the function f changes locally around the point β.

Since

$$\nabla f(\beta)^\top \nabla f(\beta) = \|\nabla f(\beta)\|_2^2,$$

we obtain

$$\boxed{f\left(\beta - \gamma \nabla f(\beta)\right) \approx f(\beta) - \gamma \|\nabla f(\beta)\|_2^2.}$$

Since

$$\gamma > 0 \qquad \text{and} \qquad \|\nabla f(\beta)\|_2^2 \geq 0,$$

the second term is non-positive. Thus, for a sufficiently small step,

$$\boxed{f\left(\beta - \gamma \nabla f(\beta)\right) \lesssim f(\beta).}$$

This provides the local mathematical justification for moving in the direction of the negative gradient.

Local Intuition versus a Rigorous Descent Guarantee

It is important to distinguish the previous Taylor argument from a formal convergence result.

The first-order Taylor relation

$$f(\beta - \gamma \nabla f(\beta)) \approx f(\beta) - \gamma \|\nabla f(\beta)\|_2^2$$

is an **approximation**. It explains why the negative gradient is locally a natural descent direction, but the approximation by itself does not prove that every gradient-descent iteration decreases f.

A rigorous inequality can be obtained under an additional smoothness assumption: the gradient of f must be Lipschitz continuous.

As shown later, if ∇f is L-Lipschitz, then the **descent lemma** gives

$$\boxed{f(\beta - \gamma \nabla f(\beta)) \leq f(\beta) - \gamma \left(1 - \frac{L\gamma}{2}\right) \|\nabla f(\beta)\|_2^2.}$$

Thus,

$$\boxed{\text{Taylor expansion} \quad \Longrightarrow \quad \text{local intuition}}$$

whereas

$$\boxed{L\text{-Lipschitz gradient + appropriate step size} \quad \Longrightarrow \quad \text{rigorous descent guarantee.}}$$

26.4 Gradient Descent Update Rule

We begin from an initial parameter vector

$$\boxed{\beta_0 \in \mathbb{R}^p.}$$

At iteration t, we compute the gradient

$$\nabla f(\beta_t)$$

and update the parameters according to

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t \nabla f(\beta_t).}$$

The dimensions are

$$\underbrace{\beta_{t+1}}_{p\times 1} = \underbrace{\beta_t}_{p\times 1} - \underbrace{\gamma_t}_{1\times 1}\underbrace{\nabla f(\beta_t)}_{p\times 1}.$$

The algorithm therefore generates a sequence

$$\beta_0, \beta_1, \beta_2, \ldots, \beta_t, \ldots$$

The behavior of this sequence depends on properties of f and on the choice of the learning rate.

It is useful to distinguish three different questions:

1. **Does each iteration decrease the objective?**

   Under an L-Lipschitz gradient and an appropriate learning rate, gradient descent satisfies a descent inequality.

2. **Do the gradients approach zero?**

   If the objective is additionally bounded below, the descent inequality can be used to show that

   $$\|\nabla f(\beta_t)\|_2 \to 0.$$

3. **Does a stationary point correspond to a minimum?**

   This depends on the shape of the objective. In particular, convexity guarantees that every stationary point is a global minimum.

We now study these conditions in detail.

26.5 Conditions for Gradient Descent

26.5.1 Differentiability

The objective function must first be differentiable:

$$f : \mathbb{R}^p \to \mathbb{R}.$$

Definition of Differentiability A function $f : \mathbb{R}^p \to \mathbb{R}$ is **differentiable at** $\beta \in \mathbb{R}^p$ if there exists a linear map, represented by the gradient $\nabla f(\beta)$, such that

$$\boxed{f(\beta + h) = f(\beta) + \nabla f(\beta)^\top h + o(\|h\|_2)}$$

as

$$h \to 0.$$

Here,

$$h = \begin{pmatrix} h_1 \\ \vdots \\ h_p \end{pmatrix} \in \mathbb{R}^p$$

represents a small change, or perturbation, in the parameter vector β.

Thus,

$$\beta + h$$

represents a point close to β.

The notation

$$o(\|h\|_2)$$

represents an error term that becomes negligible **faster than** $\|h\|_2$ as $h \to 0$.

Formally, if the remainder is denoted by $R(h)$, writing

$$\boxed{R(h) = o(\|h\|_2)}$$

means that

$$\boxed{\lim_{\|h\|_2 \to 0} \frac{R(h)}{\|h\|_2} = 0.}$$

In other words, the remainder $R(h)$ becomes negligible relative to $\|h\|_2$ when h approaches zero.

Equivalently,

$$\boxed{\lim_{\|h\|_2 \to 0} \frac{f(\beta + h) - f(\beta) - \nabla f(\beta)^\top h}{\|h\|_2} = 0.}$$

This means that, sufficiently close to β, the function can be approximated by the affine function

$$f(\beta + h) \approx f(\beta) + \nabla f(\beta)^\top h.$$

Thus, differentiability means that f admits a **first-order linear approximation** around β.

How to Show Differentiability in Practice A common sufficient condition is to show that all first-order partial derivatives exist and are continuous in a neighborhood of the point.

That is, if

$$\frac{\partial f}{\partial \beta_j}$$

exists and is continuous for every

$$j = 1, \ldots, p,$$

then

$$\boxed{f \in C^1 \quad \Longrightarrow \quad f \text{ is differentiable.}}$$

Therefore, in most optimization problems, one can proceed as follows:

1. Compute every first-order partial derivative:

$$\frac{\partial f}{\partial \beta_1}, \ldots, \frac{\partial f}{\partial \beta_p}.$$

2. Verify that these derivatives exist on the domain of interest.
3. Verify that these partial derivatives are continuous.
4. Construct the gradient:

$$\nabla f(\beta) = \begin{pmatrix} \partial f/\partial \beta_1 \\ \vdots \\ \partial f/\partial \beta_p \end{pmatrix}.$$

How to Verify That a Partial Derivative Exists The partial derivative with respect to β_j at a point β_0 is formally defined by the limit

$$\boxed{\frac{\partial f(\beta_0)}{\partial \beta_j} = \lim_{h \to 0} \frac{f(\beta_0 + h e_j) - f(\beta_0)}{h},}$$

where

$$e_j = \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix}$$

is the j-th canonical basis vector of $\mathbb{R}^p$.

The 1 appears in position j.

Therefore,

$$\beta_0 + he_j$$

changes only the j-th coordinate of β_0.

The partial derivative exists at β_0 if the limit

$$\lim_{h \to 0} \frac{f(\beta_0 + he_j) - f(\beta_0)}{h}$$

exists and is finite.

In practice, for standard functions such as polynomials, exponentials, logarithms on their domains, and trigonometric functions, the usual differentiation rules can be used directly.

It is important to note that the existence of all partial derivatives alone does **not** generally imply differentiability of a multivariate function.

The continuity of these partial derivatives provides a convenient sufficient condition.

How to Verify That a Partial Derivative Is Continuous Suppose we compute the j-th partial derivative of f:

$$g_j(\beta) = \frac{\partial f(\beta)}{\partial \beta_j}.$$

We then need to verify that g_j is a **continuous function of** β.

Formally, g_j is continuous at a point

$$\beta_0 \in \mathbb{R}^p$$

if

$$\boxed{\lim_{\beta \to \beta_0} g_j(\beta) = g_j(\beta_0).}$$

Since

$$g_j(\beta) = \frac{\partial f(\beta)}{\partial \beta_j},$$

this condition can equivalently be written as

$$\boxed{\lim_{\beta \to \beta_0} \frac{\partial f(\beta)}{\partial \beta_j} = \frac{\partial f(\beta_0)}{\partial \beta_j}.}$$

Therefore, to show that all first-order partial derivatives are continuous, we must verify that, for every $j = 1, \ldots, p$,

$$\boxed{\lim_{\beta \to \beta_0} \frac{\partial f(\beta)}{\partial \beta_j} = \frac{\partial f(\beta_0)}{\partial \beta_j}.}$$

If this property holds for every β_0 in the domain and for every $j = 1, \ldots, p$, then the gradient

$$\nabla f(\beta) = \begin{pmatrix} \frac{\partial f}{\partial \beta_1} \\ \vdots \\ \frac{\partial f}{\partial \beta_p} \end{pmatrix}$$

is continuous, and we write

$$\boxed{f \in C^1.}$$

In particular,

$$\boxed{f \in C^1 \quad \Longrightarrow \quad f \text{ is differentiable.}}$$

It is important not to confuse continuity and differentiability.

In general,

$$\boxed{f \text{ differentiable} \quad \Longrightarrow \quad f \text{ continuous,}}$$

but

$$\boxed{f \text{ continuous} \quad \Longrightarrow \quad f \text{ differentiable.}}$$

For example,

$$f(x) = |x|$$

is continuous at $x = 0$, but is not differentiable at $x = 0$.

If the first-order partial derivatives exist and are continuous, then f is continuously differentiable and therefore differentiable.

26.5.2 Lipschitz-Continuous Gradient

A second important condition concerns how quickly the gradient can change.

There must exist a constant

$$L > 0$$

such that

$$\boxed{\|\nabla f(x) - \nabla f(y)\|_2 \leq L \|x - y\|_2}$$

for all relevant x, y.

A function satisfying this condition is often called L-**smooth**.

Intuitively, the gradient cannot change arbitrarily fast.

The constant L provides an upper bound on how rapidly the slope of the objective can vary as the parameters change.

Method 1: Direct Verification The most direct approach is to compute

$$\nabla f(x) - \nabla f(y)$$

and find a constant $L > 0$ such that

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2.$$

If the same finite constant L works for every x and y in the domain of interest, then ∇f is L-Lipschitz.

Method 2: Using the Hessian For a twice continuously differentiable function

$$f : \mathbb{R}^p \to \mathbb{R},$$

the most common practical method is to study the Hessian matrix

$$\nabla^2 f(\beta).$$

If there exists a finite constant $L > 0$ such that

$$\boxed{\|\nabla^2 f(\beta)\|_2 \leq L}$$

for every β in the relevant domain, then the gradient is L-Lipschitz:

$$\boxed{\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2.}$$

Practical Procedure In practice, to show that ∇f is Lipschitz continuous:

1. Compute the gradient:

$$\nabla f(\beta).$$

2. Compute the Hessian:

$$\nabla^2 f(\beta).$$

3. Find an upper bound on its spectral norm:

$$\|\nabla^2 f(\beta)\|_2 \leq L.$$

4. Verify that the same finite L works throughout the relevant domain.

5. Conclude that

$$\boxed{\nabla f \text{ is } L\text{-Lipschitz.}}$$

Thus, a convenient sufficient condition is

$$\boxed{\sup_{\beta} \|\nabla^2 f(\beta)\|_2 < \infty \quad \Longrightarrow \quad \nabla f \text{ is Lipschitz continuous.}}$$

The domain is important. A Hessian may be bounded on a restricted or compact domain even if it is not bounded over all of $\mathbb{R}^p$.

26.5.3 Appropriate Learning Rate

The learning rate is not a regularity condition on f. It is a parameter of the gradient-descent algorithm.

Suppose that ∇f is L-Lipschitz.

For a constant learning rate

$$\gamma_t = \gamma,$$

a standard step-size condition is

$$\boxed{0 < \gamma < \frac{2}{L}.}$$

A common more conservative choice is

$$\boxed{0 < \gamma \leq \frac{1}{L}.}$$

The Descent Lemma If the gradient of f is L-Lipschitz, then for any x, y,

$$\boxed{f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2}\|y - x\|_2^2.}$$

For gradient descent,

$$y = x - \gamma \nabla f(x).$$

Therefore,

$$y - x = -\gamma \nabla f(x).$$

Substituting into the descent lemma gives

$$f(x - \gamma \nabla f(x)) \leq f(x) - \gamma \nabla f(x)^\top \nabla f(x) + \frac{L}{2}\gamma^2 \|\nabla f(x)\|_2^2.$$

Since

$$\nabla f(x)^\top \nabla f(x) = \|\nabla f(x)\|_2^2,$$

we obtain

$$f(x - \gamma \nabla f(x)) \leq f(x) - \gamma \|\nabla f(x)\|_2^2 + \frac{L\gamma^2}{2}\|\nabla f(x)\|_2^2.$$

Factorizing gives

$$\boxed{f(x - \gamma \nabla f(x)) \leq f(x) - \gamma \left(1 - \frac{L\gamma}{2}\right) \|\nabla f(x)\|_2^2.}$$

At iteration k,

$$\boxed{f(\beta_{k+1}) \leq f(\beta_k) - \gamma \left(1 - \frac{L\gamma}{2}\right) \|\nabla f(\beta_k)\|_2^2.}$$

If

$$0 < \gamma < \frac{2}{L},$$

then

$$1 - \frac{L\gamma}{2} > 0.$$

Consequently, whenever

$$\nabla f(\beta_k) \neq 0,$$

we obtain

$$\boxed{f(\beta_{k+1}) < f(\beta_k).}$$

Thus, each gradient-descent step strictly decreases the objective whenever the gradient is non-zero.

26.5.4 Boundedness Below

Showing that the objective decreases is not, by itself, sufficient to show that the gradient approaches zero.

We also impose the standard assumption that f is **bounded below**.

That is, there exists some finite value

$$f_{\inf} > -\infty$$

such that

$$\boxed{f(\beta) \geq f_{\inf} \qquad \forall \beta.}$$

This prevents gradient descent from decreasing the objective indefinitely toward $-\infty$.

Define

$$c = \gamma \left(1 - \frac{L\gamma}{2}\right).$$

When

$$0 < \gamma < \frac{2}{L},$$

we have

$$c > 0.$$

The descent inequality becomes

$$f(\beta_{k+1}) \leq f(\beta_k) - c\|\nabla f(\beta_k)\|_2^2.$$

Rearranging,

$$\boxed{c\|\nabla f(\beta_k)\|_2^2 \leq f(\beta_k) - f(\beta_{k+1}).}$$

Summing from

$$k = 0$$

to

$$k = K - 1$$

gives

$$c \sum_{k=0}^{K-1} \|\nabla f(\beta_k)\|_2^2 \leq \sum_{k=0}^{K-1} \left[f(\beta_k) - f(\beta_{k+1})\right].$$

The right-hand side is a telescoping sum:

$$\sum_{k=0}^{K-1} [f(\beta_k) - f(\beta_{k+1})] = f(\beta_0) - f(\beta_K).$$

Therefore,

$$c \sum_{k=0}^{K-1} \|\nabla f(\beta_k)\|_2^2 \leq f(\beta_0) - f(\beta_K).$$

Since

$$f(\beta_K) \geq f_{\inf},$$

we obtain

$$c \sum_{k=0}^{K-1} \|\nabla f(\beta_k)\|_2^2 \leq f(\beta_0) - f_{\inf}.$$

Letting

$$K \to \infty,$$

we obtain

$$\boxed{\sum_{k=0}^{\infty} \|\nabla f(\beta_k)\|_2^2 < \infty.}$$

A convergent series of non-negative terms must have terms converging to zero. Therefore,

$$\boxed{\|\nabla f(\beta_k)\|_2^2 \to 0,}$$

and consequently,

$$\boxed{\|\nabla f(\beta_k)\|_2 \to 0.}$$

This is the standard stationarity guarantee for smooth gradient descent.

26.6 Convergence Toward Stationarity

A point $\beta^\star$ is called a **stationary point** if

$$\boxed{\nabla f(\beta^\star) = 0.}$$

Under the assumptions developed above,

$$\boxed{\begin{array}{c} \nabla f \text{ is } L\text{-Lipschitz} \\ + \\ 0 < \gamma < 2/L \\ + \\ f \text{ is bounded below} \end{array}}$$

we obtain

$$\boxed{\|\nabla f(\beta_k)\|_2 \longrightarrow 0.}$$

Thus, gradient descent approaches **stationarity** in the sense that the gradient norm becomes arbitrarily small.

Gradient Convergence versus Parameter Convergence

An important distinction must be made between

$$\boxed{\|\nabla f(\beta_k)\|_2 \to 0}$$

and

$$\boxed{\beta_k \to \beta^\star.}$$

The first statement says that the sequence becomes increasingly **stationary**: the local slope approaches zero.

The second statement says that the parameter sequence itself converges to one particular point.

In a general non-convex optimization problem,

$$\|\nabla f(\beta_k)\|_2 \to 0$$

does not, by itself, automatically imply that the entire sequence $\{\beta_k\}$ converges to one particular stationary point.

Additional assumptions may be required for such a conclusion.

Therefore, a precise general statement is:

> Under standard smoothness, lower-boundedness, and step-size assumptions, gradient descent drives the gradient norm toward zero. Stronger assumptions are required to characterize the point toward which the parameter sequence converges.

26.7 When Does It Converge to a Minimum?

The answer depends crucially on the shape of f.

A stationary point satisfies

$$\nabla f(\beta^\star) = 0.$$

For a general differentiable function, this condition alone does not imply that $\beta^\star$ is a minimum.

A stationary point may correspond to

- a local minimum,

- a local maximum,
- or a saddle point.

Convexity changes this conclusion fundamentally.

26.7.1 Convexity

A differentiable function f is convex if

$$\boxed{f(y) \geq f(x) + \nabla f(x)^\top (y - x)}$$

for all x, y in its domain.

Geometrically, the graph of a differentiable convex function always lies above its tangent hyperplanes.

Suppose now that $\beta^\star$ is stationary:

$$\nabla f(\beta^\star) = 0.$$

Applying the convexity inequality with

$$x = \beta^\star$$

gives

$$f(y) \geq f(\beta^\star) + \nabla f(\beta^\star)^\top (y - \beta^\star).$$

Since

$$\nabla f(\beta^\star) = 0,$$

we obtain

$$f(y) \geq f(\beta^\star) \qquad \forall y.$$

Therefore,

$$\boxed{\nabla f(\beta^\star) = 0 \quad + \quad f \text{ convex} \quad \Longrightarrow \quad \beta^\star \in \arg\min_{\beta} f(\beta).}$$

Thus, for a differentiable convex objective,

$$\boxed{\text{stationary point} \quad \Longrightarrow \quad \text{global minimum.}}$$

This is one of the fundamental reasons why convex optimization problems are particularly attractive.

26.7.2 Strong Convexity

A stronger assumption is **strong convexity**.

A differentiable function f is μ-strongly convex if there exists

$$\mu > 0$$

such that

$$\boxed{f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2}\|y - x\|_2^2}$$

for all x, y.

The additional quadratic term

$$\frac{\mu}{2}\|y - x\|_2^2$$

forces the function to have a minimum amount of curvature.

Strong convexity implies convexity, but the converse is not necessarily true:

$$\boxed{\text{strong convexity} \quad \Longrightarrow \quad \text{convexity.}}$$

If f is strongly convex, its global minimizer is unique.

Therefore,

$$\boxed{\text{strong convexity} \quad \Longrightarrow \quad \text{unique global minimizer.}}$$

If f is both L-smooth and μ-strongly convex, gradient descent with an appropriate constant learning rate converges **linearly**, also called geometrically, toward the unique minimizer.

This means that the optimization error decreases proportionally at each iteration rather than merely approaching zero asymptotically.

26.7.3 The Non-Convex Case

If f is non-convex, several stationary points may exist.

These can include local minima, local maxima, and saddle points.

Gradient descent is therefore not generally guaranteed to converge to a global minimum.

The solution reached can depend on the initialization

$$\beta_0.$$

Consequently,

$$\boxed{\text{non-convexity} \quad \Longrightarrow \quad \text{stationarity does not imply global optimality.}}$$

This distinction is particularly important in machine learning, where many objective functions are non-convex.

26.8 Summary of the Convergence Conditions

The assumptions introduced above play different roles.

$$\boxed{\begin{array}{c} f \text{ differentiable} \\ + \\ \nabla f \text{ is } L\text{-Lipschitz} \\ + \\ 0 < \gamma < 2/L \\ \Downarrow \\ f(\beta_{k+1}) \leq f(\beta_k) \\ \text{(descent)} \end{array}}$$

If we additionally assume that f is bounded below,

$$\boxed{\begin{array}{c} \text{descent conditions} \\ + \\ f(\beta) \geq f_{\text{inf}} > -\infty \\ \Downarrow \\ \|\nabla f(\beta_k)\|_2 \to 0 \\ \text{(stationarity)} \end{array}}$$

If f is additionally convex,

$$\boxed{\begin{array}{c} \text{stationarity} \\ + \\ f \text{ convex} \\ \Downarrow \\ \text{global minimum} \end{array}}$$

Finally, if f is strongly convex,

$$\boxed{\begin{array}{c} f \text{ strongly convex} \\ + \\ \nabla f \text{ is } L\text{-Lipschitz} \\ + \\ \text{appropriate learning rate} \\ \Downarrow \\ \text{unique global minimum} \\ + \\ \text{linear convergence} \end{array}}$$

Hence, the different assumptions should not be confused:

Assumption	Main role
Differentiability	Makes the gradient well defined
L-Lipschitz gradient	Controls how rapidly the gradient changes
$0 < \gamma < 2/L$	Provides an appropriate gradient-descent step size
f bounded below	Prevents indefinite descent toward $-\infty$
Convexity	Makes stationary points global minima
Strong convexity	Gives a unique minimum and stronger convergence

26.9 The Step Size

The parameter

$$\gamma_t > 0$$

determines how far the algorithm moves in the direction of the negative gradient at iteration t.

A constant learning rate can be used:

$$\gamma_t = \gamma.$$

For an L-smooth objective, we have seen that a standard constant step-size condition is

$$0 < \gamma < \frac{2}{L},$$

with the frequently used conservative condition

$$0 < \gamma \leq \frac{1}{L}.$$

Alternatively, the learning rate can decrease with the number of iterations. For example,

$$\boxed{\gamma_t = \frac{c}{\sqrt{t}}}$$

for $t \geq 1$, where $c > 0$ controls the initial magnitude of the steps.

The intuition is that early iterations can make relatively large moves, while later iterations make smaller adjustments.

In practice, the choice of γ_t is crucial:

Step size	Possible consequence
Too small	Very slow progress
Appropriate	Stable descent and convergence under suitable assumptions
Too large	Oscillation or divergence

The precise theoretical conditions depend on whether the learning rate is constant or time-varying. The condition

$$0 < \gamma < \frac{2}{L}$$

derived above applies to the constant-step-size case.

26.10 Stopping Criterion

At a differentiable local minimum $\beta^\star$, a necessary first-order condition is

$$\boxed{\nabla f(\beta^\star) = 0.}$$

However, the converse is not generally true:

$$\nabla f(\beta^\star) = 0$$

only tells us that $\beta^\star$ is stationary unless additional assumptions, such as convexity, are imposed.
Numerically, the gradient will generally not be exactly zero.
We therefore introduce a tolerance

$$\varepsilon > 0$$

and stop when

$$\boxed{\|\nabla f(\beta_t)\|_2 < \varepsilon.}$$

For example,

$$\varepsilon = 10^{-8}.$$

This criterion means that the current point is approximately stationary.
The algorithm also stops if a predefined maximum number of iterations

$$K_{\max}$$

is reached.

26.11 Complete Gradient Descent Algorithm

Given

$$f : \mathbb{R}^p \to \mathbb{R}, \qquad \beta_0 \in \mathbb{R}^p, \qquad \varepsilon > 0, \qquad K_{\max} \in \mathbb{N},$$

repeat for

$$t = 0, 1, \ldots, K_{\max} - 1 :$$

1. Compute the gradient:

$$g_t = \nabla f(\beta_t).$$

2. Check the stopping criterion:

$$\|g_t\|_2 < \varepsilon.$$

   If this condition is satisfied, stop and return

$$\beta_t.$$

3. Otherwise, choose a step size

$$\gamma_t > 0$$

   and update:

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t g_t.}$$

4. Repeat.

26.12 One-Dimensional Case

In the TD, we first consider the simpler case

$$f : \mathbb{R} \to \mathbb{R}.$$

There is now only one parameter,

$$\beta \in \mathbb{R},$$

so the gradient reduces to the ordinary derivative:

$$\boxed{\nabla f(\beta) = f'(\beta).}$$

The gradient descent update therefore becomes

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t f'(\beta_t).}$$

The sign of the derivative determines the direction of the update.
If

$$f'(\beta_t) > 0,$$

the function is locally increasing, and

$$\beta_{t+1} = \beta_t - \gamma_t f'(\beta_t) < \beta_t.$$

The algorithm therefore moves to the left.

Conversely, if

$$f'(\beta_t) < 0,$$

then

$$-\gamma_t f'(\beta_t) > 0,$$

and therefore

$$\beta_{t+1} > \beta_t.$$

The algorithm moves to the right.

Finally, near a stationary point,

$$f'(\beta_t) \approx 0,$$

so

$$\beta_{t+1} \approx \beta_t.$$

26.13 Local Minimum versus Global Minimum

Gradient descent uses **local first-order information**: its update at iteration t depends only on the gradient evaluated at the current point,

$$\nabla f(\beta_t).$$

If f is convex, then every local minimum is also a global minimum. Furthermore, every stationary point of a differentiable convex function is a global minimum.

Therefore, under suitable convergence conditions, gradient descent can converge to

$$\boxed{\beta^\star \in \arg\min_\beta f(\beta).}$$

If the minimizer is unique, we may write

$$\boxed{\beta^\star = \arg\min_\beta f(\beta).}$$

If f is non-convex, several local minima and other stationary points may exist, and the solution found by gradient descent can depend on the initialization

$$\beta_0.$$

Therefore, in general,

Gradient descent seeks a stationary solution; without convexity, this need not be the global minimum.

26.14 Key Idea

Gradient descent can ultimately be summarized by three operations:

Evaluate slope $\longrightarrow$	Move downhill
$\nabla f(\beta_t)$	$-\nabla f(\beta_t)$
	$\downarrow$
	$\beta_{t+1} = \beta_t - \gamma_t \nabla f(\beta_t)$

At every iteration, the gradient tells us which direction locally increases the objective the fastest, so gradient descent moves in the opposite direction in order to minimize it.

The theoretical logic of gradient descent can be summarized as

f differentiable
$+$
∇f is L-Lipschitz
$+$
$0 < \gamma < 2/L$
$+$
f bounded below

$\Downarrow$

$\|\nabla f(\beta_k)\|_2 \to 0$
stationarity

$+$ convexity
$\Downarrow$
global minimum

$+$ strong convexity
$\Downarrow$
unique global minimum
$+$
linear convergence under an appropriate step size.

The key conceptual distinction is therefore:

Smoothness	$\longrightarrow$	controls the behavior of gradient descent
Convexity	$\longrightarrow$	determines what stationarity implies.

27 Log-Likelihood Maximization Exercise Using Gradient Descent

27.1 Logistic Regression Model

Logistic regression is a statistical model used when the dependent variable is binary. In a machine-learning context, it can equivalently be viewed as a binary classification model.

The objective of this exercise is to recover the parameter $\beta^\star$ that was used to generate the binary observations $Y_1, \ldots, Y_n$.

We will derive:

1. the probability model implied by the latent-variable specification;
2. the likelihood of the observations;
3. the log-likelihood;
4. the negative log-likelihood;
5. its gradient;
6. the gradient-descent update used to estimate $\beta^\star$.

27.2 Latent Variable Representation

For each observation $i = 1, \ldots, n$, consider the latent variable

$$\boxed{Y_i^\star = \beta x_i + \varepsilon_i,}$$

where

$$\beta \in \mathbb{R}, \qquad x_i \in \mathbb{R}, \qquad \varepsilon_i \in \mathbb{R}.$$

The latent variable $Y_i^\star$ is not directly observed.

Instead, we observe the binary variable

$$\boxed{Y_i = \begin{cases} 1, & \text{if } Y_i^\star > 0, \\ 0, & \text{otherwise.} \end{cases}}$$

Substituting the expression for $Y_i^\star$,

$$Y_i = 1 \quad \Longleftrightarrow \quad \beta x_i + \varepsilon_i > 0.$$

Equivalently,

$$\boxed{Y_i = 1 \quad \Longleftrightarrow \quad \varepsilon_i > -\beta x_i.}$$

27.3 Distribution of the Error Term

The noise term ε_i follows a standard logistic distribution.

Its probability density function is

$$\boxed{f_\varepsilon(z) = \frac{e^{-z}}{(1+e^{-z})^2}, \qquad z \in \mathbb{R}.}$$

Derivation of the Logistic Cumulative Distribution Function

By definition, the cumulative distribution function (CDF) is obtained by integrating the probability density function from $-\infty$ to z:

$$F_\varepsilon(z) = \mathbb{P}(\varepsilon_i \leq z) = \int_{-\infty}^{z} f_\varepsilon(u)\, du.$$

Substituting the logistic density,

$$\boxed{F_\varepsilon(z) = \int_{-\infty}^{z} \frac{e^{-u}}{(1+e^{-u})^2}\, du.}$$

We therefore need to compute the definite integral

$$\int_{-\infty}^{z} \frac{e^{-u}}{(1+e^{-u})^2}\, du.$$

Step 1: Change of variable. Define

$$\boxed{v = 1 + e^{-u}.}$$

Differentiating with respect to u,

$$\frac{dv}{du} = -e^{-u},$$

so

$$dv = -e^{-u}\, du,$$

or equivalently,

$$\boxed{e^{-u}\, du = -dv.}$$

Since we are working with a **definite integral**, we must also transform the bounds of integration.

For the lower bound,

$$u \to -\infty.$$

Then

$$-u \to +\infty,$$

and therefore

$$e^{-u} \to +\infty.$$

Hence,

$$v = 1 + e^{-u} \to +\infty.$$

Thus,

$$\boxed{u \to -\infty \quad \Longrightarrow \quad v \to +\infty.}$$

For the upper bound,

$$u = z,$$

so

$$v = 1 + e^{-z}.$$

Thus,

$$\boxed{u = z \quad \Longrightarrow \quad v = 1 + e^{-z}.}$$

The transformation of the bounds can therefore be summarized as

u	$v = 1 + e^{-u}$
$-\infty$	$+\infty$
z	$1 + e^{-z}$

Consequently,

$$F_{\varepsilon}(z) = \int_{-\infty}^{z} \frac{e^{-u}}{(1 + e^{-u})^2}\, du$$

becomes

$$\boxed{F_{\varepsilon}(z) = -\int_{+\infty}^{1+e^{-z}} \frac{1}{v^2}\, dv.}$$

Since

$$\frac{1}{v^2} = v^{-2},$$

we have

$$\boxed{F_\varepsilon(z) = -\int_{+\infty}^{1+e^{-z}} v^{-2}\,dv.}$$

Step 2: Compute the integral. Recall the power rule for integration:

$$\boxed{\int v^a\,dv = \frac{v^{a+1}}{a+1} + C, \qquad a \neq -1.}$$

Here,

$$a = -2.$$

Therefore,

$$a + 1 = -1,$$

and

$$\int v^{-2}\,dv = \frac{v^{-1}}{-1} + C.$$

Hence,

$$\boxed{\int v^{-2}\,dv = -v^{-1} + C.}$$

Since

$$v^{-1} = \frac{1}{v},$$

this can equivalently be written as

$$\boxed{\int v^{-2}\,dv = -\frac{1}{v} + C.}$$

We can verify this result by differentiation:

$$\frac{d}{dv}\left(-\frac{1}{v}\right) = \frac{d}{dv}\left(-v^{-1}\right) = -(-1)v^{-2} = v^{-2}.$$

Returning to our definite integral,

$$F_\varepsilon(z) = -\int_{+\infty}^{1+e^{-z}} v^{-2}\,dv.$$

Using the antiderivative

$$\int v^{-2}\,dv = -v^{-1},$$

we obtain

$$F_\varepsilon(z) = -\left[-v^{-1}\right]_{+\infty}^{1+e^{-z}}.$$

Equivalently,

$$F_\varepsilon(z) = -\left[-\frac{1}{v}\right]_{+\infty}^{1+e^{-z}}.$$

The two minus signs cancel, giving

$$\boxed{F_\varepsilon(z) = \left[\frac{1}{v}\right]_{+\infty}^{1+e^{-z}}.}$$

Step 3: Evaluate the integral at its bounds. Recall that for any antiderivative G,

$$\boxed{\int_a^b g(v)\,dv = [G(v)]_a^b = G(b) - G(a).}$$

Therefore,

$$F_\varepsilon(z) = \left[\frac{1}{v}\right]_{+\infty}^{1+e^{-z}}$$

means

$$F_\varepsilon(z) = \frac{1}{1+e^{-z}} - \lim_{v\to+\infty}\frac{1}{v}.$$

Since

$$\boxed{\lim_{v\to+\infty}\frac{1}{v} = 0,}$$

we obtain

$$F_\varepsilon(z) = \frac{1}{1+e^{-z}} - 0.$$

Hence,

$$\boxed{F_\varepsilon(z) = \frac{1}{1+e^{-z}}.}$$

Therefore, the cumulative distribution function of the standard logistic distribution is

$$\boxed{F_\varepsilon(z) = \mathbb{P}(\varepsilon_i \leq z) = \frac{1}{1+e^{-z}}.}$$

The Logistic Function

The function

$$\boxed{\sigma(z) = \frac{1}{1+e^{-z}}}$$

is called the **logistic function**, or **sigmoid function**.

Hence, the CDF of a standard logistic random variable is exactly the logistic function:

$$\boxed{F_\varepsilon(z) = \sigma(z).}$$

Verification: Recovering the Density from the CDF

A cumulative distribution function must satisfy

$$f_\varepsilon(z) = \frac{d}{dz} F_\varepsilon(z)$$

whenever the density exists.

We found

$$F_\varepsilon(z) = \frac{1}{1+e^{-z}}.$$

Writing

$$F_\varepsilon(z) = (1+e^{-z})^{-1},$$

Reminder: Chain Rule Suppose that

$$h(z) = g(u(z)).$$

Then the chain rule states that

$$\boxed{h'(z) = g'(u(z))\, u'(z).}$$

In other words, we differentiate the **outer function** first and multiply by the derivative of the **inner function**.

In our case,

$$F_\varepsilon(z) = (1 + e^{-z})^{-1}.$$

We can identify the outer and inner functions as

$$g(u) = u^{-1}$$

and

$$u(z) = 1 + e^{-z}.$$

Thus,

$$\boxed{F_\varepsilon(z) = g(u(z)).}$$

Step 1: Differentiate the outer function. The outer function is

$$g(u) = u^{-1}.$$

Using the power rule,

$$\boxed{\frac{d}{du}u^a = au^{a-1},}$$

with

$$a = -1,$$

we obtain

$$g'(u) = -1u^{-2}.$$

Therefore,

$$\boxed{g'(u) = -u^{-2}.}$$

Replacing u by its expression

$$u(z) = 1 + e^{-z},$$

gives

$$\boxed{g'(u(z)) = -(1 + e^{-z})^{-2}.}$$

Step 2: Differentiate the inner function. The inner function is

$$u(z) = 1 + e^{-z}.$$

Therefore,

$$u'(z) = \frac{d}{dz}\left(1 + e^{-z}\right).$$

Using the derivative of a sum,

$$\frac{d}{dz}\left(1 + e^{-z}\right) = \frac{d}{dz}(1) + \frac{d}{dz}(e^{-z}).$$

The derivative of a constant is zero:

$$\boxed{\frac{d}{dz}(1) = 0.}$$

We therefore need to calculate

$$\frac{d}{dz}(e^{-z}).$$

which is

$$\boxed{\frac{d}{dz}e^{-z} = -e^{-z}.}$$

Returning to the derivative of the inner function,

$$u'(z) = \frac{d}{dz}(1) + \frac{d}{dz}(e^{-z}),$$

we obtain

$$u'(z) = 0 - e^{-z}.$$

Therefore,

$$\boxed{u'(z) = -e^{-z}.}$$

Step 4: Combine the derivatives. The chain rule gives

$$F_\varepsilon'(z) = g'(u(z))u'(z).$$

We have shown that

$$g'(u(z)) = -(1 + e^{-z})^{-2}$$

and

$$u'(z) = -e^{-z}.$$

Therefore,

$$F'_{\varepsilon}(z) = \left[-(1+e^{-z})^{-2}\right]\left[-e^{-z}\right].$$

The two negative signs cancel. Hence,

$$F'_{\varepsilon}(z) = e^{-z}(1+e^{-z})^{-2}.$$

Finally, we obtain

$$e^{-z}(1+e^{-z})^{-2} = e^{-z}\frac{1}{(1+e^{-z})^2}.$$

Therefore,

$$\boxed{F'_{\varepsilon}(z) = \frac{e^{-z}}{(1+e^{-z})^2}.}$$

But the probability density function of the standard logistic distribution is

$$f_{\varepsilon}(z) = \frac{e^{-z}}{(1+e^{-z})^2}.$$

Consequently,

$$\boxed{F'_{\varepsilon}(z) = f_{\varepsilon}(z).}$$

Complete derivation in one line. The entire differentiation can be summarized as

$$\boxed{\begin{aligned} F'_{\varepsilon}(z) &= \frac{d}{dz}(1+e^{-z})^{-1} \\ &= -1(1+e^{-z})^{-2}\frac{d}{dz}(1+e^{-z}) \\ &= -(1+e^{-z})^{-2}\left[0+e^{-z}\frac{d}{dz}(-z)\right] \\ &= -(1+e^{-z})^{-2}\left[-e^{-z}\right] \\ &= e^{-z}(1+e^{-z})^{-2} \\ &= \frac{e^{-z}}{(1+e^{-z})^2} \\ &= f_{\varepsilon}(z). \end{aligned}}$$

Thus, differentiating the derived CDF correctly recovers the original logistic probability density function.

Symmetry of the Logistic Distribution

The standard logistic distribution is symmetric around zero.

Using the CDF,

$$F_\varepsilon(-z) = \frac{1}{1+e^z}.$$

Therefore,

$$1 - F_\varepsilon(-z) = 1 - \frac{1}{1+e^z}.$$

Putting the terms over the same denominator,

$$1 - F_\varepsilon(-z) = \frac{1+e^z-1}{1+e^z} = \frac{e^z}{1+e^z}.$$

Dividing the numerator and denominator by e^z,

$$\frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}.$$

But

$$F_\varepsilon(z) = \frac{1}{1+e^{-z}}.$$

Hence,

$$\boxed{1 - F_\varepsilon(-z) = F_\varepsilon(z).}$$

Equivalently,

$$\boxed{F_\varepsilon(-z) = 1 - F_\varepsilon(z).}$$

This symmetry property will be particularly useful when deriving

$$\mathbb{P}(Y_i = 1 \mid x_i)$$

from the latent-variable representation of the logistic regression model.

27.4 Explanatory Variables

For

$$i = 1, \ldots, n,$$

we observe the explanatory variables

$$x_i \overset{iid}{\sim} \mathcal{N}(0, 1).$$

The observations x_i are therefore independently and identically distributed according to a standard normal distribution.

Conditional on x_i, however, the probability distribution of Y_i depends on x_i. Consequently, the binary observations Y_i are independent but, conditional on the realized regressors, are generally **not identically distributed**.

Indeed,

$$\mathbb{P}(Y_i = 1 \mid x_i)$$

depends on the value of x_i.

27.5 Derivation of the Conditional Probability

We now derive the probability that

$$Y_i = 1.$$

From the latent-variable model,

$$Y_i = 1 \quad \Longleftrightarrow \quad Y_i^\star > 0.$$

Since

$$Y_i^\star = \beta x_i + \varepsilon_i,$$

we have

$$\mathbb{P}(Y_i = 1 \mid x_i) = \mathbb{P}\left(\beta x_i + \varepsilon_i > 0 \mid x_i\right).$$

Therefore,

$$\mathbb{P}(Y_i = 1 \mid x_i) = \mathbb{P}\left(\varepsilon_i > -\beta x_i\right).$$

Using the cumulative distribution function,

$$\mathbb{P}\left(\varepsilon_i > -\beta x_i\right) = 1 - F_\varepsilon(-\beta x_i).$$

Because the logistic distribution is symmetric,

$$1 - F_\varepsilon(-z) = F_\varepsilon(z),$$

so

$$\mathbb{P}(Y_i = 1 \mid x_i) = F_\varepsilon(\beta x_i).$$

Using

$$F_\varepsilon(z) = \frac{1}{1 + e^{-z}},$$

we obtain

$$\boxed{\mathbb{P}(Y_i = 1 \mid x_i) = \frac{1}{1 + e^{-\beta x_i}}.}$$

Define

$$\boxed{p_i(\beta) = \sigma(\beta x_i) = \frac{1}{1 + e^{-\beta x_i}}.}$$

Then

$$\boxed{Y_i \mid x_i \sim \text{Bernoulli}\left(p_i(\beta)\right).}$$

Similarly,

$$\mathbb{P}(Y_i = 0 \mid x_i) = 1 - p_i(\beta).$$

Since

$$1 - \frac{1}{1 + e^{-\beta x_i}} = \frac{e^{-\beta x_i}}{1 + e^{-\beta x_i}},$$

we have

$$\boxed{\mathbb{P}(Y_i = 0 \mid x_i) = \frac{e^{-\beta x_i}}{1 + e^{-\beta x_i}} = \frac{1}{1 + e^{\beta x_i}}.}$$

27.6 Probability of an Observed Binary Outcome

Since

$$Y_i \in \{0, 1\},$$

the probability of observing the realized value y_i can be written compactly as

$$\boxed{\mathbb{P}(Y_i = y_i \mid x_i; \beta) = p_i(\beta)^{y_i} \left(1 - p_i(\beta)\right)^{1-y_i}.}$$

To see why, consider the two possible cases.

If

$$y_i = 1,$$

then

$$p_i^{y_i}(1-p_i)^{1-y_i} = p_i^1(1-p_i)^0 = p_i.$$

If

$$y_i = 0,$$

then

$$p_i^{y_i}(1-p_i)^{1-y_i} = p_i^0(1-p_i)^1 = 1 - p_i.$$

Thus the same expression covers both possible outcomes.

27.7 Likelihood Function

Let

$$y_1, \ldots, y_n$$

denote the observed realizations.

Conditional on the regressors $x_1, \ldots, x_n$, the observations are independent. Therefore, the joint probability of observing the complete sample is the product of the individual probabilities:

$$\boxed{L(\beta) = \prod_{i=1}^{n} \mathbb{P}(Y_i = y_i \mid x_i; \beta).}$$

Using the Bernoulli representation,

$$\boxed{L(\beta) = \prod_{i=1}^{n} p_i(\beta)^{y_i} \left(1 - p_i(\beta)\right)^{1-y_i}.}$$

The likelihood is interpreted as a function of the unknown parameter β, while the observed data

$$(x_1, y_1), \ldots, (x_n, y_n)$$

are treated as fixed.

The maximum-likelihood estimator is

$$\boxed{\hat{\beta}_{MLE} = \arg\max_{\beta \in \mathbb{R}} L(\beta).}$$

27.8 Log-Likelihood Function

Products are inconvenient to differentiate and can also become numerically very small.

We therefore take the logarithm of the likelihood.

Since the logarithm is strictly increasing,

$$\arg\max_{\beta} L(\beta) = \arg\max_{\beta} \log L(\beta).$$

Define the log-likelihood as

$$\boxed{\ell(\beta) = \log L(\beta).}$$

Then

$$\ell(\beta) = \log \left[\prod_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1-y_i} \right].$$

Using

$$\log \left(\prod_{i=1}^{n} a_i \right) = \sum_{i=1}^{n} \log(a_i),$$

we obtain

$$\ell(\beta) = \sum_{i=1}^{n} \log \left[p_i^{y_i} (1 - p_i)^{1-y_i} \right].$$

we first use the logarithm of a product:

$$\boxed{\log(ab) = \log(a) + \log(b).}$$

For each observation i,

$$\log \left[p_i^{y_i} (1 - p_i)^{1-y_i} \right] = \log \left(p_i^{y_i} \right) + \log \left((1 - p_i)^{1-y_i} \right).$$

Therefore,

$$\ell(\beta) = \sum_{i=1}^{n} \left[\log \left(p_i^{y_i} \right) + \log \left((1 - p_i)^{1-y_i} \right) \right].$$

We now use the logarithm of a power:

$$\boxed{\log(a^b) = b\log(a).}$$

Applying this property to the first term,

$$\log\left(p_i^{y_i}\right) = y_i \log(p_i),$$

and to the second term,

$$\log\left((1-p_i)^{1-y_i}\right) = (1-y_i)\log(1-p_i).$$

Hence,

$$\ell(\beta) = \sum_{i=1}^{n} \left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right].$$

Therefore, the standard Bernoulli log-likelihood is

$$\boxed{\ell(\beta) = \sum_{i=1}^{n} \left[y_i \log p_i + (1-y_i)\log(1-p_i)\right].}$$

27.9 Simplification of the Log-Likelihood

Recall that

$$p_i = \frac{1}{1+e^{-\beta x_i}}.$$

Therefore,

$$\log p_i = \log\left(\frac{1}{1+e^{-\beta x_i}}\right) = -\log(1+e^{-\beta x_i}).$$

Also,

$$1 - p_i = \frac{e^{-\beta x_i}}{1+e^{-\beta x_i}}.$$

Hence,

$$\log(1-p_i) = \log\left(\frac{e^{-\beta x_i}}{1+e^{-\beta x_i}}\right).$$

Using

$$\log\left(\frac{a}{b}\right) = \log(a) - \log(b),$$

we obtain

$$\log(1-p_i) = -\beta x_i - \log(1+e^{-\beta x_i}).$$

Substituting these expressions into

$$\ell(\beta) = \sum_{i=1}^{n} \left[y_i \log p_i + (1-y_i) \log(1-p_i) \right],$$

gives

$$\ell(\beta) = \sum_{i=1}^{n} \left[-y_i \log(1+e^{-\beta x_i}) + (1-y_i) \left(-\beta x_i - \log(1+e^{-\beta x_i}) \right) \right].$$

Expanding,

$$\ell(\beta) = \sum_{i=1}^{n} \left[-y_i \log(1+e^{-\beta x_i}) - (1-y_i)\beta x_i - (1-y_i) \log(1+e^{-\beta x_i}) \right].$$

Combining the logarithmic terms,

$$-y_i \log(1+e^{-\beta x_i}) - (1-y_i) \log(1+e^{-\beta x_i})$$

becomes

$$-\left[y_i + (1-y_i) \right] \log(1+e^{-\beta x_i}).$$

Since

$$y_i + (1-y_i) = 1,$$

We obtain

$$-\left[y_i + (1-y_i) \right] \log(1+e^{-\beta x_i}) = -\log(1+e^{-\beta x_i})$$

Thus,

$$\ell(\beta) = -\sum_{i=1}^{n} \left[y_i + (1-y_i) \right] \log(1+e^{-\beta x_i}) - \beta \sum_{i=1}^{n} (1-y_i) x_i.$$

Giving

$$\boxed{\ell(\beta) = -\sum_{i=1}^{n} \log(1+e^{-\beta x_i}) - \beta \sum_{i=1}^{n} (1-y_i) x_i.}$$

Because $y_i \in \{0,1\}$,

$$1 - y_i = \begin{cases} 0, & y_i = 1, \\ 1, & y_i = 0. \end{cases}$$

Therefore,

$$\sum_{i=1}^{n}(1-y_i)x_i = \sum_{i:y_i=0} x_i.$$

Hence the log-likelihood can be written exactly as in the exercise:

$$\boxed{\ell(\beta) = -\sum_{i=1}^{n}\log(1+e^{-\beta x_i}) - \beta\sum_{i:y_i=0} x_i.}$$

The maximum-likelihood problem is therefore

$$\boxed{\max_{\beta\in\mathbb{R}}\left[-\sum_{i=1}^{n}\log(1+e^{-\beta x_i}) - \beta\sum_{i:y_i=0} x_i\right].}$$

27.10 From Maximization to Minimization

Gradient descent is conventionally formulated as a **minimization algorithm**.

However, maximum likelihood requires

$$\max_{\beta}\ell(\beta).$$

We can transform the maximization problem into a minimization problem by multiplying the objective by -1:

$$\boxed{\arg\max_{\beta}\ell(\beta) = \arg\min_{\beta}[-\ell(\beta)].}$$

Define the negative log-likelihood

$$\boxed{J(\beta) = -\ell(\beta).}$$

Using the previous expression,

$$\boxed{J(\beta) = \beta\sum_{i:y_i=0} x_i + \sum_{i=1}^{n}\log(1+e^{-\beta x_i}).}$$

The estimation problem therefore becomes

$$\boxed{\hat{\beta}_{MLE} = \arg\min_{\beta\in\mathbb{R}} J(\beta).}$$

27.11 Gradient of the Negative Log-Likelihood

Because there is only one parameter,

$$\beta \in \mathbb{R},$$

the gradient is simply the ordinary derivative:

$$\nabla J(\beta) = J'(\beta).$$

Recall

$$J(\beta) = \beta \sum_{i:y_i=0} x_i + \sum_{i=1}^{n} \log(1 + e^{-\beta x_i}).$$

We differentiate the two terms separately.

Derivative of the First Term

Consider

$$\beta \sum_{i:y_i=0} x_i.$$

The observations x_i are fixed with respect to β. Therefore,

$$\boxed{\frac{d}{d\beta}\left[\beta \sum_{i:y_i=0} x_i\right] = \sum_{i:y_i=0} x_i.}$$

Derivative of the Second Term

Now consider one element of the second sum:

$$g_i(\beta) = \log(1 + e^{-\beta x_i}).$$

We apply the chain rule.

Define

$$u(\beta) = 1 + e^{-\beta x_i}.$$

Then

$$g_i(\beta) = \log u(\beta).$$

Since

$$\frac{d}{du}\log u = \frac{1}{u},$$

we have

$$g_i'(\beta) = \frac{1}{1+e^{-\beta x_i}}\frac{d}{d\beta}\left(1+e^{-\beta x_i}\right).$$

Now,

$$\frac{d}{d\beta}1 = 0,$$

and

$$\frac{d}{d\beta}e^{-\beta x_i} = e^{-\beta x_i}\frac{d}{d\beta}(-\beta x_i).$$

Since

$$\frac{d}{d\beta}(-\beta x_i) = -x_i,$$

we obtain

$$\frac{d}{d\beta}e^{-\beta x_i} = -x_i e^{-\beta x_i}.$$

Therefore,

$$\boxed{g_i'(\beta) = -\frac{x_i e^{-\beta x_i}}{1+e^{-\beta x_i}}.}$$

Consequently,

$$\frac{d}{d\beta}\sum_{i=1}^{n}\log(1+e^{-\beta x_i}) = -\sum_{i=1}^{n}\frac{x_i e^{-\beta x_i}}{1+e^{-\beta x_i}}.$$

Complete Gradient

Combining the two derivatives gives

$$\boxed{J'(\beta) = \sum_{i:y_i=0} x_i - \sum_{i=1}^{n}\frac{x_i e^{-\beta x_i}}{1+e^{-\beta x_i}}.}$$

Since

$$J(\beta) = -\ell(\beta),$$

we can equivalently write

$$\boxed{-\ell'(\beta) = \sum_{i:y_i=0} x_i - \sum_{i=1}^{n} \frac{x_i e^{-\beta x_i}}{1 + e^{-\beta x_i}}.}$$

This is the expression appearing in the exercise.

27.12 Connection with the Standard Logistic Regression Gradient

The previous expression can be simplified considerably.

Recall that

$$p_i(\beta) = \frac{1}{1 + e^{-\beta x_i}}.$$

Therefore,

$$1 - p_i(\beta) = \frac{e^{-\beta x_i}}{1 + e^{-\beta x_i}}.$$

Hence,

$$\frac{x_i e^{-\beta x_i}}{1 + e^{-\beta x_i}} = x_i(1 - p_i).$$

The gradient becomes

$$J'(\beta) = \sum_{i:y_i=0} x_i - \sum_{i=1}^{n} x_i(1 - p_i).$$

Moreover,

$$\sum_{i:y_i=0} x_i = \sum_{i=1}^{n} (1 - y_i)x_i.$$

Thus,

$$J'(\beta) = \sum_{i=1}^{n} (1 - y_i)x_i - \sum_{i=1}^{n} (1 - p_i)x_i.$$

Combining the two sums,

$$J'(\beta) = \sum_{i=1}^{n} \left[(1 - y_i) - (1 - p_i)\right] x_i.$$

But

$$(1 - y_i) - (1 - p_i) = p_i - y_i.$$

Therefore,

$$\boxed{J'(\beta) = \sum_{i=1}^{n} x_i \left(p_i(\beta) - y_i\right).}$$

Since

$$p_i(\beta) = \sigma(\beta x_i),$$

we obtain the standard logistic-regression gradient:

$$\boxed{J'(\beta) = \sum_{i=1}^{n} x_i \left[\sigma(\beta x_i) - y_i\right].}$$

Equivalently, the gradient of the log-likelihood is

$$\boxed{\ell'(\beta) = \sum_{i=1}^{n} x_i \left[y_i - \sigma(\beta x_i)\right].}$$

27.13 Interpretation of the Gradient

The negative log-likelihood gradient is

$$J'(\beta) = \sum_{i=1}^{n} x_i(p_i - y_i).$$

The quantity

$$p_i - y_i$$

measures the difference between the probability predicted by the model and the observed binary outcome.

For example, if

$$y_i = 1 \qquad \text{and} \qquad p_i = 0.2,$$

then

$$p_i - y_i = -0.8.$$

The model assigns a low probability to an event that actually occurred.

Conversely, if

$$y_i = 0 \qquad \text{and} \qquad p_i = 0.9,$$

then

$$p_i - y_i = 0.9.$$

The model assigns a high probability to an event that did not occur.

The gradient aggregates these prediction errors, weighted by the explanatory variables:

$$\boxed{J'(\beta) = \sum_{i=1}^{n} \underbrace{x_i}_{\text{exposure}} \underbrace{(p_i - y_i)}_{\text{prediction error}}}.$$

27.14 Second Derivative and Convexity

To understand why gradient descent is particularly well suited to this problem, consider the second derivative.

We have

$$J'(\beta) = \sum_{i=1}^{n} x_i(p_i - y_i).$$

Since x_i and y_i do not depend on β,

$$J''(\beta) = \sum_{i=1}^{n} x_i \frac{\partial p_i}{\partial \beta}.$$

Recall

$$p_i = \sigma(\beta x_i).$$

The derivative of the logistic function is

$$\boxed{\sigma'(z) = \sigma(z)\left[1 - \sigma(z)\right].}$$

Therefore, using the chain rule,

$$\frac{\partial p_i}{\partial \beta} = x_i p_i(1 - p_i).$$

Hence,

$$\boxed{J''(\beta) = \sum_{i=1}^{n} x_i^2 p_i(1 - p_i).}$$

Since

$$x_i^2 \geq 0$$

and

$$0 < p_i < 1,$$

we have

$$p_i(1 - p_i) > 0.$$

Therefore,

$$\boxed{J''(\beta) \geq 0.}$$

Thus, the negative log-likelihood is a **convex function of** β.

Equivalently, the log-likelihood is concave:

$$\boxed{\ell''(\beta) \leq 0.}$$

Consequently, any finite stationary point of the negative log-likelihood is a global minimizer:

$$\boxed{J'(\hat{\beta}) = 0 \quad \Longrightarrow \quad \hat{\beta} \in \arg\min_{\beta} J(\beta).}$$

27.15 Lipschitz Continuity of the Gradient

Recall that

$$J''(\beta) = \sum_{i=1}^{n} x_i^2 p_i(1 - p_i).$$

For every probability $p \in [0, 1]$,

$$p(1 - p) \leq \frac{1}{4}.$$

Indeed,

$$p(1 - p) = p - p^2$$

is maximized at

$$p = \frac{1}{2},$$

where

$$p(1-p) = \frac{1}{4}.$$

Therefore,

$$J''(\beta) \leq \frac{1}{4}\sum_{i=1}^{n} x_i^2.$$

Hence a valid Lipschitz constant for J' is

$$\boxed{L = \frac{1}{4}\sum_{i=1}^{n} x_i^2.}$$

Thus,

$$\boxed{|J'(x) - J'(y)| \leq L|x-y|.}$$

This connects directly to the convergence theory of gradient descent.

27.16 Gradient Descent Algorithm

We want to solve

$$\hat{\beta} = \arg\min_{\beta} J(\beta).$$

Gradient descent uses the update

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t J'(\beta_t).}$$

Using the derived gradient,

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t \left[\sum_{i:y_i=0} x_i - \sum_{i=1}^{n} \frac{x_i e^{-\beta_t x_i}}{1+e^{-\beta_t x_i}}\right].}$$

Using the standard logistic representation,

$$p_{i,t} = \sigma(\beta_t x_i) = \frac{1}{1+e^{-\beta_t x_i}},$$

the same update can be written much more simply as

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t \sum_{i=1}^{n} x_i \left(p_{i,t} - y_i\right).}$$

27.17 Step-by-Step Estimation Procedure

Choose:

$$\beta_0 \in \mathbb{R}, \qquad \varepsilon > 0, \qquad K_{\max} \in \mathbb{N}.$$

Then, for

$$t = 0, 1, \ldots, K_{\max} - 1,$$

perform the following steps.

1. **Compute the linear score**

$$z_{i,t} = \beta_t x_i, \qquad i = 1, \ldots, n.$$

2. **Transform each score into a probability**

$$p_{i,t} = \sigma(z_{i,t}) = \frac{1}{1 + e^{-z_{i,t}}}.$$

Therefore,

$$p_{i,t} = \frac{1}{1 + e^{-\beta_t x_i}}.$$

3. **Compute the gradient**

$$\boxed{g_t = J'(\beta_t) = \sum_{i=1}^{n} x_i (p_{i,t} - y_i).}$$

4. **Check the stopping criterion**

If

$$\boxed{|g_t| < \varepsilon,}$$

stop and return

$$\hat{\beta} = \beta_t.$$

5. **Choose the learning rate**

Select

$$\gamma_t > 0.$$

For a constant learning rate, the Lipschitz bound derived above gives the standard condition

$$0 < \gamma < \frac{2}{L},$$

where

$$L = \frac{1}{4}\sum_{i=1}^{n} x_i^2.$$

A conservative choice is

$$0 < \gamma \leq \frac{1}{L}.$$

6. **Update the parameter**

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t g_t.}$$

Equivalently,

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t \sum_{i=1}^{n} x_i \left[\sigma(\beta_t x_i) - y_i\right].}$$

7. Repeat until convergence or until $K_{\max}$ iterations have been reached.

27.18 Complete Derivation at a Glance

The entire derivation can be summarized as follows:

$$\boxed{Y_i^\star = \beta x_i + \varepsilon_i}$$

with

$$Y_i = 1_{\{Y_i^\star > 0\}}.$$

Therefore,

$$Y_i = 1 \iff \varepsilon_i > -\beta x_i.$$

Since ε_i follows a logistic distribution,

$$\boxed{p_i(\beta) = \mathbb{P}(Y_i = 1 \mid x_i) = \sigma(\beta x_i) = \frac{1}{1 + e^{-\beta x_i}}.}$$

Thus,

$$Y_i \mid x_i \sim \text{Bernoulli}(p_i).$$

The likelihood is

$$\boxed{L(\beta) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i}.}$$

Taking logarithms,

$$\boxed{\ell(\beta) = \sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right].}$$

This simplifies to

$$\boxed{\ell(\beta) = -\sum_{i=1}^{n} \log(1 + e^{-\beta x_i}) - \beta \sum_{i:y_i=0} x_i.}$$

Define

$$J(\beta) = -\ell(\beta).$$

Then

$$\boxed{J(\beta) = \beta \sum_{i:y_i=0} x_i + \sum_{i=1}^{n} \log(1 + e^{-\beta x_i}).}$$

Differentiating,

$$\boxed{J'(\beta) = \sum_{i:y_i=0} x_i - \sum_{i=1}^{n} \frac{x_i e^{-\beta x_i}}{1 + e^{-\beta x_i}}.}$$

Equivalently,

$$\boxed{J'(\beta) = \sum_{i=1}^{n} x_i \left[\sigma(\beta x_i) - y_i \right].}$$

The gradient-descent update is therefore

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t \sum_{i=1}^{n} x_i \left[\sigma(\beta_t x_i) - y_i \right].}$$

Finally,

$$\boxed{J''(\beta) = \sum_{i=1}^{n} x_i^2 p_i (1 - p_i) \geq 0,}$$

so the negative log-likelihood is convex.

Hence, whenever a finite minimizer exists, the optimization problem has no spurious local minimum:

$$\boxed{J'(\hat{\beta}) = 0 \quad \Longrightarrow \quad \hat{\beta} \text{ is a global minimizer of } J.}$$

27.19 Key Ideas

This exercise connects statistical estimation and numerical optimization:

$$\boxed{\begin{array}{c}\text{Latent-variable model}\\ \Downarrow\\ \text{Logistic probability}\\ p_i=\sigma(\beta x_i)\\ \Downarrow\\ \text{Bernoulli likelihood}\\ L(\beta)\\ \Downarrow\\ \text{Log-likelihood}\\ \ell(\beta)\\ \Downarrow\\ \text{Negative log-likelihood}\\ J(\beta)=-\ell(\beta)\\ \Downarrow\\ \text{Gradient}\\ J'(\beta)=\sum_i x_i(p_i-y_i)\\ \Downarrow\\ \text{Gradient descent}\\ \beta_{t+1}=\beta_t-\gamma_t J'(\beta_t)\\ \Downarrow\\ \hat{\beta}_{MLE}.\end{array}}$$

Thus,

$$\boxed{\text{Maximum Likelihood Estimation} \quad \Longleftrightarrow \quad \text{Optimization of the log-likelihood.}}$$

For logistic regression,

$$\boxed{\text{maximize } \ell(\beta) \quad \Longleftrightarrow \quad \text{minimize } -\ell(\beta),}$$

which allows the maximum-likelihood estimator to be computed directly using gradient descent.

The `weights` Parameter in a Histogram

The `weights` parameter controls how much each observation contributes to the height of the histogram.

Without specifying `weights`, `plt.hist()` displays the **number of observations** contained in each bin. Therefore, if bin j contains n_j observations, its height is simply

$$\boxed{\text{Height}_j = n_j.}$$

In our case, we use

```
weights = np.ones_like(unif_sample_estimation) \
          / len(unif_sample_estimation)
```

Suppose that the sample contains N estimated values of $\hat{\beta}$:

$$\hat{\beta}_1, \hat{\beta}_2, \ldots, \hat{\beta}_N.$$

The expression

`np.ones_like(unif_sample_estimation)`

creates an array containing N ones:

$$\begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix}.$$

Dividing by the number of observations N gives

$$\boxed{\begin{bmatrix} \frac{1}{N} & \frac{1}{N} & \cdots & \frac{1}{N} \end{bmatrix}.}$$

Hence, each observation $\hat{\beta}_i$ contributes a weight

$$\boxed{w_i = \frac{1}{N}.}$$

If bin j contains n_j observations, its height is therefore

$$\text{Height}_j = \sum_{i \in \text{bin } j} w_i = \sum_{i \in \text{bin } j} \frac{1}{N},$$

which gives

$$\boxed{\text{Height}_j = \frac{n_j}{N}.}$$

Thus, the height of each bin represents the **proportion of observations** falling into that bin rather than the absolute number of observations.

For example, if

$$N = 1000$$

and a particular bin contains

$$n_j = 200$$

estimated values, then

$$\text{Height}_j = \frac{200}{1000} = 0.20.$$

Hence, 20% of the estimated values of $\hat{\beta}$ fall inside this bin.

Moreover, since every observation belongs to exactly one bin,

$$\sum_j n_j = N.$$

Therefore,

$$\sum_j \text{Height}_j = \sum_j \frac{n_j}{N} = \frac{1}{N} \sum_j n_j = \frac{N}{N} = 1.$$

Hence,

$$\boxed{\sum_j \text{Height}_j = 1.}$$

Difference with `density=True`. It is important not to confuse this normalization with

```
plt.hist(x, bins=30, density=True)
```

Using weights $w_i = 1/N$ ensures that the **sum of the bin heights** equals one:

$$\boxed{\sum_j \text{Height}_j = 1.}$$

By contrast, using `density=True` ensures that the **total area of the histogram** equals one:

$$\boxed{\sum_j \text{Height}_j \times \text{Bin Width}_j = 1.}$$

Therefore, the two normalizations have different interpretations:

Normalization	Interpretation
$w_i = 1/N$	Proportion of observations in each bin
`density=True`	Probability density; total histogram area equals 1

In this example, using

```
weights=np.ones_like(unif_sample_estimation) \
        / len(unif_sample_estimation)
```

therefore allows the vertical axis to be interpreted directly as the **fraction of estimated values of** $\hat{\beta}$ contained in each interval.

Log Returns

A **log return**, also called a **continuously compounded return**, measures the return of an asset using the logarithm of the ratio between two consecutive prices.

Let P_t denote the price of an asset at time t, and P_{t-1} its price at the previous period. The log return between $t-1$ and t is defined as

$$\boxed{r_t^{\log} = \ln\left(\frac{P_t}{P_{t-1}}\right).}$$

Using the logarithmic property

$$\ln\left(\frac{a}{b}\right) = \ln(a) - \ln(b),$$

the log return can equivalently be written as

$$\boxed{r_t^{\log} = \ln(P_t) - \ln(P_{t-1}).}$$

Relationship with Simple Returns The simple return between $t-1$ and t is defined as

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

Equivalently,

$$R_t = \frac{P_t}{P_{t-1}} - 1.$$

Therefore,

$$1 + R_t = \frac{P_t}{P_{t-1}}.$$

Substituting this expression into the definition of the log return gives

$$\boxed{r_t^{\log} = \ln(1 + R_t).}$$

Conversely, exponentiating both sides,

$$e^{r_t^{\log}} = 1 + R_t,$$

and therefore

$$\boxed{R_t = e^{r_t^{\log}} - 1.}$$

Time Additivity of Log Returns One important property of log returns is that they are **additive through time**.

Consider the cumulative log return between dates 0 and T:

$$\sum_{t=1}^{T} r_t^{\log} = \sum_{t=1}^{T} \ln\left(\frac{P_t}{P_{t-1}}\right).$$

Expanding the sum,

$$\sum_{t=1}^{T} r_t^{\log} = \ln\left(\frac{P_1}{P_0}\right) + \ln\left(\frac{P_2}{P_1}\right) + \cdots + \ln\left(\frac{P_T}{P_{T-1}}\right).$$

Using

$$\ln(a) + \ln(b) = \ln(ab),$$

we obtain

$$\sum_{t=1}^{T} r_t^{\log} = \ln\left(\frac{P_1}{P_0}\frac{P_2}{P_1}\cdots\frac{P_T}{P_{T-1}}\right).$$

All intermediate prices cancel, giving

$$\boxed{\sum_{t=1}^{T} r_t^{\log} = \ln\left(\frac{P_T}{P_0}\right).}$$

Thus, the cumulative log return can simply be obtained by summing the individual log returns.

Approximation for Small Returns Recall the first-order approximation

$$\ln(1+x) \approx x \qquad \text{when } x \text{ is close to } 0.$$

Since

$$r_t^{\log} = \ln(1 + R_t),$$

for sufficiently small simple returns,

$$\boxed{r_t^{\log} \approx R_t.}$$

Therefore, for small daily returns, simple returns and log returns are often numerically very close, although they are not exactly equal.

Why Stationarity Matters for Autoregressive Processes

For the standard stationary AR framework, we impose stationarity so that the process has stable unconditional moments and dependence structure, allowing observations across time to inform the same population quantities. Nonstationary AR processes can still be modeled, but require different theory and interpretation.

Stationarity is particularly important for autoregressive (AR) processes because an AR model uses the **past behavior of a variable to describe and predict its future behavior**. For this relationship to remain meaningful, the statistical properties of the process should remain stable over time.

Consider an AR(1) process:

$$\boxed{X_t = c + \phi X_{t-1} + \varepsilon_t,}$$

where

$$\varepsilon_t \sim WN(0, \sigma_\varepsilon^2).$$

For the AR(1) process to be covariance-stationary, we require

$$\boxed{|\phi| < 1.}$$

Why does the condition $|\phi| < 1$ matter? Starting from

$$X_t = c + \phi X_{t-1} + \varepsilon_t,$$

we can substitute the expression for X_{t-1}:

$$X_{t-1} = c + \phi X_{t-2} + \varepsilon_{t-1}.$$

Therefore,

$$X_t = c + \phi\left(c + \phi X_{t-2} + \varepsilon_{t-1}\right) + \varepsilon_t.$$

Expanding,

$$X_t = c(1 + \phi) + \phi^2 X_{t-2} + \phi\varepsilon_{t-1} + \varepsilon_t.$$

Repeating this substitution recursively gives

$$X_t = c(1 + \phi + \phi^2 + \cdots) + \varepsilon_t + \phi\varepsilon_{t-1} + \phi^2\varepsilon_{t-2} + \cdots.$$

If

$$|\phi| < 1,$$

then

$$\boxed{\phi^k \longrightarrow 0 \qquad \text{as } k \longrightarrow \infty.}$$

Therefore, the influence of observations and shocks from the distant past progressively disappears.

Moreover, the geometric series converges:

$$1 + \phi + \phi^2 + \cdots = \frac{1}{1-\phi}.$$

Hence, the AR(1) process admits the infinite moving-average representation

$$\boxed{X_t = \frac{c}{1-\phi} + \sum_{j=0}^{\infty} \phi^j \varepsilon_{t-j}.}$$

This representation makes the stationarity properties of the process particularly clear.

Constant Mean Taking expectations,

$$\mathbb{E}[X_t] = \frac{c}{1-\phi} + \sum_{j=0}^{\infty} \phi^j \mathbb{E}[\varepsilon_{t-j}].$$

Since

$$\mathbb{E}[\varepsilon_t] = 0,$$

we obtain

$$\boxed{\mathbb{E}[X_t] = \frac{c}{1-\phi}.}$$

The mean therefore does not depend on time t.

Constant Variance Since the innovations are uncorrelated,

$$\mathrm{Var}(X_t) = \sigma_\varepsilon^2 \sum_{j=0}^{\infty} \phi^{2j}.$$

Because $|\phi| < 1$,

$$\sum_{j=0}^{\infty} \phi^{2j} = \frac{1}{1-\phi^2}.$$

Therefore,

$$\boxed{\mathrm{Var}(X_t) = \frac{\sigma_\varepsilon^2}{1-\phi^2}.}$$

Again, the variance does not depend on t.

Autocovariance Depends Only on the Lag For a stationary AR(1) process, the autocovariance at lag h is

$$\boxed{\gamma(h) = \mathrm{Cov}(X_t, X_{t-h}) = \phi^{|h|}\frac{\sigma_\varepsilon^2}{1-\phi^2}.}$$

Thus, the covariance between two observations depends only on the distance between them,

$$|h|,$$

and not on the particular calendar date t.

These properties correspond precisely to covariance stationarity:

$$\boxed{\begin{aligned}\mathbb{E}[X_t] &= \mu,\\ \mathrm{Var}(X_t) &= \sigma_X^2,\\ \mathrm{Cov}(X_t, X_{t-h}) &= \gamma(h).\end{aligned}}$$

In other words, the mean and variance are constant through time, while the covariance depends only on the lag between observations.

Economic Interpretation: Shocks Die Out Suppose that an innovation ε_t occurs at time t.

Its effect on the process one period later is proportional to

$$\phi,$$

two periods later to

$$\phi^2,$$

and h periods later to

$$\phi^h.$$

Therefore,

$$\boxed{\text{Effect of a shock after } h \text{ periods} = \phi^h.}$$

If

$$|\phi| < 1,$$

then

$$\phi^h \to 0.$$

Hence, shocks have only a **temporary effect**: their influence progressively disappears over time.

This is closely related to the concept of **mean reversion**. The process tends to return toward its long-run mean

$$\boxed{\mu = \frac{c}{1-\phi}.}$$

What Happens When the Process Is Not Stationary? Consider the important special case

$$\phi = 1.$$

The AR(1) process becomes

$$X_t = X_{t-1} + \varepsilon_t,$$

assuming $c = 0$ for simplicity.

This is a **random walk**.

By recursive substitution,

$$X_t = X_0 + \varepsilon_1 + \varepsilon_2 + \cdots + \varepsilon_t,$$

or

$$\boxed{X_t = X_0 + \sum_{j=1}^{t} \varepsilon_j.}$$

Assuming X_0 is independent of the innovations,

$$\mathrm{Var}(X_t) = \mathrm{Var}(X_0) + \mathrm{Var}\left(\sum_{j=1}^{t} \varepsilon_j\right).$$

Since the innovations are uncorrelated,

$$\mathrm{Var}\left(\sum_{j=1}^{t} \varepsilon_j\right) = t\sigma_\varepsilon^2.$$

Therefore,

$$\boxed{\mathrm{Var}(X_t) = \mathrm{Var}(X_0) + t\sigma_\varepsilon^2.}$$

The variance now explicitly depends on time and grows as t increases. Therefore, the process is not stationary.

Moreover, when $\phi = 1$, the effect of a shock after h periods is

$$\phi^h = 1^h = 1.$$

Thus,

$$\boxed{\text{a shock has a permanent effect on a random walk.}}$$

This is fundamentally different from a stationary AR(1), where

$$|\phi| < 1 \quad \Longrightarrow \quad \phi^h \to 0.$$

Stationary AR(1) versus Random Walk

	$\lvert\phi\rvert < 1$	$\phi = 1$
Process	Stationary AR(1)	Random walk
Effect of shock after h periods	$\phi^h \to 0$	1
Mean reversion	Yes	No
Variance	$\dfrac{\sigma_\varepsilon^2}{1-\phi^2}$	Grows with t
Long-run distribution	Stable	Not stable

Therefore, the central intuition is

$$\boxed{\text{Stationarity ensures that the statistical dynamics of the process remain stable over time.}}$$

For autoregressive models in particular, stationarity ensures that

- the unconditional mean is constant over time;
- the unconditional variance is finite and constant over time;
- autocovariances depend only on the lag;
- shocks progressively disappear;
- the process exhibits stable long-run dynamics;
- relationships estimated from past observations remain meaningful for describing the same underlying data-generating process.

Extension to an AR(p) Process For a general AR(p) process,

$$X_t = c + \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-p} + \varepsilon_t,$$

we can write

$$\phi(L) X_t = c + \varepsilon_t,$$

where L is the lag operator and

$$\phi(L) = 1 - \phi_1 L - \phi_2 L^2 - \cdots - \phi_p L^p.$$

The process is stationary when the roots of the characteristic equation

$$\boxed{1 - \phi_1 z - \phi_2 z^2 - \cdots - \phi_p z^p = 0}$$

all lie **outside the unit circle**:

$$\boxed{|z_j| > 1 \qquad \text{for every root } z_j.}$$

For the AR(1) case,

$$1 - \phi z = 0,$$

so

$$z = \frac{1}{\phi}.$$

The condition

$$|z| > 1$$

is therefore equivalent to

$$\left|\frac{1}{\phi}\right| > 1 \quad \Longleftrightarrow \quad \boxed{|\phi| < 1.}$$

Reminder: Augmented Dickey–Fuller (ADF) Test

The **Augmented Dickey–Fuller (ADF) test** is commonly used to test whether a time series contains a **unit root**, and therefore whether it is non-stationary.

Consider first an AR(1) process:

$$X_t = \phi X_{t-1} + \varepsilon_t.$$

If

$$|\phi| < 1,$$

the process is stationary, whereas

$$\phi = 1$$

corresponds to a unit root:

$$X_t = X_{t-1} + \varepsilon_t,$$

which is a random walk and is non-stationary.

Subtracting X_{t-1} from both sides,

$$X_t - X_{t-1} = (\phi - 1)X_{t-1} + \varepsilon_t.$$

Defining

$$\Delta X_t = X_t - X_{t-1}$$

and

$$\gamma = \phi - 1,$$

we obtain the Dickey–Fuller regression

$$\boxed{\Delta X_t = \gamma X_{t-1} + \varepsilon_t.}$$

Testing whether

$$\phi = 1$$

is therefore equivalent to testing whether

$$\gamma = 0.$$

The hypotheses of the Dickey–Fuller test are

$$\boxed{\begin{array}{llll} H_0 : \gamma = 0 & \iff & \phi = 1 & \implies \text{unit root / non-stationary,} \\ H_1 : \gamma < 0 & \iff & \phi < 1 & \implies \text{stationary.} \end{array}}$$

The **Augmented** Dickey–Fuller test extends this regression by adding lagged differences of X_t:

$$\boxed{\Delta X_t = \alpha + \gamma X_{t-1} + \sum_{j=1}^{p} \delta_j \Delta X_{t-j} + \varepsilon_t.}$$

The lagged differences

$$\Delta X_{t-1}, \Delta X_{t-2}, \ldots, \Delta X_{t-p}$$

are included to account for additional serial correlation in the process, so that the remaining error term is approximately white noise.

Depending on the specification, a deterministic time trend may also be included:

$$\boxed{\Delta X_t = \alpha + \lambda t + \gamma X_{t-1} + \sum_{j=1}^{p} \delta_j \Delta X_{t-j} + \varepsilon_t.}$$

The choice between no constant, a constant, or a constant plus trend should reflect the deterministic components believed to be present in the series.

Interpretation of the Test The ADF test is a **unit-root test**. Its null hypothesis is

$$\boxed{H_0 : \text{the series has a unit root}}$$

against the alternative

$$\boxed{H_1 : \text{the series is stationary around the deterministic components included in the regression.}}$$

Therefore:

ADF result	Interpretation
p-value $< \alpha$	Reject $H_0 \Rightarrow$ evidence against a unit root
p-value $\geq \alpha$	Fail to reject $H_0 \Rightarrow$ insufficient evidence against a unit root

For example, using a 5% significance level,

$$p\text{-value} < 0.05$$

leads us to reject the unit-root null.

Importantly,

$$\boxed{p\text{-value} \geq 0.05 \quad \not\Rightarrow \quad \text{“the series is proven to be non-stationary.”}}$$

It only means that we do not have sufficient evidence to reject the unit-root null hypothesis.

ADF Test in Python Using `statsmodels`,

```
from statsmodels.tsa.stattools import adfuller

result = adfuller(x)
```

```
adf_stat = result[0]
p_value = result[1]
critical_values = result[4]

print("ADF statistic:", adf_stat)
print("p-value:", p_value)
print("Critical values:", critical_values)
```

The ADF statistic can also be compared directly with the appropriate Dickey–Fuller critical values:

$$\boxed{\text{ADF statistic} < \text{critical value} \quad \Longrightarrow \quad \text{reject } H_0.}$$

Here, "smaller" means **more negative**. For example,

$$-4.2 < -2.9$$

would lead to rejection at a significance level whose critical value is -2.9.

Key Point The ADF test should therefore be remembered as

$$\boxed{\begin{array}{c} H_0: \text{ Unit root / non-stationarity} \\ H_1: \text{ Stationarity} \\ p\text{-value small} \implies \text{reject the unit-root hypothesis.} \end{array}}$$

In financial applications, price levels often exhibit unit-root-like behavior, while returns or log returns are much more commonly modeled as stationary processes.

Order Selection in Autoregressive Models and the Partial Autocorrelation Function

When modeling a real time series using an AR model, the appropriate order p is generally unknown.

Recall that an AR(p) model is written as

$$\boxed{X_t = \phi_0 + \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-p} + \varepsilon_t.}$$

The order p determines how many past observations are directly used to predict the current observation X_t.

For example,

$$\text{AR}(1): \qquad X_t = \phi_0 + \phi_1 X_{t-1} + \varepsilon_t,$$

while

$$\text{AR}(3): \qquad X_t = \phi_0 + \phi_1 X_{t-1} + \phi_2 X_{t-2} + \phi_3 X_{t-3} + \varepsilon_t.$$

In practice, we observe the time series but do not know the true value of p. Determining the appropriate number of lags is called the **order selection problem**.

Common approaches include

- the **Partial Autocorrelation Function (PACF)**;
- the **Akaike Information Criterion (AIC)**;
- the **Bayesian Information Criterion (BIC)**.

Best Linear Prediction

To understand the PACF, we first introduce the concept of **best linear prediction**.

Let

$$X_1, \ldots, X_n, Y$$

be random variables.

We want to predict Y using a linear combination of $X_1, \ldots, X_n$:

$$\widehat{Y} = b_0 + b_1 X_1 + \cdots + b_n X_n.$$

Among all possible choices of coefficients

$$b_0, b_1, \ldots, b_n,$$

the best linear predictor is the one that minimizes the expected squared prediction error:

$$\boxed{L(Y \mid X_1, \ldots, X_n) = \arg \min_{\widehat{Y} = b_0 + b_1 X_1 + \cdots + b_n X_n} \mathbb{E}\left[(Y - \widehat{Y})^2\right].}$$

More explicitly, this corresponds to choosing

$$b_0^\star, b_1^\star, \ldots, b_n^\star$$

such that

$$(b_0^\star, \ldots, b_n^\star) = \arg \min_{b_0, \ldots, b_n} \mathbb{E}\left[(Y - b_0 - b_1 X_1 - \cdots - b_n X_n)^2\right].$$

The resulting prediction is

$$\boxed{L(Y \mid X_1, \ldots, X_n) = b_0^\star + b_1^\star X_1 + \cdots + b_n^\star X_n.}$$

The corresponding prediction error, or residual, is

$$\boxed{Y - L(Y \mid X_1, \ldots, X_n).}$$

It represents the part of Y that cannot be linearly explained by $X_1, \ldots, X_n$.

Partial Correlation

Suppose now that we want to measure the relationship between two random variables Y and Z, while removing the linear effects of $X_1, \ldots, X_n$.

First, predict Y using $X_1, \ldots, X_n$:

$$L(Y \mid X_1, \ldots, X_n).$$

The residual is

$$\boxed{e_Y = Y - L(Y \mid X_1, \ldots, X_n).}$$

Similarly, predict Z using the same variables:

$$L(Z \mid X_1, \ldots, X_n),$$

and define

$$\boxed{e_Z = Z - L(Z \mid X_1, \ldots, X_n).}$$

The residuals e_Y and e_Z represent the parts of Y and Z that are not linearly explained by $X_1, \ldots, X_n$.

The **partial correlation** between Y and Z, after controlling for $X_1, \ldots, X_n$, is defined as

$$\boxed{\operatorname{Corr}\left(Y - L(Y \mid X_1, \ldots, X_n),\, Z - L(Z \mid X_1, \ldots, X_n)\right).}$$

In other words,

$$\boxed{\text{Partial correlation} = \text{correlation between the residual parts of } Y \text{ and } Z.}$$

The intuition is important: instead of measuring the total correlation between Y and Z, we first remove the linear dependence that can be explained by the control variables.

Why Ordinary Autocorrelation Is Not Enough

Suppose that we want to understand the relationship between

$$X_t$$

and

$$X_{t-2}.$$

The ordinary autocorrelation

$$\rho_2 = \operatorname{Corr}(X_t, X_{t-2})$$

measures their total linear association.

However, part of this association may simply be transmitted through X_{t-1}:

$$\boxed{X_{t-2} \longrightarrow X_{t-1} \longrightarrow X_t.}$$

Therefore, X_t and X_{t-2} may be correlated even if X_{t-2} provides no additional linear predictive information once X_{t-1} is already known.

The PACF addresses precisely this issue.

At lag 2, it asks:

$$\boxed{\text{After controlling for } X_{t-1}, \text{ is there still a linear relationship between } X_t \text{ and } X_{t-2}\,?}$$

More generally, at lag n, it asks:

$$\boxed{\text{After controlling for } X_{t-1}, \ldots, X_{t-n+1}, \text{ does } X_{t-n} \text{ still provide additional linear information about } X_t\,?}$$

Partial Autocorrelation Function

For a stationary linear time series, consider the best linear prediction of X_t using its previous n observations:

$$L(X_t \mid X_{t-1}, \ldots, X_{t-n}).$$

Because the process is stationary, the coefficients of this linear projection do not depend on the particular date t.

We can therefore write

$$\boxed{L(X_t \mid X_{t-1}, \ldots, X_{t-n}) = \phi_{n0} + \phi_{n1}X_{t-1} + \cdots + \phi_{nn}X_{t-n}.}$$

The first index in

$$\phi_{nj}$$

indicates that we are using an n-lag prediction model, while the second index identifies the corresponding lag.

For example, with $n = 3$,

$$L(X_t \mid X_{t-1}, X_{t-2}, X_{t-3}) = \phi_{30} + \phi_{31}X_{t-1} + \phi_{32}X_{t-2} + \phi_{33}X_{t-3}.$$

The coefficient

$$\boxed{\phi_{nn}}$$

attached to the *last lag* X_{t-n} is the **partial autocorrelation coefficient at lag** n.

Therefore, the PACF is the sequence

$$\boxed{\phi_{11}, \phi_{22}, \phi_{33}, \ldots}$$

or, equivalently,

$$\boxed{\text{PACF}(n) = \phi_{nn}.}$$

Interpretation of ϕ_{nn}

The coefficient ϕ_{nn} measures the relationship between

$$X_t \qquad \text{and} \qquad X_{t-n}$$

after removing the linear effects of the intermediate lags

$$X_{t-1}, X_{t-2}, \ldots, X_{t-n+1}.$$

Thus,

$$\boxed{\phi_{nn} = \text{partial correlation between } X_t \text{ and } X_{t-n} \text{ controlling for } X_{t-1}, \ldots, X_{t-n+1}.}$$

Conceptually,

$$\boxed{\begin{array}{c}\text{Correlation at lag } n \\ \Downarrow \\ \text{Total linear association between } X_t \text{ and } X_{t-n}\end{array}}$$

whereas

$$\boxed{\begin{array}{c}\text{Partial correlation at lag } n \\ \Downarrow \\ \text{Direct additional linear association after removing lags } 1, \ldots, n-1.\end{array}}$$

Lag 1. At lag 1, there are no intermediate lags to remove.

Therefore,

$$\boxed{\phi_{11} = \text{Corr}(X_t, X_{t-1}) = \rho_1.}$$

Hence, the first PACF coefficient is equal to the first autocorrelation coefficient.

Lag 2. At lag 2,

$$\phi_{22}$$

measures the relationship between

$$X_t \qquad \text{and} \qquad X_{t-2}$$

after removing the linear effect of

$$X_{t-1}.$$

Thus,

$$\boxed{\phi_{22} = \text{Corr}\left(X_t - L(X_t \mid X_{t-1}),\, X_{t-2} - L(X_{t-2} \mid X_{t-1})\right).}$$

Lag 3. Similarly,

$$\phi_{33}$$

measures the relationship between

$$X_t \qquad \text{and} \qquad X_{t-3}$$

after removing the effects of

$$X_{t-1} \qquad \text{and} \qquad X_{t-2}.$$

Thus, the PACF isolates the **incremental information** associated with each additional lag.

The Cut-Off Property for an AR(p) Process

Suppose that the true process follows an AR(p) model:

$$\boxed{X_t = \phi_0 + \phi_1 X_{t-1} + \cdots + \phi_p X_{t-p} + \varepsilon_t, \qquad \phi_p \neq 0.}$$

The current observation X_t depends directly on the previous p observations:

$$X_{t-1}, X_{t-2}, \ldots, X_{t-p}.$$

Once these p lags have been included, older observations such as

$$X_{t-p-1}, X_{t-p-2}, \ldots$$

do not provide additional *linear* predictive information about X_t.

For example, consider an AR(2):

$$X_t = \phi_0 + \phi_1 X_{t-1} + \phi_2 X_{t-2} + \varepsilon_t.$$

The lag X_{t-3} may still be correlated with X_t. Indeed, information can propagate through the autoregressive dynamics:

$$X_{t-3} \longrightarrow X_{t-2} \longrightarrow X_{t-1} \longrightarrow X_t.$$

Therefore,

$$\mathrm{Corr}(X_t, X_{t-3})$$

does not necessarily equal zero.

However, once we control for the intermediate relevant lags, X_{t-3} does not provide an additional direct linear contribution to predicting X_t.

Consequently, for a theoretical AR(p) process,

$$\boxed{\phi_{kk} = 0, \qquad k > p.}$$

This is called the **cut-off property of the PACF**:

$$\boxed{\begin{array}{c} \mathrm{AR}(p) \\ \Downarrow \\ \text{PACF may be non-zero for lags } 1, \ldots, p \\ \text{and is zero for lags } k > p. \end{array}}$$

This property explains why the PACF is useful for selecting the order of an autoregressive model.

Population PACF versus Sample PACF

The theoretical PACF coefficient

$$\phi_{nn}$$

is a population quantity.

In practice, we observe only a finite sample

$$X_1, \ldots, X_T.$$

We therefore estimate the PACF coefficients:

$$\boxed{\hat{\phi}_{11}, \hat{\phi}_{22}, \hat{\phi}_{33}, \ldots}$$

The sequence

$$\boxed{\left\{\hat{\phi}_{nn}\right\}_{n \geq 1}}$$

is called the **sample partial autocorrelation function**.

An important distinction is that, even if the true process is AR(p),

$$\phi_{kk} = 0 \qquad \text{for } k > p,$$

but in a finite sample we generally do *not* observe

$$\hat{\phi}_{kk} = 0$$

exactly.

Instead, sampling noise produces small non-zero estimated values.

Therefore, in practice, we look for PACF coefficients that are **statistically significant**.

PACF Plot and Confidence Bands

A sample PACF plot typically displays

$$\hat{\phi}_{11}, \hat{\phi}_{22}, \ldots, \hat{\phi}_{KK}$$

together with approximate confidence bands around zero.

For large samples, a commonly used rough approximation is

$$\boxed{\pm \frac{1.96}{\sqrt{T}},}$$

corresponding approximately to a 95% confidence interval under suitable conditions.

A PACF coefficient lying clearly outside these bands is commonly interpreted as evidence that the corresponding partial autocorrelation is different from zero.

Thus, for an AR(p) process, we typically expect a pattern such as

Lag	1	2	$\cdots$	p	$p+1$	$p+2$	$\cdots$
PACF	possibly significant	possibly significant	$\cdots$	significant	≈ 0	≈ 0	$\cdots$

The PACF therefore provides a graphical method for identifying a plausible AR order p.

PACF in Python

The sample PACF can be estimated and plotted using `statsmodels`:

```
from statsmodels.graphics.tsaplots import plot_pacf
import matplotlib.pyplot as plt

plot_pacf(x, lags=20)
plt.show()
```

If the PACF shows significant coefficients at lags

$$1, 2, \ldots, p$$

followed by coefficients close to zero, this provides evidence in favor of an AR(p) specification.

ACF versus PACF

It is important to distinguish the autocorrelation function (ACF) from the partial autocorrelation function (PACF).

The ACF at lag k measures

$$\boxed{\rho_k = \text{Corr}(X_t, X_{t-k}),}$$

which includes both direct and indirect relationships.

The PACF at lag k measures the relationship between X_t and X_{t-k} after controlling for the intermediate lags

$$X_{t-1}, \ldots, X_{t-k+1}.$$

Hence,

ACF	PACF
Total correlation	Partial correlation
$\text{Corr}(X_t, X_{t-k})$	Controls for intermediate lags
Contains direct and indirect effects	Measures incremental linear association

For a pure AR(p) process, the characteristic identification pattern is

$$\boxed{\text{PACF cuts off after lag } p,}$$

while the ACF generally decays gradually rather than becoming exactly zero after lag p.

Order Selection Using AIC and BIC

The PACF provides a graphical approach to order selection. An alternative is to estimate several candidate AR models and compare information criteria.

For each candidate order

$$p = 0, 1, 2, \ldots, p_{\max},$$

we estimate an AR(p) model and compute, for example,

$$\boxed{\text{AIC} = -2\ell(\widehat{\theta}) + 2k,}$$

or

$$\boxed{\text{BIC} = -2\ell(\widehat{\theta}) + k \log T,}$$

where

$$\ell(\widehat{\theta})$$

is the maximized log-likelihood,

$$k$$

is the number of estimated parameters, and T is the number of observations.

Both criteria balance two objectives:

$$\boxed{\text{goodness of fit} \qquad \text{versus} \qquad \text{model complexity.}}$$

Adding more lags generally improves in-sample fit, but it also increases the number of parameters.

AIC and BIC therefore penalize unnecessarily complex models.

The selected order is typically

$$\boxed{\widehat{p}_{\text{AIC}} = \arg\min_{p} \text{AIC}(p)}$$

or

$$\boxed{\widehat{p}_{\text{BIC}} = \arg\min_{p} \text{BIC}(p).}$$

The BIC penalty,

$$k \log T,$$

increases more strongly with sample size than the AIC penalty $2k$. Consequently, BIC generally favors more parsimonious models in sufficiently large samples.

Summary

The logic behind AR order selection can be summarized as

We observe $X_1, \ldots, X_T$

$\Downarrow$

The true AR order p is unknown

$\Downarrow$

Measure the incremental contribution of each lag

$\Downarrow$

PACF at lag k = relationship between X_t and X_{t-k}
after controlling for lags $1, \ldots, k-1$

$\Downarrow$

For an AR(p): theoretical PACF cuts off after p

$\Downarrow$

Use the sample PACF and/or AIC/BIC to select a plausible p.

Interpretation of an Estimated AR(1) Model

The table reports the estimation of an **AR(1)** model using $T = 3938$ observations.

The estimated model is

$$\boxed{X_t = 0.0646 - 0.0889 X_{t-1} + \varepsilon_t,}$$

where the estimated standard deviation of the innovations is

$$\boxed{\widehat{\sigma}_\varepsilon = 1.738.}$$

Interpretation of the AR(1) Coefficient The estimated autoregressive coefficient is

$$\boxed{\widehat{\phi}_1 = -0.0889.}$$

Its standard error is

$$SE(\widehat{\phi}_1) = 0.016.$$

The corresponding z-statistic is

$$z = \frac{\widehat{\phi}_1}{SE(\widehat{\phi}_1)} = \frac{-0.0889}{0.016} \approx -5.602.$$

The reported p-value is

$$\boxed{p < 0.001.}$$

Therefore, we reject

$$H_0 : \phi_1 = 0$$

at conventional significance levels.

The 95% confidence interval is

$$\boxed{[-0.120, -0.058],}$$

which does not contain zero.

Hence, there is statistically significant evidence of a **negative first-order autoregressive relationship**.

Economically, the negative sign means that a positive deviation in the previous period tends to be followed by a movement in the opposite direction in the next period.

For example, ignoring the constant for intuition, if

$$X_{t-1} = 1,$$

the autoregressive component predicts

$$\widehat{\phi}_1 X_{t-1} = -0.0889.$$

Thus, the dependence is statistically significant but relatively small in magnitude:

$$|\widehat{\phi}_1| = 0.0889.$$

This distinction is important:

$$\boxed{\text{statistical significance} \neq \text{large economic magnitude.}}$$

Interpretation of the Constant The estimated constant is

$$\boxed{\widehat{\phi}_0 = 0.0646.}$$

Its standard error is 0.028, with

$$z = 2.330$$

and

$$p = 0.020.$$

Therefore, the constant is statistically significantly different from zero at the 5% level.

However, in an AR(1) model, the constant

$$\phi_0$$

should not generally be interpreted directly as the unconditional mean of the process.

For a stationary AR(1),

$$X_t = \phi_0 + \phi_1 X_{t-1} + \varepsilon_t,$$

taking expectations gives

$$\mu = \phi_0 + \phi_1 \mu.$$

Therefore,

$$\mu(1 - \phi_1) = \phi_0,$$

and hence

$$\boxed{\mu = \frac{\phi_0}{1 - \phi_1}.}$$

Using the estimated parameters,

$$\widehat{\mu} = \frac{0.0646}{1 - (-0.0889)} = \frac{0.0646}{1.0889} \approx 0.0593.$$

Thus, the estimated long-run mean of the process is approximately

$$\boxed{\widehat{\mu} \approx 0.059.}$$

Stationarity of the Estimated Process For an AR(1),

$$X_t = \phi_0 + \phi_1 X_{t-1} + \varepsilon_t,$$

stationarity requires

$$\boxed{|\phi_1| < 1.}$$

Here,

$$|\widehat{\phi}_1| = |-0.0889| = 0.0889 < 1.$$

Therefore, the fitted AR(1) dynamics satisfy the stationarity condition.

The negative coefficient also implies an **oscillating mean-reversion pattern**.

Indeed, the effect of a shock after h periods is proportional to

$$\phi_1^h.$$

For example,

$$\phi_1 = -0.0889,$$

so

$$\phi_1^1 = -0.0889,$$

$$\phi_1^2 \approx 0.0079,$$

$$\phi_1^3 \approx -0.0007.$$

The sign alternates because $\phi_1 < 0$, while the magnitude decreases very quickly because

$$|\phi_1| \ll 1.$$

Thus,

$$\boxed{\text{shocks are highly transitory and disappear rapidly.}}$$

Interpretation of the AR Root The table reports the AR root

$$\boxed{z = -11.2472.}$$

Recall that for an AR(1), the autoregressive polynomial is

$$1 - \phi_1 z.$$

Its root satisfies

$$1 - \phi_1 z = 0,$$

and therefore

$$z = \frac{1}{\phi_1}.$$

Using the estimated coefficient,

$$\frac{1}{-0.0889} \approx -11.25,$$

which corresponds to the reported root.

Its modulus is

$$\boxed{|z| = 11.2472.}$$

For stationarity, the roots of the AR polynomial must lie **outside the unit circle**:

$$|z| > 1.$$

Since

$$11.2472 > 1,$$

the root confirms that the estimated AR(1) process is stationary.

Notice that the two stationarity conditions are equivalent:

$$\boxed{|\phi_1| < 1 \quad \Longleftrightarrow \quad \left|\frac{1}{\phi_1}\right| > 1.}$$

Standard Deviation of the Innovations The reported standard deviation of the innovations is

$$\boxed{\widehat{\sigma}_\varepsilon = 1.738.}$$

This measures the typical magnitude of the unpredictable component

$$\varepsilon_t$$

remaining after accounting for the AR(1) dynamics.

The estimated conditional model can therefore be summarized as

$$\boxed{X_t = 0.0646 - 0.0889 X_{t-1} + \varepsilon_t, \qquad \sigma_\varepsilon \approx 1.738.}$$

Log-Likelihood, AIC and BIC The estimated model has log-likelihood

$$\boxed{\ell(\widehat{\theta}) = -7761.702.}$$

The table also reports

$$\boxed{\text{AIC} = 15529.404,}$$

$$\boxed{\text{BIC} = 15548.239,}$$

and

$$\boxed{\text{HQIC} = 15536.086.}$$

These quantities are primarily useful for **comparing alternative model specifications**.

For example, one could estimate

$$\text{AR(1)}, \quad \text{AR(2)}, \quad \text{AR(3)}, \quad \ldots$$

and compare their information criteria.

The preferred specification according to a given criterion is the model with the **lowest** value:

$$\boxed{\widehat{p}_{AIC} = \arg\min_{p} AIC(p),}$$

and similarly,

$$\boxed{\widehat{p}_{BIC} = \arg\min_{p} BIC(p).}$$

Therefore, the value

$$AIC = 15529.404$$

does not have a meaningful interpretation by itself. It becomes useful when compared with the AIC of alternative models.

Interpretation of the Estimated ARIMA(1,0,0) Model

The table reports the estimation of an

$$\boxed{\text{ARIMA}(1,0,0)}$$

model using

$$\boxed{T = 3938}$$

observations.

Since the integration order is

$$d = 0,$$

no differencing is applied to the series. Moreover, since

$$q = 0,$$

there is no moving-average component.

Therefore,

$$\boxed{\text{ARIMA}(1,0,0) \equiv \text{AR}(1).}$$

The estimated model can be written as

$$\boxed{X_t = 0.0597 - 0.0889 X_{t-1} + \varepsilon_t,}$$

with estimated innovation variance

$$\boxed{\widehat{\sigma}_\varepsilon^2 = 3.0196.}$$

Hence, the estimated innovation standard deviation is

$$\widehat{\sigma}_\varepsilon = \sqrt{3.0196} \approx 1.738.$$

Interpretation of the AR(1) Coefficient The estimated autoregressive coefficient is

$$\boxed{\widehat{\phi}_1 = -0.0889.}$$

Its estimated standard error is

$$SE(\widehat{\phi}_1) = 0.007.$$

The corresponding z-statistic is

$$z = -11.859,$$

with

$$\boxed{p < 0.001.}$$

Therefore, we strongly reject

$$H_0 : \phi_1 = 0.$$

The 95% confidence interval is

$$\boxed{[-0.104, -0.074],}$$

which does not contain zero.

There is therefore strong statistical evidence of a **negative first-order autoregressive relationship**.

The negative sign means that positive deviations tend to be followed by movements in the opposite direction, and vice versa.

For example, ignoring the constant for intuition, if

$$X_{t-1} = 1,$$

the autoregressive contribution to the conditional expectation is

$$\widehat{\phi}_1 X_{t-1} = -0.0889.$$

However, although the coefficient is highly statistically significant, its absolute magnitude is relatively small:

$$\boxed{|\widehat{\phi}_1| = 0.0889.}$$

Thus,

$$\boxed{\text{statistical significance} \neq \text{large economic or predictive magnitude.}}$$

Conditional Expectation Under the fitted AR(1) model,

$$X_t = 0.0597 - 0.0889X_{t-1} + \varepsilon_t,$$

and assuming

$$\mathbb{E}[\varepsilon_t \mid \mathcal{F}_{t-1}] = 0,$$

the one-step-ahead conditional expectation is

$$\boxed{\mathbb{E}\left[X_t \mid \mathcal{F}_{t-1}\right] = 0.0597 - 0.0889X_{t-1}.}$$

Therefore, the previous observation contains statistically significant information about the expected value of the next observation.

However, because

$$|\widehat{\phi}_1|$$

is small, this predictable component is quantitatively limited.

Interpretation of the Constant in `statsmodels ARIMA` An important point concerns the parameterization used by

```
from statsmodels.tsa.arima.model import ARIMA

ar_model = ARIMA(jpm_return, order=(1, 0, 0)).fit()
```

With

```
ARIMA(jpm_return, order=(1, 0, 0))
```

`statsmodels` parameterizes the AR(1) model as

$$\boxed{X_t - \mu = \phi_1(X_{t-1} - \mu) + \varepsilon_t,}$$

where the parameter reported as `const` corresponds to the long-run mean μ.

Therefore, in the present output,

$$\boxed{\widehat{\mu} = \texttt{const} = 0.0597.}$$

Thus, we should **not** compute

$$\frac{0.0597}{1 - (-0.0889)}$$

to obtain the unconditional mean: the reported constant is already the estimated unconditional mean.

To connect this parameterization with the conventional AR(1) representation,

$$X_t = c + \phi_1 X_{t-1} + \varepsilon_t,$$

expand the `statsmodels` representation:

$$X_t - \mu = \phi_1 X_{t-1} - \phi_1 \mu + \varepsilon_t.$$

Adding μ to both sides gives

$$X_t = \mu - \phi_1 \mu + \phi_1 X_{t-1} + \varepsilon_t,$$

and therefore

$$X_t = \mu(1 - \phi_1) + \phi_1 X_{t-1} + \varepsilon_t.$$

Hence, the intercept c of the conventional AR(1) representation is

$$\boxed{c = \mu(1 - \phi_1).}$$

Using the estimated parameters,

$$\widehat{\mu} = 0.0597, \qquad \widehat{\phi}_1 = -0.0889,$$

we obtain

$$\widehat{c} = 0.0597\,(1 - (-0.0889))\,,$$

so that

$$\boxed{\widehat{c} \approx 0.0650.}$$

Consequently, the two equivalent representations of the fitted model are

$$\boxed{X_t - 0.0597 = -0.0889\,(X_{t-1} - 0.0597) + \varepsilon_t,}$$

or, after expansion,

$$\boxed{X_t \approx 0.0650 - 0.0889 X_{t-1} + \varepsilon_t.}$$

Therefore,

$$\boxed{\underbrace{0.0597}_{\texttt{const in statsmodels ARIMA}} = \underbrace{\mu}_{\text{unconditional mean}},}$$

whereas

$$\boxed{\underbrace{0.0650}_{c} = \underbrace{\mu(1-\phi_1)}_{\text{AR intercept}}.}$$

This distinction is important when comparing the output of `statsmodels.tsa.arima.model.ARIMA` with other AR estimation functions, since the same word "constant" may refer to different parameterizations.

Stationarity and Mean Reversion For an AR(1) process,

$$X_t = c + \phi_1 X_{t-1} + \varepsilon_t,$$

the stationarity condition is

$$\boxed{|\phi_1| < 1.}$$

Here,

$$|\widehat{\phi}_1| = 0.0889 < 1.$$

Therefore,

$$\boxed{\text{the estimated AR dynamics are stationary.}}$$

The effect of a shock h periods into the future is proportional to

$$\phi_1^h.$$

Since

$$\widehat{\phi}_1 = -0.0889,$$

we obtain approximately

$$\widehat{\phi}_1^1 = -0.0889,$$

$$\widehat{\phi}_1^2 = 0.0079,$$

$$\widehat{\phi}_1^3 = -0.0007.$$

Thus, the response alternates in sign because

$$\widehat{\phi}_1 < 0,$$

but its magnitude decreases extremely rapidly because

$$|\widehat{\phi}_1| \ll 1.$$

Hence,

$$\boxed{\text{shocks are highly transitory and the process mean-reverts rapidly.}}$$

Innovation Variance The parameter

$$\boxed{\widehat{\sigma}^2 = 3.0196}$$

is the estimated variance of the innovations.

Its 95% confidence interval is

$$\boxed{[2.960, 3.080].}$$

Therefore, the corresponding innovation standard deviation is

$$\boxed{\widehat{\sigma} = \sqrt{3.0196} \approx 1.738.}$$

This measures the typical magnitude of the unpredictable component

$$\varepsilon_t$$

after accounting for the AR(1) dependence.

Since the autoregressive coefficient is relatively small while the innovation standard deviation is approximately 1.738, most of the period-to-period variation remains associated with the innovation term.

Log-Likelihood and Information Criteria The maximized log-likelihood is

$$\boxed{\ell(\widehat{\theta}) = -7763.745.}$$

The reported information criteria are

$$\boxed{AIC = 15533.491,}$$

$$\boxed{BIC = 15552.326,}$$

and

$$\boxed{HQIC = 15540.173.}$$

These quantities are useful primarily for comparing alternative model specifications.

For example, one could estimate

$$ARIMA(0,0,0), \quad ARIMA(1,0,0), \quad ARIMA(2,0,0), \quad ARIMA(3,0,0),$$

and select the specification minimizing a chosen information criterion.

For instance,

$$\boxed{\widehat{p}_{AIC} = \arg\min_{p} AIC(p).}$$

Similarly,

$$\boxed{\widehat{p}_{BIC} = \arg\min_{p} BIC(p).}$$

AIC and BIC balance

$$\boxed{\text{quality of fit} \qquad \text{against} \qquad \text{model complexity.}}$$

The numerical values themselves should therefore not generally be interpreted in isolation.

Residual Diagnostics

An important advantage of this output is that it also reports several diagnostic tests on the estimated residuals.

A well-specified time-series model should ideally leave residuals that behave approximately like innovations:

$$\boxed{\text{Residuals} \approx \text{unpredictable shocks.}}$$

The diagnostics therefore help determine whether the AR(1) model has successfully captured the systematic dynamics of the series.

Ljung–Box Test for Residual Autocorrelation The table reports

$$\boxed{Q(1) = 0.12}$$

with

$$\boxed{p = 0.73.}$$

The Ljung–Box test examines whether residual autocorrelations are zero.

At lag 1, the hypotheses are approximately

$$\boxed{\begin{aligned} &H_0 : \text{no residual autocorrelation up to the tested lag,} \\ &H_1 : \text{residual autocorrelation remains.} \end{aligned}}$$

Since

$$p = 0.73 > 0.05,$$

we fail to reject H_0.

Thus,

$$\boxed{\text{there is no evidence of remaining first-order residual autocorrelation.}}$$

This is a positive diagnostic result: the AR(1) component appears to have captured the lag-one linear dependence present in the conditional mean.

However, the reported test concerns only lag 1. A more complete residual analysis should examine the Ljung–Box statistic over several lags.

Jarque–Bera Test for Normality The Jarque–Bera statistic is

$$\boxed{JB = 12660.48,}$$

with

$$\boxed{p < 0.001.}$$

The hypotheses are

$$\boxed{\begin{aligned} &H_0 : \text{residuals are normally distributed,} \\ &H_1 : \text{residuals are not normally distributed.} \end{aligned}}$$

Since the p-value is essentially zero,

$$\boxed{\text{we strongly reject residual normality.}}$$

The table provides additional information explaining this result.

Skewness The estimated residual skewness is

$$\boxed{\text{Skew} = -0.16.}$$

For a perfectly symmetric distribution,

$$\text{Skew} = 0.$$

The value

$$-0.16$$

indicates a relatively mild negative asymmetry, meaning that the left tail is somewhat more pronounced than the right tail.

Kurtosis The reported kurtosis is

$$\boxed{\text{Kurtosis} = 11.78.}$$

For a normal distribution,

$$\text{Kurtosis} = 3.$$

Here,

$$11.78 \gg 3.$$

The residual distribution therefore exhibits very high kurtosis.

This indicates

$$\boxed{\text{heavy tails and substantially more extreme observations than under a Gaussian distribution.}}$$

The very high kurtosis is likely the main reason for the extremely large Jarque–Bera statistic.

This behavior is common in financial time series, where extreme observations occur more frequently than predicted by a normal distribution.

Heteroskedasticity Test The table reports

$$\boxed{H = 0.85}$$

with

$$\boxed{p < 0.001.}$$

The null hypothesis of the reported heteroskedasticity diagnostic is that the residual variance is stable across the portions of the sample being compared.

Since

$$p < 0.05,$$

we reject the null of constant variance.

Thus, the residuals provide evidence of

$$\boxed{\text{heteroskedasticity.}}$$

In other words, although the AR(1) specification appears to remove the first-order dependence in the *conditional mean*, the variability of the residuals is not constant over time.

This distinction is important:

$$\boxed{\text{modeling the conditional mean} \neq \text{modeling the conditional variance.}}$$

The AR(1) model specifies

$$\mathbb{E}[X_t \mid \mathcal{F}_{t-1}],$$

but does not by itself capture time variation in

$$\mathrm{Var}(X_t \mid \mathcal{F}_{t-1}).$$

For financial time series, such behavior can motivate models of conditional volatility such as ARCH or GARCH.

Are the Residuals White Noise? The diagnostics illustrate an important distinction.

The Ljung–Box result suggests that

$$\mathrm{Corr}(\widehat{\varepsilon}_t, \widehat{\varepsilon}_{t-1}) \approx 0,$$

so there is no evidence of remaining first-order linear autocorrelation.

However,

$$\text{Jarque–Bera } p < 0.001$$

and

$$\text{heteroskedasticity } p < 0.001.$$

Therefore, the residuals should not be described as **i.i.d. Gaussian white noise**.

We instead have evidence consistent with

little remaining first-order linear autocorrelation but non-normality and non-constant variance.

Consequently, checking only residual autocorrelation is not sufficient for a complete model diagnostic.

Why Do `AutoReg` and `ARIMA(1,0,0)` Produce Similar Coefficients but Different Likelihoods, AICs, and BICs?

An AR(1) model can be estimated in `statsmodels` using, among others, the following two approaches:

```
from statsmodels.tsa.ar_model import AutoReg

ar_model = AutoReg(jpm_return, lags=1).fit()
```

or

```
from statsmodels.tsa.arima.model import ARIMA

ar_model = ARIMA(jpm_return, order=(1, 0, 0)).fit()
```

Since

$$\text{ARIMA}(1,0,0) \equiv \text{AR}(1),$$

both approaches estimate essentially the same autoregressive dynamics.

In the present example, both methods produce approximately

$$\boxed{\widehat{\phi}_1 = -0.0889.}$$

However, the reported log-likelihoods, AICs, and BICs are slightly different. The main reason is that the two implementations do not construct the likelihood in exactly the same way.

Different Parameterizations of the Constant The `AutoReg` representation is naturally written as

$$\boxed{X_t = c + \phi_1 X_{t-1} + \varepsilon_t.}$$

The estimation gives approximately

$$\widehat{c} = 0.0646, \qquad \widehat{\phi}_1 = -0.0889.$$

By contrast, with

`ARIMA(jpm_return, order=(1, 0, 0))`

the stationary AR(1) component is parameterized around its long-run mean μ:

$$\boxed{X_t - \mu = \phi_1(X_{t-1} - \mu) + \varepsilon_t.}$$

The reported `const` is therefore approximately

$$\widehat{\mu} = 0.0597,$$

while

$$\widehat{\phi}_1 = -0.0889.$$

Expanding the mean-centered representation,

$$X_t - \mu = \phi_1 X_{t-1} - \phi_1 \mu + \varepsilon_t,$$

so

$$X_t = \mu(1 - \phi_1) + \phi_1 X_{t-1} + \varepsilon_t.$$

Consequently, the two parameterizations are related through

$$\boxed{c = \mu(1 - \phi_1).}$$

Using the ARIMA estimates,

$$\widehat{c} = 0.0597\,[1 - (-0.0889)]\,,$$

which gives approximately

$$\boxed{\widehat{c} \approx 0.0650.}$$

This is extremely close to the intercept estimated by `AutoReg`:

$$\boxed{0.0650 \approx 0.0646.}$$

Thus, the apparent difference in the reported constants comes mainly from the different parameterizations rather than from fundamentally different estimated dynamics.

`AutoReg`: Conditional Likelihood The more important difference concerns the treatment of the initial observation.

For an AR(1),

$$X_t = c + \phi_1 X_{t-1} + \varepsilon_t,$$

the conditional distribution of X_t depends on X_{t-1}.

For example,

$$X_2$$

can be modeled conditional on X_1,

$$X_3$$

conditional on X_2, and so forth.

However, modeling the first observation X_1 through the AR equation would require an earlier observation X_0, which is not available in the sample.

The `AutoReg` estimation uses a conditional likelihood approach. It conditions on the required initial observation and constructs the likelihood from the subsequent observations.

For an AR(1), this can be represented conceptually as

$$\boxed{L_{\text{conditional}} = \prod_{t=2}^{T} f(X_t \mid X_{t-1}).}$$

Equivalently, the log-likelihood is

$$\boxed{\ell_{\text{conditional}} = \sum_{t=2}^{T} \log f(X_t \mid X_{t-1}).}$$

The initial observation X_1 is used as an initial condition, but its own probability distribution does not contribute in the same way to this conditional likelihood.

This is why the `AutoReg` output explicitly reports

```
Method: Conditional MLE
```

and why the effective estimation sample begins after the observations needed to construct the lags.

`ARIMA`: State-Space Likelihood The class

```
statsmodels.tsa.arima.model.ARIMA
```

is implemented using the state-space machinery underlying the SARIMAX framework.

The likelihood therefore handles the initialization of the stochastic process differently from the conditional likelihood used by `AutoReg`.

For a stationary Gaussian AR(1),

$$X_t - \mu = \phi_1(X_{t-1} - \mu) + \varepsilon_t,$$

with

$$\varepsilon_t \sim \mathcal{N}(0, \sigma_\varepsilon^2),$$

the stationary distribution has variance

$$\boxed{\mathrm{Var}(X_t) = \frac{\sigma_\varepsilon^2}{1 - \phi_1^2}.}$$

Hence, conceptually, the initial observation is associated with the stationary distribution

$$X_1 \sim \mathcal{N}\left(\mu, \frac{\sigma_\varepsilon^2}{1 - \phi_1^2}\right).$$

The likelihood can therefore be understood schematically as

$$\boxed{L_{\mathrm{ARIMA}} = f(X_1) \prod_{t=2}^{T} f(X_t \mid X_{t-1}),}$$

rather than simply conditioning on X_1.

Thus, the likelihood functions optimized by `AutoReg` and `ARIMA` are not numerically identical.

Consequences for the Log-Likelihood In the present example, the reported log-likelihoods are approximately

$$\boxed{\ell_{\mathrm{AutoReg}} = -7761.702,}$$

and

$$\boxed{\ell_{\mathrm{ARIMA}} = -7763.745.}$$

The difference does not imply that the two models have discovered substantially different autoregressive dynamics.

Indeed, both estimate approximately

$$\widehat{\phi}_1 = -0.0889.$$

Instead, the likelihood values differ because the likelihood functions, initialization conventions, and effective treatment of the beginning of the sample differ.

Why Are the Estimated AR Coefficients Almost Identical? The sample contains

$$T = 3938$$

observations.

The treatment of a very small number of initial observations therefore has little influence on the estimated autoregressive relationship when compared with the information contained in almost 4000 observations.

Consequently,

$$\boxed{\widehat{\phi}_{1,\text{AutoReg}} \approx \widehat{\phi}_{1,\text{ARIMA}} \approx -0.0889.}$$

In a much smaller sample, differences caused by initialization and likelihood construction could be more noticeable.

Thus,

$$\boxed{\text{same underlying AR dynamics} \quad \not\Rightarrow \quad \text{exactly the same likelihood value.}}$$

Consequences for the AIC Recall that the Akaike Information Criterion is

$$\boxed{AIC = -2\ell(\widehat{\theta}) + 2k,}$$

where

$$\ell(\widehat{\theta})$$

is the maximized log-likelihood and k is the number of estimated parameters counted by the likelihood specification.

For the ARIMA model, the estimated parameters are

$$\mu, \qquad \phi_1, \qquad \sigma_\varepsilon^2.$$

Hence,

$$k = 3.$$

Using

$$\ell(\widehat{\theta}) = -7763.745,$$

we obtain

$$AIC = -2(-7763.745) + 2(3),$$

and therefore

$$AIC = 15527.490 + 6 = 15533.490.$$

This corresponds, up to rounding, to the reported value

$$\boxed{AIC_{\text{ARIMA}} = 15533.491.}$$

For `AutoReg`, the reported value is

$$\boxed{AIC_{\text{AutoReg}} = 15529.404.}$$

Since the underlying likelihood value and estimation conventions differ, the resulting AIC values also differ.

Therefore,

$$\boxed{\text{different likelihood} \Longrightarrow \text{different AIC.}}$$

Consequences for the BIC The Bayesian Information Criterion is generally written as

$$\boxed{BIC = -2\ell(\widehat{\theta}) + k\log(n),}$$

where n denotes the number of observations entering the relevant likelihood calculation.

The two implementations report

$$\boxed{BIC_{\text{AutoReg}} = 15548.239,}$$

and

$$\boxed{BIC_{\text{ARIMA}} = 15552.326.}$$

Again, the difference comes from the fact that the implementations use different likelihood constructions and treatment of the initial observations, which affects the quantities entering the information criterion.

The same reasoning applies to other likelihood-based criteria such as HQIC.

Important: Comparing AIC and BIC Across Implementations One should therefore be careful when directly comparing

$$AIC_{\text{AutoReg}}$$

with

$$AIC_{\text{ARIMA}}.$$

For example, observing that

$$AIC_{\text{AutoReg}} < AIC_{\text{ARIMA}}$$

should **not** immediately be interpreted as evidence that `AutoReg` provides a better model.

The reason is that AIC comparisons are most meaningful when the competing models are estimated

- on the same data;
- using a consistent likelihood definition;
- with the same treatment of initial observations;
- under comparable distributional assumptions.

Here, the two AIC values are produced under different likelihood conventions.

Therefore,

$$\boxed{AIC_{\text{AutoReg}} < AIC_{\text{ARIMA}} \quad \not\Rightarrow \quad \texttt{AutoReg} \text{ is a better AR(1) model.}}$$

Implication for AR Order Selection Suppose the objective is to determine the appropriate autoregressive order

$$p.$$

A consistent approach would be to estimate

$$ARIMA(1,0,0), \quad ARIMA(2,0,0), \quad ARIMA(3,0,0), \quad \ldots$$

using the same `ARIMA` implementation and then compare

$$AIC(1), AIC(2), AIC(3), \ldots$$

or

$$BIC(1), BIC(2), BIC(3), \ldots$$

within that common framework.

For example,

$$\boxed{\widehat{p}_{AIC} = \arg\min_{p} AIC\left(ARIMA(p,0,0)\right).}$$

Alternatively, all candidate AR models could be estimated consistently using `AutoReg`.

The important principle is

$$\boxed{\text{compare information criteria within a common estimation and likelihood framework.}}$$

Summary

The relationship between the two approaches can be summarized as follows:

	`AutoReg`	`ARIMA(1,0,0)`
Underlying dynamics	$AR(1)$	$AR(1)$
AR coefficient	≈ -0.0889	≈ -0.0889
Reported constant	AR intercept c	long-run mean μ
Relation	$c = \mu(1 - \phi_1)$	
Likelihood treatment	Conditional	State-space
Initial observations	Conditioned upon	Handled through initialization
Log-likelihood	-7761.702	-7763.745
AIC	15529.404	15533.491
BIC	15548.239	15552.326

Hence, the key conclusion is

`AutoReg` and `ARIMA(1,0,0)`
estimate essentially the same AR(1) dynamics,
but use different likelihood and initialization conventions,
$\Downarrow$
very similar coefficient estimates
but different log-likelihood, AIC, and BIC values.

Different Conventions for AIC and BIC

Different textbooks and software packages may report apparently different formulas for the Akaike Information Criterion (AIC) and the Bayesian Information Criterion (BIC).

The standard definitions commonly used by statistical software such as `statsmodels` are

$$\boxed{AIC = -2\ell(\widehat{\theta}) + 2k,}$$

and

$$\boxed{BIC = -2\ell(\widehat{\theta}) + k \ln T,}$$

where

$$\ell(\widehat{\theta}) = \ln L(\widehat{\theta})$$

is the maximized log-likelihood,

$$k$$

is the number of estimated parameters, and

$$T$$

is the number of observations.

However, another common convention, particularly in time-series textbooks, is to express these criteria **per observation**.

Normalized AIC and BIC Dividing the standard AIC by the sample size T gives

$$\frac{AIC}{T} = -\frac{2}{T}\ell(\widehat{\theta}) + \frac{2k}{T}.$$

Therefore, the normalized AIC can be written as

$$\boxed{AIC_{\text{norm}} = -\frac{2}{T}\ln L(\widehat{\theta}) + \frac{2k}{T}.}$$

Similarly, dividing BIC by T gives

$$\boxed{BIC_{\text{norm}} = -\frac{2}{T}\ln L(\widehat{\theta}) + \frac{k\ln T}{T}.}$$

Thus,

$$\boxed{AIC_{\text{norm}} = \frac{AIC_{\text{standard}}}{T},}$$

and

$$\boxed{BIC_{\text{norm}} = \frac{BIC_{\text{standard}}}{T}.}$$

The two conventions therefore differ only by a positive scaling factor $1/T$, provided the same sample size is used for all candidate models.

Why Does the Normalization Not Affect Model Selection? Suppose that several candidate models are indexed by their order p.

The standard AIC selects

$$\widehat{p} = \arg\min_{p} AIC(p).$$

Since $T > 0$, dividing every criterion by the same positive constant does not change their ordering:

$$AIC(p_1) < AIC(p_2)$$

if and only if

$$\frac{AIC(p_1)}{T} < \frac{AIC(p_2)}{T}.$$

Therefore,

$$\boxed{\arg\min_p AIC(p) = \arg\min_p \frac{AIC(p)}{T}.}$$

The same reasoning applies to BIC:

$$\boxed{\arg\min_p BIC(p) = \arg\min_p \frac{BIC(p)}{T}.}$$

Hence, when all models are estimated on the same sample,

$$\boxed{\text{normalizing AIC or BIC by } T \text{ does not change the selected model.}}$$

Simplification for a Gaussian AR Model

For an AR(p) model,

$$X_t = \phi_0 + \phi_1 X_{t-1} + \cdots + \phi_p X_{t-p} + \varepsilon_t,$$

suppose that the innovations are independent Gaussian random variables:

$$\boxed{\varepsilon_t \overset{iid}{\sim} \mathcal{N}(0, \sigma^2).}$$

The Gaussian log-likelihood can be written, under the usual conditional likelihood formulation, as

Step 1: Probability Density of One Innovation Recall that if a random variable Z follows

$$Z \sim \mathcal{N}(\mu, \sigma^2),$$

its probability density function is

$$f_Z(z) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(z-\mu)^2}{2\sigma^2}\right).$$

Here,

$$\varepsilon_t \sim \mathcal{N}(0, \sigma^2),$$

so

$$\mu = 0.$$

Therefore,

$$\boxed{f_\varepsilon(\varepsilon_t) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\varepsilon_t^2}{2\sigma^2}\right).}$$

Equivalently,

$$f_\varepsilon(\varepsilon_t) = (2\pi\sigma^2)^{-1/2} \exp\left(-\frac{\varepsilon_t^2}{2\sigma^2}\right).$$

Step 2: Innovations Are Prediction Errors From the AR(p) equation,

$$X_t = \phi_0 + \sum_{j=1}^{p} \phi_j X_{t-j} + \varepsilon_t,$$

we can isolate the innovation:

$$\boxed{\varepsilon_t = X_t - \phi_0 - \sum_{j=1}^{p} \phi_j X_{t-j}.}$$

Therefore, for a given parameter vector

$$\theta = (\phi_0, \phi_1, \ldots, \phi_p, \sigma^2),$$

the innovation is a function of both the observed data and the parameters:

$$\varepsilon_t = \varepsilon_t(\theta).$$

It represents the difference between the observed value X_t and the value predicted by the autoregressive component:

$$\boxed{\varepsilon_t = X_t - \mathbb{E}[X_t \mid X_{t-1}, \ldots, X_{t-p}].}$$

Under the Gaussian assumption,

$$X_t \mid X_{t-1}, \ldots, X_{t-p} \sim \mathcal{N}\left(\phi_0 + \sum_{j=1}^{p} \phi_j X_{t-j}, \sigma^2\right).$$

Derivation:

Condition on the Past:

Start from

$$X_t = \phi_0 + \sum_{j=1}^{p} \phi_j X_{t-j} + \varepsilon_t.$$

Before conditioning, the lagged variables

$$X_{t-1}, \ldots, X_{t-p}$$

are themselves random variables.

However, suppose that we condition on their observed values.

Then

$$X_{t-1}, \ldots, X_{t-p}$$

are treated as **known quantities**.

Consequently,

$$\phi_0 + \sum_{j=1}^{p} \phi_j X_{t-j}$$

is also known once the past has been observed.

Define

$$\boxed{a_t = \phi_0 + \sum_{j=1}^{p} \phi_j X_{t-j}.}$$

Conditionally on the past, a_t is therefore a constant.

The AR model becomes

$$\boxed{X_t = a_t + \varepsilon_t.}$$

Thus, once we know the past, the only remaining source of randomness in X_t is the new innovation

$$\varepsilon_t.$$

Adding a Constant to a Gaussian Random Variable:

Recall the following fundamental property.

If

$$Z \sim \mathcal{N}(\mu, \sigma^2),$$

then, for any constant a,

$$a + Z \sim \mathcal{N}(a + \mu, \sigma^2).$$

In particular, if

$$Z \sim \mathcal{N}(0, \sigma^2),$$

then

$$\boxed{a + Z \sim \mathcal{N}(a, \sigma^2).}$$

Adding a constant shifts the center of the Gaussian distribution but does not change its variance.

In our AR model,

$$\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$$

and, conditionally on the past,

$$X_t = a_t + \varepsilon_t.$$

Therefore,

$$X_t \mid X_{t-1}, \ldots, X_{t-p} \sim \mathcal{N}(a_t, \sigma^2).$$

Replacing a_t by its definition gives

$$\boxed{X_t \mid X_{t-1}, \ldots, X_{t-p} \sim \mathcal{N}\left(\phi_0 + \sum_{j=1}^{p} \phi_j X_{t-j}, \sigma^2\right).}$$

Step 3: Conditional Density of X_t The conditional density of X_t, given its previous observations, is

$$f\left(X_t \mid X_{t-1}, \ldots, X_{t-p}; \theta\right)$$

$$= \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{\left(X_t - \phi_0 - \sum_{j=1}^{p} \phi_j X_{t-j}\right)^2}{2\sigma^2}\right].$$

Since

$$\varepsilon_t = X_t - \phi_0 - \sum_{j=1}^{p} \phi_j X_{t-j},$$

this becomes

$$\boxed{f\left(X_t \mid X_{t-1}, \ldots, X_{t-p}; \theta\right) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\varepsilon_t^2}{2\sigma^2}\right).}$$

Step 4: Construct the Likelihood Because the innovations are assumed independent,

$$\varepsilon_t \overset{iid}{\sim} \mathcal{N}(0, \sigma^2),$$

the joint likelihood is obtained by multiplying the individual conditional densities:

$$L(\theta) = \prod_{t=1}^{T} f_\varepsilon(\varepsilon_t).$$

Therefore,

$$L(\theta) = \prod_{t=1}^{T} \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\varepsilon_t^2}{2\sigma^2} \right) \right].$$

We can separate the two components of the product:

$$L(\theta) = \left[\prod_{t=1}^{T} \frac{1}{\sqrt{2\pi\sigma^2}} \right] \left[\prod_{t=1}^{T} \exp\left(-\frac{\varepsilon_t^2}{2\sigma^2} \right) \right].$$

Since the first term is identical for every observation,

$$\prod_{t=1}^{T} \frac{1}{\sqrt{2\pi\sigma^2}} = \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right)^T.$$

Using

$$\prod_{t=1}^{T} e^{a_t} = e^{\sum_{t=1}^{T} a_t},$$

the second term becomes

$$\prod_{t=1}^{T} \exp\left(-\frac{\varepsilon_t^2}{2\sigma^2} \right) = \exp\left(-\frac{1}{2\sigma^2} \sum_{t=1}^{T} \varepsilon_t^2 \right).$$

Hence,

$$\boxed{L(\theta) = \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right)^T \exp\left(-\frac{1}{2\sigma^2} \sum_{t=1}^{T} \varepsilon_t^2 \right).}$$

Equivalently,

$$\boxed{L(\theta) = (2\pi\sigma^2)^{-T/2} \exp\left(-\frac{1}{2\sigma^2} \sum_{t=1}^{T} \varepsilon_t^2 \right).}$$

Step 5: Take the Logarithm The log-likelihood is defined as

$$\boxed{\ell(\theta) = \ln L(\theta).}$$

Therefore,

$$\ell(\theta) = \ln \left[(2\pi\sigma^2)^{-T/2} \exp\left(-\frac{1}{2\sigma^2} \sum_{t=1}^{T} \varepsilon_t^2 \right) \right].$$

Using

$$\ln(ab) = \ln(a) + \ln(b),$$

we obtain

$$\ell(\theta) = \ln \left[(2\pi\sigma^2)^{-T/2} \right] + \ln \left[\exp\left(-\frac{1}{2\sigma^2} \sum_{t=1}^{T} \varepsilon_t^2 \right) \right].$$

For the first term, using

$$\ln(a^b) = b \ln(a),$$

we have

$$\ln \left[(2\pi\sigma^2)^{-T/2} \right] = -\frac{T}{2} \ln(2\pi\sigma^2).$$

For the second term,

$$\ln(e^x) = x,$$

so

$$\ln \left[\exp\left(-\frac{1}{2\sigma^2} \sum_{t=1}^{T} \varepsilon_t^2 \right) \right] = -\frac{1}{2\sigma^2} \sum_{t=1}^{T} \varepsilon_t^2.$$

Thus,

$$\ell(\theta) = -\frac{T}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{t=1}^{T} \varepsilon_t^2.$$

Step 6: Expand $\ln(2\pi\sigma^2)$ Using

$$\ln(ab) = \ln(a) + \ln(b),$$

we have

$$\ln(2\pi\sigma^2) = \ln(2\pi) + \ln(\sigma^2).$$

Therefore,

$$-\frac{T}{2}\ln(2\pi\sigma^2) = -\frac{T}{2}\ln(2\pi) - \frac{T}{2}\ln(\sigma^2).$$

Substituting this into the previous expression gives

$$\boxed{\ell(\theta) = -\frac{T}{2}\ln(2\pi) - \frac{T}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{t=1}^{T}\varepsilon_t^2.}$$

where the notation T here represents the number of observations entering the likelihood.

At the maximum likelihood estimate,

$$\boxed{\widehat{\sigma}^2 = \frac{1}{T}\sum_{t=1}^{T}\widehat{\varepsilon}_t^2.}$$

Therefore,

$$\sum_{t=1}^{T}\widehat{\varepsilon}_t^2 = T\widehat{\sigma}^2.$$

Substituting this result into the maximized log-likelihood gives

$$\ell(\widehat{\theta}) = -\frac{T}{2}\ln(2\pi) - \frac{T}{2}\ln(\widehat{\sigma}^2) - \frac{1}{2\widehat{\sigma}^2}T\widehat{\sigma}^2.$$

Hence,

$$\ell(\widehat{\theta}) = -\frac{T}{2}\left[\ln(2\pi) + \ln(\widehat{\sigma}^2) + 1\right].$$

Multiplying by $-2/T$,

$$-\frac{2}{T}\ell(\widehat{\theta}) = \ln(2\pi) + \ln(\widehat{\sigma}^2) + 1.$$

Thus,

$$\boxed{-\frac{2}{T}\ell(\widehat{\theta}) = \ln(\widehat{\sigma}^2) + \underbrace{\ln(2\pi) + 1}_{\text{constant}}.}$$

Simplified AIC Formula Starting from the normalized AIC,

$$AIC = -\frac{2}{T}\ell(\widehat{\theta}) + \frac{2k}{T},$$

and substituting the Gaussian likelihood expression gives

$$AIC = \ln(\widehat{\sigma}^2) + \ln(2\pi) + 1 + \frac{2k}{T}.$$

Therefore,

$$\boxed{AIC = \ln(\widehat{\sigma}^2) + \frac{2k}{T} + \underbrace{\ln(2\pi) + 1}_{\text{constant across models}}}.$$

When comparing several AR models estimated on the same sample, the term

$$\ln(2\pi) + 1$$

is identical for every model.

Adding or removing the same constant from every criterion does not change which model minimizes the criterion.

Therefore, for model selection, we can use the simplified expression

$$\boxed{AIC(k) = \ln \widehat{\sigma}_k^2 + \frac{2k}{T}.}$$

This is the form commonly presented in time-series courses.

Simplified BIC Formula Starting from the normalized BIC,

$$BIC = -\frac{2}{T}\ell(\widehat{\theta}) + \frac{k \ln T}{T},$$

and using

$$-\frac{2}{T}\ell(\widehat{\theta}) = \ln(\widehat{\sigma}^2) + \ln(2\pi) + 1,$$

we obtain

$$BIC = \ln(\widehat{\sigma}^2) + \frac{k \ln T}{T} + \ln(2\pi) + 1.$$

Again, the last two terms are constant across candidate models.

Hence, for model selection, we may use

$$\boxed{BIC(k) = \ln \widehat{\sigma}_k^2 + \frac{k \ln T}{T}.}$$

Interpretation: Fit versus Complexity

Both AIC and BIC contain two components.

For AIC,

$$\boxed{AIC(k) = \underbrace{\ln \widehat{\sigma}_k^2}_{\text{goodness of fit}} + \underbrace{\frac{2k}{T}}_{\text{complexity penalty}}}.$$

For BIC,

$$\boxed{BIC(k) = \underbrace{\ln \widehat{\sigma}_k^2}_{\text{goodness of fit}} + \underbrace{\frac{k \ln T}{T}}_{\text{complexity penalty}}}.$$

Adding more lags generally improves the in-sample fit, which tends to reduce

$$\widehat{\sigma}_k^2.$$

Consequently,

$$\ln \widehat{\sigma}_k^2$$

tends to decrease.

However, adding more lags also increases the number of parameters, which increases the penalty term.

Therefore, information criteria implement the trade-off

$$\boxed{\text{better fit} \quad \text{versus} \quad \text{greater model complexity.}}$$

The selected AR order is the value of k minimizing the chosen criterion:

$$\boxed{\widehat{k}_{AIC} = \arg\min_k AIC(k),}$$

or

$$\boxed{\widehat{k}_{BIC} = \arg\min_k BIC(k).}$$

Why Does BIC Usually Penalize Complexity More Strongly? The AIC penalty per parameter is

$$\frac{2}{T},$$

whereas the BIC penalty per parameter is

$$\frac{\ln T}{T}.$$

Therefore, whenever

$$\ln T > 2,$$

or equivalently

$$T > e^2 \approx 7.39,$$

BIC penalizes each additional parameter more strongly than AIC.

For practically relevant time-series sample sizes,

$$\boxed{\frac{\ln T}{T} > \frac{2}{T}.}$$

Consequently, BIC tends to favor more parsimonious models than AIC.

A Note on the Number of Parameters

There is another convention that can create apparently different formulas: the definition of the number of parameters k.

For an AR(p) model,

$$X_t = \phi_0 + \phi_1 X_{t-1} + \cdots + \phi_p X_{t-p} + \varepsilon_t,$$

one may count the conditional-mean parameters as

$$\phi_0, \phi_1, \ldots, \phi_p.$$

There are therefore

$$\boxed{p+1}$$

such parameters.

If the innovation variance is also included as an estimated parameter, then the parameter vector additionally contains

$$\sigma^2,$$

giving

$$\boxed{p+2}$$

estimated parameters.

Different textbooks and software implementations may use slightly different parameter-counting conventions.

However, when comparing AR orders consistently within the same convention, the relevant principle remains unchanged:

$$\boxed{\text{use the same parameter-counting convention for every candidate model.}}$$

Relation with `statsmodels`

The values reported by `statsmodels` use the standard unnormalized information-criterion convention:

$$\boxed{AIC_{\texttt{statsmodels}} = -2\ell(\widehat{\theta}) + 2k,}$$

and analogously,

$$\boxed{BIC_{\texttt{statsmodels}} = -2\ell(\widehat{\theta}) + k \ln T,}$$

subject to the precise likelihood, effective sample size, and parameter-counting conventions of the fitted model class.

Therefore, a value such as

$$AIC = 15533.491$$

reported by `statsmodels` should not be expected to numerically equal the normalized criterion

$$\ln \widehat{\sigma}^2 + \frac{2k}{T}.$$

Their numerical scales are different.

Nevertheless, under a common likelihood and sample convention, they encode the same model-selection trade-off.

Summary of the Different Formulas

The relationship between the different expressions can be summarized as

Standard AIC

$$AIC = -2\ell(\widehat{\theta}) + 2k$$

$\Downarrow$ divide by T

$$-\frac{2}{T}\ell(\widehat{\theta}) + \frac{2k}{T}$$

$\Downarrow$ Gaussian AR likelihood

$$\ln(\widehat{\sigma}^2) + \ln(2\pi) + 1 + \frac{2k}{T}$$

$\Downarrow$ remove constants common to all models

$$\boxed{AIC(k) = \ln \widehat{\sigma}_k^2 + \frac{2k}{T}}$$

and similarly,

Standard BIC

$$BIC = -2\ell(\widehat{\theta}) + k \ln T$$

$\Downarrow$ divide by T

$$-\frac{2}{T}\ell(\widehat{\theta}) + \frac{k \ln T}{T}$$

$\Downarrow$ Gaussian AR likelihood and removal of common constants

$$\boxed{BIC(k) = \ln \widehat{\sigma}_k^2 + \frac{k \ln T}{T}.}$$

Engle's ARCH Test for Conditional Heteroskedasticity

After estimating an ARMA model, it is important to check whether the residuals exhibit **conditional heteroskedasticity**.

In Python, Engle's ARCH test can be performed using

```
from statsmodels.stats.diagnostic import het_arch

arch_test_result = het_arch(arma_residuals)
```

The purpose of the test is to determine whether the variance of the residuals is constant over time or whether it depends on past shocks.

Constant Conditional Variance Suppose that the ARMA model produces residuals

$$\widehat{\varepsilon}_t.$$

Under the standard homoskedastic assumption, the conditional variance of the innovations is constant:

$$\boxed{\mathrm{Var}\left(\varepsilon_t \mid \mathcal{F}_{t-1}\right) = \sigma^2,}$$

where

$$\mathcal{F}_{t-1}$$

represents all information available up to time $t-1$.

This means that knowing what happened in previous periods does not allow us to predict whether volatility will be high or low at time t.

Volatility Clustering Financial returns frequently violate this assumption.

Periods of large price movements tend to be followed by other periods of large price movements, while calm periods tend to be followed by calm periods.

This phenomenon is called

$$\boxed{\text{volatility clustering.}}$$

For example, consider the residual sequence

$$0.1,\ -0.2,\ 0.1,\ 2.5,\ -3.0,\ 2.1,\ -0.1,\ 0.2.$$

The signs of the residuals alternate, so there may be little obvious autocorrelation in the residuals themselves.

However, consider their squares:

$$0.01,\ 0.04,\ 0.01,\ 6.25,\ 9.00,\ 4.41,\ 0.01,\ 0.04.$$

We can clearly observe a cluster of large squared residuals.

Therefore, it is possible to have

$$\boxed{\mathrm{Corr}(\varepsilon_t, \varepsilon_{t-1}) \approx 0}$$

while simultaneously having

$$\boxed{\mathrm{Corr}(\varepsilon_t^2, \varepsilon_{t-1}^2) > 0.}$$

This distinction is particularly important for financial time series.

Why Do We Study Squared Residuals? Volatility measures the **magnitude** of fluctuations rather than their direction.

For example,

$$\varepsilon_t = 3$$

and

$$\varepsilon_t = -3$$

represent shocks with opposite signs but the same magnitude.

Their squared values are identical:

$$3^2 = (-3)^2 = 9.$$

Therefore,

$$\boxed{\varepsilon_t^2}$$

acts as a simple proxy for the magnitude of the shock and, consequently, for volatility.

The ARCH test therefore investigates whether past squared residuals help predict current squared residuals.

The ARCH Auxiliary Regression

Engle's ARCH test is based on the following auxiliary regression:

$$\boxed{\widehat{\varepsilon}_t^2 = \alpha_0 + \alpha_1 \widehat{\varepsilon}_{t-1}^2 + \alpha_2 \widehat{\varepsilon}_{t-2}^2 + \cdots + \alpha_q \widehat{\varepsilon}_{t-q}^2 + u_t.}$$

The dependent variable is the current squared residual

$$\widehat{\varepsilon}_t^2,$$

while the explanatory variables are previous squared residuals

$$\widehat{\varepsilon}_{t-1}^2, \ldots, \widehat{\varepsilon}_{t-q}^2.$$

The question is therefore:

> Do past squared shocks contain information about the magnitude of the current shock?

Null and Alternative Hypotheses The null hypothesis is

$$\boxed{H_0 : \alpha_1 = \alpha_2 = \cdots = \alpha_q = 0.}$$

Under H_0, past squared residuals have no predictive power for the current squared residual.

Thus,

$$\boxed{H_0 : \text{no ARCH effects.}}$$

The alternative hypothesis is

$$\boxed{H_1 : \exists j \in \{1, \ldots, q\} \text{ such that } \alpha_j \neq 0.}$$

Thus,

$$\boxed{H_1 : \text{ARCH effects are present.}}$$

Under the alternative, past shocks contain information about current volatility.

Engle's Lagrange Multiplier Statistic

After estimating the auxiliary regression,

$$\widehat{\varepsilon}_t^2 = \alpha_0 + \sum_{j=1}^{q} \alpha_j \widehat{\varepsilon}_{t-j}^2 + u_t,$$

we obtain its coefficient of determination

$$R^2.$$

Engle's ARCH LM statistic is approximately

$$\boxed{LM = TR^2,}$$

where T denotes the number of observations used in the auxiliary regression.

Under the null hypothesis,

$$H_0 : \alpha_1 = \cdots = \alpha_q = 0,$$

the LM statistic is asymptotically distributed as

$$\boxed{LM \overset{a}{\sim} \chi_q^2,}$$

where q is the number of lagged squared residuals included in the auxiliary regression.

Decision Rule The test produces a p-value.

For a significance level

$$\alpha = 5\%,$$

if

$$p\text{-value} < 0.05,$$

we reject

$$H_0.$$

Therefore,

$$\boxed{p\text{-value} < 0.05 \quad \Longrightarrow \quad \text{evidence of ARCH effects.}}$$

Conversely, if

$$p\text{-value} \geq 0.05,$$

we fail to reject the null:

$$\boxed{p\text{-value} \geq 0.05 \quad \Longrightarrow \quad \text{insufficient evidence of ARCH effects.}}$$

Importantly, failing to reject H_0 does not prove that the variance is perfectly constant; it only means that the test does not detect significant ARCH dependence at the lags considered.

Output of `het_arch()`

The Python command

```
arch_test_result = het_arch(arma_residuals)
```

returns four quantities:

```
lm_stat, lm_pvalue, f_stat, f_pvalue = het_arch(arma_residuals)
```

They correspond to

`lm_stat`	Engle LM test statistic,
`lm_pvalue`	p-value associated with the LM statistic,
`f_stat`	F-statistic testing the joint significance of the ARCH terms,
`f_pvalue`	p-value associated with the F-statistic.

The F-Statistic from the Auxiliary Regression

The F-statistic provides another way of testing the same null hypothesis as Engle's LM statistic. Recall the ARCH auxiliary regression:

$$\boxed{\widehat{\varepsilon}_t^2 = \alpha_0 + \alpha_1 \widehat{\varepsilon}_{t-1}^2 + \cdots + \alpha_q \widehat{\varepsilon}_{t-q}^2 + u_t.}$$

The null hypothesis is

$$\boxed{H_0 : \alpha_1 = \alpha_2 = \cdots = \alpha_q = 0,}$$

against

$$\boxed{H_1 : \exists j \in \{1, \ldots, q\} \text{ such that } \alpha_j \neq 0.}$$

Thus, the F-test asks whether the lagged squared residuals are **jointly significant** in explaining the current squared residual.

Equivalently, it asks:

> Does adding past squared residuals significantly improve our ability to explain the current squared residual?

Restricted and Unrestricted Models To understand the F-statistic, consider two regressions.

Under the null hypothesis,

$$\alpha_1 = \cdots = \alpha_q = 0.$$

The auxiliary regression therefore reduces to

$$\boxed{\widehat{\varepsilon}_t^2 = \alpha_0 + u_t.}$$

This is called the **restricted model**, because the restrictions

$$\alpha_1 = \cdots = \alpha_q = 0$$

have been imposed.

It says that past squared residuals contain no information about the current squared residual.

The unrestricted model is the complete ARCH auxiliary regression:

$$\boxed{\widehat{\varepsilon}_t^2 = \alpha_0 + \sum_{j=1}^{q} \alpha_j \widehat{\varepsilon}_{t-j}^2 + u_t.}$$

The unrestricted model allows past squared residuals to explain the current squared residual.

Comparing the Two Models Let

$$SSR_R$$

denote the sum of squared residuals from the **restricted** model, and

$$SSR_U$$

the sum of squared residuals from the **unrestricted** model.

Because the unrestricted model contains all the regressors of the restricted model plus q additional variables, we necessarily have

$$\boxed{SSR_U \leq SSR_R.}$$

The relevant question is therefore not whether the unrestricted model fits better—it necessarily fits at least as well in sample—but whether the improvement in fit is sufficiently large to be statistically significant.

The improvement in fit is measured by

$$\boxed{SSR_R - SSR_U.}$$

If

$$SSR_R - SSR_U$$

is very small, adding lagged squared residuals provides little additional explanatory power.

If it is large, the lagged squared residuals substantially improve the fit of the auxiliary regression.

Construction of the F-Statistic Suppose there are n observations in the auxiliary regression and q lagged squared residuals.

The F-statistic is

$$\boxed{F = \frac{(SSR_R - SSR_U)/q}{SSR_U/(n-q-1)}.}$$

This statistic can be understood as a ratio of two quantities.

The numerator is

$$\boxed{\frac{SSR_R - SSR_U}{q},}$$

which measures the average improvement in fit per restriction that is relaxed.

There are q restrictions:

$$\alpha_1 = 0, \ldots, \alpha_q = 0.$$

The denominator is

$$\boxed{\frac{SSR_U}{n-q-1},}$$

which estimates the residual variance of the unrestricted auxiliary regression.

The unrestricted model estimates

$$q + 1$$

parameters:

$$\alpha_0, \alpha_1, \ldots, \alpha_q,$$

so its residual degrees of freedom are

$$n - (q + 1) = n - q - 1.$$

Consequently,

$$\boxed{F = \frac{\text{improvement in fit per additional ARCH parameter}}{\text{remaining unexplained variation per residual degree of freedom}}.}$$

Intuition If the null hypothesis is true,

$$H_0 : \alpha_1 = \cdots = \alpha_q = 0,$$

then adding

$$\widehat{\varepsilon}_{t-1}^2, \ldots, \widehat{\varepsilon}_{t-q}^2$$

should not substantially improve the regression.

We would therefore expect

$$SSR_R \approx SSR_U,$$

which implies

$$SSR_R - SSR_U \approx 0.$$

Consequently, the F-statistic should be relatively small.

Conversely, if past squared residuals strongly predict the current squared residual, then

$$SSR_U \ll SSR_R.$$

Therefore,

$$SSR_R - SSR_U$$

becomes large, producing a large F-statistic.

Hence,

$$\boxed{\text{large } F \quad \Longrightarrow \quad \text{lagged squared residuals have substantial joint explanatory power.}}$$

Distribution Under the Null Hypothesis Under the usual assumptions of the auxiliary regression, the F-statistic is compared with an F-distribution:

$$\boxed{F \sim F_{q,n-q-1} \qquad \text{under } H_0.}$$

The first degree of freedom,

$$q,$$

corresponds to the number of restrictions being tested.

The second degree of freedom,

$$n - q - 1,$$

corresponds to the residual degrees of freedom of the unrestricted auxiliary regression.

Decision Rule for the F-Test The Python output

`f_pvalue`

is the p-value associated with the observed F-statistic.

At the 5% significance level,

$$\boxed{\texttt{f_pvalue} < 0.05 \quad \Longrightarrow \quad \text{reject } H_0.}$$

We therefore conclude that the lagged squared residuals are jointly significant:

$$\boxed{\text{evidence of ARCH effects.}}$$

Conversely,

$$\boxed{\texttt{f_pvalue} \geq 0.05 \quad \Longrightarrow \quad \text{fail to reject } H_0.}$$

In this case, there is insufficient evidence that past squared residuals help predict the current squared residual.

LM Test versus F-Test

The LM statistic and the F-statistic test the same null hypothesis:

$$\boxed{H_0 : \alpha_1 = \cdots = \alpha_q = 0.}$$

However, they construct the test statistic differently.

Engle's LM statistic is based on the coefficient of determination of the auxiliary regression:

$$\boxed{LM = nR^2.}$$

Under H_0, it is asymptotically distributed as

$$\boxed{LM \overset{a}{\sim} \chi^2_q.}$$

The F-statistic instead compares the restricted and unrestricted auxiliary regressions:

$$\boxed{F = \frac{(SSR_R - SSR_U)/q}{SSR_U/(n-q-1)}.}$$

Its reference distribution is

$$\boxed{F_{q,n-q-1}.}$$

Thus,

Test	Statistic	Reference distribution
Engle LM	nR^2	χ^2_q
F-test	$\dfrac{(SSR_R - SSR_U)/q}{SSR_U/(n-q-1)}$	$F_{q,n-q-1}$

Both tests ask the same economic and statistical question:

Do past squared shocks contain information about current volatility?

In large samples, the two tests will generally lead to the same conclusion, although their numerical test statistics and p-values are not identical.

The `mean` Parameter in a GARCH Model

When estimating a GARCH model using the `arch` package,

```
from arch import arch_model

garch_model = arch_model(
    jpm_return,
    mean='Zero',
    vol='GARCH',
    p=1,
```

```
    q=1,
    dist='Normal'
).fit(disp="off")
```

the parameter `mean` determines how the **conditional mean of the return** is modeled.

This is distinct from the parameter

```
vol='GARCH'
```

which determines how the **conditional variance** is modeled.

Conditional Mean and Conditional Variance A return series can generally be written as

$$\boxed{r_t = \mu_t + \varepsilon_t,}$$

where

$$\boxed{\mu_t = \mathbb{E}\left[r_t \mid \mathcal{F}_{t-1}\right]}$$

is the conditional mean and

$$\varepsilon_t$$

is the unpredictable innovation.

The innovation can itself be written as

$$\boxed{\varepsilon_t = \sigma_t z_t,}$$

where

$$\mathbb{E}[z_t] = 0, \qquad \mathrm{Var}(z_t) = 1.$$

Therefore,

$$\boxed{\mathrm{Var}\left(\varepsilon_t \mid \mathcal{F}_{t-1}\right) = \sigma_t^2.}$$

The complete model therefore contains two distinct components:

$$\boxed{\begin{array}{cc} \text{Conditional mean:} & \mu_t = \mathbb{E}[r_t \mid \mathcal{F}_{t-1}], \\ \text{Conditional variance:} & \sigma_t^2 = \mathrm{Var}(r_t \mid \mathcal{F}_{t-1}). \end{array}}$$

The parameter `mean` specifies the first equation, while `vol` specifies the second.

`mean='Zero'`

With

`mean='Zero'`

we impose

$$\boxed{\mu_t = 0.}$$

Therefore,

$$r_t = \varepsilon_t.$$

Since

$$\varepsilon_t = \sigma_t z_t,$$

the return equation becomes

$$\boxed{r_t = \sigma_t z_t.}$$

Thus, the model assumes that there is no predictable component in the conditional mean:

$$\boxed{\mathbb{E}[r_t \mid \mathcal{F}_{t-1}] = 0.}$$

All the dynamics modeled by the GARCH specification concern the **conditional variance**. For a GARCH(1,1),

$$\boxed{\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2.}$$

Hence, the complete Zero-Mean GARCH(1,1) model is

$$\boxed{\begin{aligned} r_t &= \varepsilon_t, \\ \varepsilon_t &= \sigma_t z_t, \\ \sigma_t^2 &= \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2. \end{aligned}}$$

This explains why the model output displays

```
Mean Model: Zero Mean
```

and does not report any estimated coefficient for the conditional mean.

`mean='Constant'`

With

`mean='Constant'`

the conditional mean is assumed to be constant:

$$\boxed{\mu_t = \mu.}$$

The return equation becomes

$$\boxed{r_t = \mu + \varepsilon_t.}$$

Therefore,

$$\mathbb{E}[r_t \mid \mathcal{F}_{t-1}] = \mu.$$

Unlike the Zero-Mean model, μ is now an additional parameter that must be estimated.

Combined with a GARCH(1,1) variance equation, we obtain

$$\boxed{\begin{aligned} r_t &= \mu + \varepsilon_t, \\ \varepsilon_t &= \sigma_t z_t, \\ \sigma_t^2 &= \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2. \end{aligned}}$$

In Python:

```
garch_model = arch_model(
    jpm_return,
    mean='Constant',
    vol='GARCH',
    p=1,
    q=1,
    dist='Normal'
)
```

`mean='AR'`

The conditional mean can also contain autoregressive dynamics.

For example, an AR(1) conditional mean is

$$\boxed{\mu_t = \mu + \phi_1 r_{t-1}.}$$

The return equation becomes

$$\boxed{r_t = \mu + \phi_1 r_{t-1} + \varepsilon_t.}$$

Therefore,

$$\boxed{\mathbb{E}[r_t \mid \mathcal{F}_{t-1}] = \mu + \phi_1 r_{t-1}.}$$

If the variance simultaneously follows a GARCH(1,1),

$$\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2,$$

the complete model is

$$\boxed{\begin{aligned} r_t &= \mu + \phi_1 r_{t-1} + \varepsilon_t, \\ \varepsilon_t &= \sigma_t z_t, \\ \sigma_t^2 &= \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2. \end{aligned}}$$

This can be viewed as an AR(1)–GARCH(1,1) model.

In Python:

```
garch_model = arch_model(
    jpm_return,
    mean='AR',
    lags=1,
    vol='GARCH',
    p=1,
    q=1,
    dist='Normal'
)
```

Important: `lags` versus `p` and `q` It is important not to confuse the order of the conditional mean model with the order of the GARCH model.

In

```
arch_model(
    jpm_return,
    mean='AR',
    lags=1,
    vol='GARCH',
    p=1,
    q=1
)
```

the argument

`lags=1`

determines the autoregressive order of the **conditional mean**.

Thus,

$$\texttt{lags=1} \quad \Longrightarrow \quad r_t = \mu + \phi_1 r_{t-1} + \varepsilon_t.$$

By contrast,

$$\boxed{\texttt{p=1,} \qquad \texttt{q=1}}$$

determine the order of the **GARCH conditional variance equation**:

$$\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2.$$

Therefore,

`lags`	$\longrightarrow$	lags in the conditional mean,
`p`	$\longrightarrow$	lags of squared innovations in the variance,
`q`	$\longrightarrow$	lags of conditional variance in the variance equation.

Summary of the Different Mean Specifications

The main choices can be summarized as follows:

`mean='Zero'`	$\Longrightarrow$	$\mu_t = 0,$
`mean='Constant'`	$\Longrightarrow$	$\mu_t = \mu,$
`mean='AR'`	$\Longrightarrow$	$\mu_t = \mu + \sum_{i=1}^{P} \phi_i r_{t-i}.$

Thus, the choice of `mean` answers the question

$$\boxed{\text{How should } \mathbb{E}[r_t \mid \mathcal{F}_{t-1}] \text{ be modeled?}}$$

whereas the choice

`vol='GARCH'`

answers the separate question

$$\boxed{\text{How should } \operatorname{Var}(r_t \mid \mathcal{F}_{t-1}) \text{ be modeled?}}$$

Using `delimiter=r"\s+"` in `pd.read_csv()`

Consider the following Python instruction:

```
data = pd.read_csv(
    'housing.csv',
    header=None,
    delimiter=r"\s+",
    names=column_names
)
```

The argument

```
delimiter=r"\s+"
```

specifies how the columns of the file are separated.

In this case, the delimiter is defined using a **regular expression**:

`\s+` = one or more whitespace characters.

Understanding `\s+` The expression

`\s+`

contains two components.

First,

`\s` = any whitespace character.

A whitespace character can be, for example,

space, tab, newline, etc.

Second,

`+` = one or more occurrences of the preceding expression.

Therefore,

`\s+` = one or more consecutive whitespace characters.

For example, all of the following can act as separators:

```
" "
"   "
"       "
"\t"
```

Thus, the number of spaces between two values does not need to be constant.

The Global F-Test in OLS Regression

Consider the multiple linear regression model

$$\boxed{Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \beta_k X_{ik} + \varepsilon_i.}$$

The F-statistic reported in an OLS regression summary tests whether the explanatory variables have **joint explanatory power**.

Null and Alternative Hypotheses The null hypothesis is

$$\boxed{H_0 : \beta_1 = \beta_2 = \cdots = \beta_k = 0.}$$

The alternative hypothesis is

$$\boxed{H_1 : \text{at least one } \beta_j \neq 0.}$$

Notice that the intercept

$$\beta_0$$

is **not** included in the hypothesis.

Under H_0, the regression becomes

$$\boxed{Y_i = \beta_0 + \varepsilon_i.}$$

Thus, under the null hypothesis, none of the explanatory variables provides additional explanatory power beyond the intercept.

If the regression contains an intercept, the fitted value from this restricted model is simply the sample mean:

$$\widehat{Y_i} = \overline{Y}.$$

Therefore, the F-test essentially asks:

> Do the explanatory variables jointly improve the model relative to simply predicting Y using its mean?

Restricted and Unrestricted Models

The F-test compares two models.

The **restricted model**, corresponding to H_0, is

$$\boxed{Y_i = \beta_0 + \varepsilon_i.}$$

The **unrestricted model** is the complete regression:

$$\boxed{Y_i = \beta_0 + \beta_1 X_{i1} + \cdots + \beta_k X_{ik} + \varepsilon_i.}$$

Let

$$SSR_R$$

denote the residual sum of squares of the restricted model and

$$SSR_U$$

the residual sum of squares of the unrestricted model.

Since the unrestricted model contains all the variables of the restricted model plus k additional regressors,

$$\boxed{SSR_U \leq SSR_R.}$$

The question is therefore not whether adding regressors improves the in-sample fit—it necessarily does, or at least cannot worsen it—but whether the improvement is sufficiently large to be statistically significant.

Construction of the F-Statistic

The global F-statistic is

$$\boxed{F = \frac{(SSR_R - SSR_U)/k}{SSR_U/(n-k-1)},}$$

where

$$\begin{aligned} SSR_R &= \text{residual sum of squares of the restricted model,} \\ SSR_U &= \text{residual sum of squares of the unrestricted model,} \\ k &= \text{number of explanatory variables excluding the intercept,} \\ n &= \text{number of observations.} \end{aligned}$$

The numerator

$$\frac{SSR_R - SSR_U}{k}$$

measures the average improvement in fit obtained by introducing the k explanatory variables.

The denominator

$$\frac{SSR_U}{n-k-1}$$

measures the remaining unexplained variation per residual degree of freedom.

Thus,

$$\boxed{F = \frac{\text{improvement in fit per additional regressor}}{\text{remaining unexplained variation per degree of freedom}}.}$$

Intuition If

$$H_0 : \beta_1 = \cdots = \beta_k = 0$$

is true, introducing the explanatory variables should not substantially improve the model.

We would therefore expect

$$SSR_R \approx SSR_U,$$

so that

$$SSR_R - SSR_U \approx 0.$$

Consequently, the F-statistic should be relatively small.

Conversely, if the explanatory variables contain substantial information about Y, then

$$SSR_U \ll SSR_R.$$

The difference

$$SSR_R - SSR_U$$

will then be large, producing a large F-statistic.

Hence,

$$\boxed{\text{large } F \quad \Longrightarrow \quad \text{strong evidence against } H_0.}$$

Equivalent Formula Using R^2

When the regression contains an intercept, the global F-statistic can also be expressed directly using R^2:

$$\boxed{F = \frac{R^2/k}{(1-R^2)/(n-k-1)}.}$$

This formula makes the intuition particularly clear.

The numerator

$$\frac{R^2}{k}$$

measures explained variation relative to the number of explanatory variables.

The denominator

$$\frac{1 - R^2}{n - k - 1}$$

measures unexplained variation relative to the residual degrees of freedom.

Application to the Regression Output

For the regression considered here,

$$n = 401,$$

the number of explanatory variables is

$$k = 13,$$

and

$$R^2 = 0.735.$$

Therefore,

$$F = \frac{0.735/13}{(1 - 0.735)/(401 - 13 - 1)}.$$

The residual degrees of freedom are

$$401 - 13 - 1 = 387,$$

which corresponds to the value reported in the regression output:

$$\boxed{Df_{\text{Residuals}} = 387.}$$

Thus,

$$F = \frac{0.735/13}{0.265/387} \approx 82.6.$$

This corresponds approximately to the reported value

$$\boxed{F = 82.66.}$$

Distribution Under the Null Hypothesis Under the classical OLS assumptions and under

$$H_0 : \beta_1 = \cdots = \beta_k = 0,$$

the statistic follows an F-distribution:

$$\boxed{F \sim F_{k,n-k-1}.}$$

In this example,

$$\boxed{F \sim F_{13,387} \qquad \text{under } H_0.}$$

The observed value

$$F = 82.66$$

is then compared with this reference distribution.

The `Prob(F-statistic)`

The regression output reports

$$\boxed{\text{Prob}(F\text{-statistic}) = 5.72 \times 10^{-103}.}$$

This is the p-value associated with the observed F-statistic.

It represents, under the null hypothesis, the probability of observing an F-statistic at least as large as the one obtained:

$$p = P\left(F_{13,387} \geq 82.66 \mid H_0\right).$$

Since

$$5.72 \times 10^{-103} \ll 0.05,$$

we strongly reject

$$H_0 : \beta_1 = \cdots = \beta_{13} = 0.$$

Therefore,

$$\boxed{\text{the explanatory variables are jointly statistically significant.}}$$

In other words, taken together, the regressors explain variation in `MEDV` beyond what would be explained by an intercept-only model.

F-Test versus Individual t-Tests

It is important to distinguish the global F-test from the individual t-tests reported for each regression coefficient.

For a particular coefficient β_j, the t-test considers

$$\boxed{H_0 : \beta_j = 0}$$

against, typically,

$$\boxed{H_1 : \beta_j \neq 0.}$$

Thus, a t-test asks whether one particular explanatory variable is statistically significant, conditional on the other regressors.

By contrast, the global F-test considers

$$\boxed{H_0 : \beta_1 = \beta_2 = \cdots = \beta_k = 0.}$$

It therefore asks whether the regressors are **jointly statistically significant**.

The distinction can be summarized as

t-test	$\longrightarrow$	Is one particular β_j significant?
F-test	$\longrightarrow$	Are the regressors jointly significant?

Consequently, it is entirely possible to obtain

$$\boxed{\text{a highly significant global } F\text{-test}}$$

while some individual coefficients have

$$\boxed{p_j > 0.05.}$$

The global F-test does **not** imply that every individual coefficient is statistically significant. It only provides evidence that the coefficients are not all simultaneously equal to zero.

Omnibus Test versus Jarque–Bera Test

The Omnibus test and the Jarque–Bera test are both used to test whether the residuals of a regression are normally distributed.

They therefore consider essentially the same hypotheses:

$$\boxed{H_0 : \text{the residuals are normally distributed}}$$

against

$$\boxed{H_1 : \text{the residuals are not normally distributed.}}$$

For a normal distribution, we expect

$$\boxed{S = 0, \qquad K = 3,}$$

where

$$S$$

denotes skewness and

$$K$$

denotes kurtosis.

Both tests therefore use information about deviations from these two properties. The main difference lies in **how skewness and kurtosis are combined into a test statistic**.

The Jarque–Bera Test

The Jarque–Bera test directly combines the sample skewness and kurtosis.

Its test statistic is

$$\boxed{JB = \frac{n}{6}\left[S^2 + \frac{(K-3)^2}{4}\right],}$$

where

$$\begin{aligned} n &= \text{number of observations}, \\ S &= \text{sample skewness}, \\ K &= \text{sample kurtosis}. \end{aligned}$$

The first component,

$$S^2,$$

measures the departure from the normal-distribution condition

$$S = 0.$$

The second component,

$$\frac{(K-3)^2}{4},$$

measures the departure from

$$K = 3.$$

Thus,

$$\boxed{JB = \frac{n}{6}\left[\underbrace{S^2}_{\text{departure from symmetry}} + \underbrace{\frac{(K-3)^2}{4}}_{\text{departure from normal kurtosis}}\right].}$$

The quantity

$$K - 3$$

is called the **excess kurtosis**.

For a Gaussian distribution,

$$S = 0 \qquad \text{and} \qquad K - 3 = 0,$$

so the Jarque–Bera statistic should be relatively small.

Large deviations in either skewness or kurtosis increase the statistic.

Under the null hypothesis, asymptotically,

$$\boxed{JB \overset{a}{\sim} \chi^2_2.}$$

The Omnibus Normality Test

The Omnibus statistic reported by `statsmodels` corresponds to D'Agostino's K^2 normality test.

Like the Jarque–Bera test, it uses both skewness and kurtosis.

However, instead of inserting the raw sample skewness and kurtosis directly into a formula, the test first transforms them.

Let

$$Z_S$$

denote a transformation of the sample skewness, and let

$$Z_K$$

denote a transformation of the sample kurtosis.

These transformations are constructed so that, under normality, the resulting statistics are approximately standard normal:

$$Z_S \approx \mathcal{N}(0, 1),$$

and

$$Z_K \approx \mathcal{N}(0,1).$$

The Omnibus statistic then combines them as

$$\boxed{K^2 = Z_S^2 + Z_K^2.}$$

Under the null hypothesis of normality,

$$\boxed{K^2 \overset{a}{\sim} \chi_2^2.}$$

Thus, the Omnibus test can be understood as testing simultaneously whether the residuals exhibit abnormal skewness and abnormal kurtosis.

The transformations used by D'Agostino's test are designed to better account for the sampling distributions of sample skewness and kurtosis, particularly in finite samples.

Main Difference

The fundamental distinction can therefore be summarized as

Jarque–Bera	$\longrightarrow$	uses S and $K-3$ directly,
D'Agostino Omnibus	$\longrightarrow$	transforms S and K first, then combines them.

More explicitly,

$$\boxed{JB = \frac{n}{6}\left[S^2 + \frac{(K-3)^2}{4}\right]}$$

whereas

$$\boxed{K^2 = Z_S^2 + Z_K^2.}$$

Both statistics ultimately test departures from normality through skewness and kurtosis, but they construct their statistics differently.

Application to the Regression Output

In the regression output, we observe

$$\boxed{S = 1.189}$$

and

$$\boxed{K = 8.128.}$$

For a normal distribution, we would expect

$$S = 0, \qquad K = 3.$$

Here,

$$S = 1.189 > 0$$

indicates substantial positive skewness, while

$$K = 8.128 \gg 3$$

indicates a distribution with much heavier tails than a Gaussian distribution.

The reported normality statistics are

$$\boxed{\text{Omnibus} = 117.115}$$

and

$$\boxed{JB = 533.859.}$$

Their associated p-values are

$$\boxed{\text{Prob(Omnibus)} \approx 0}$$

and

$$\boxed{\text{Prob}(JB) = 1.19 \times 10^{-116}.}$$

Both p-values are far below a conventional significance level such as

$$\alpha = 5\%.$$

Therefore, both tests strongly reject

$$H_0 : \text{the residuals are normally distributed.}$$

Hence,

$$\boxed{\text{there is strong statistical evidence that the residuals are not normal.}}$$

The Durbin–Watson Statistic

The Durbin–Watson statistic is a regression diagnostic used to detect **first-order serial correlation** in the residuals of a regression model.

Suppose that the estimated OLS residuals are

$$\widehat{\varepsilon}_1, \widehat{\varepsilon}_2, \ldots, \widehat{\varepsilon}_T.$$

The Durbin–Watson statistic investigates whether consecutive residuals are correlated, that is, whether

$$\operatorname{Corr}(\varepsilon_t, \varepsilon_{t-1}) \neq 0.$$

Definition of the Durbin–Watson Statistic

The Durbin–Watson statistic is defined as

$$\boxed{DW = \frac{\displaystyle\sum_{t=2}^{T} (\widehat{\varepsilon}_t - \widehat{\varepsilon}_{t-1})^2}{\displaystyle\sum_{t=1}^{T} \widehat{\varepsilon}_t^2}.}$$

The numerator measures the squared differences between consecutive residuals:

$$\sum_{t=2}^{T} (\widehat{\varepsilon}_t - \widehat{\varepsilon}_{t-1})^2.$$

The denominator measures the overall magnitude of the residuals:

$$\sum_{t=1}^{T} \widehat{\varepsilon}_t^2.$$

The Durbin–Watson statistic therefore compares the variation between consecutive residuals with the overall residual variation.

Range and Interpretation

The Durbin–Watson statistic lies approximately between 0 and 4:

$$\boxed{0 \leq DW \leq 4.}$$

Its interpretation is

$DW \approx 2$	$\Longrightarrow$	little or no first-order autocorrelation,
$DW < 2$	$\Longrightarrow$	positive first-order autocorrelation,
$DW > 2$	$\Longrightarrow$	negative first-order autocorrelation.

Why Does $DW < 2$ Indicate Positive Autocorrelation? Suppose that consecutive residuals are positively correlated.

Then a positive residual tends to be followed by another positive residual, and a negative residual tends to be followed by another negative residual.

Consequently,

$$\widehat{\varepsilon}_t \approx \widehat{\varepsilon}_{t-1}.$$

Their difference is therefore relatively small:

$$\widehat{\varepsilon}_t - \widehat{\varepsilon}_{t-1} \approx 0.$$

Hence,

$$(\widehat{\varepsilon}_t - \widehat{\varepsilon}_{t-1})^2$$

is also small.

The numerator of the Durbin–Watson statistic,

$$\sum_{t=2}^{T} (\widehat{\varepsilon}_t - \widehat{\varepsilon}_{t-1})^2,$$

therefore becomes relatively small, leading to

$$\boxed{DW < 2.}$$

Thus,

$$\boxed{\text{positive residual autocorrelation} \quad \Longrightarrow \quad DW < 2.}$$

Why Does $DW > 2$ Indicate Negative Autocorrelation? Suppose instead that consecutive residuals are negatively correlated.

A positive residual then tends to be followed by a negative residual, and vice versa.

Schematically,

$$\widehat{\varepsilon}_t \approx -\widehat{\varepsilon}_{t-1}.$$

Consequently,

$$\widehat{\varepsilon}_t - \widehat{\varepsilon}_{t-1}$$

tends to be relatively large in absolute value.

For example, if

$$\widehat{\varepsilon}_{t-1} = 2 \qquad \text{and} \qquad \widehat{\varepsilon}_t = -2,$$

then

$$\widehat{\varepsilon}_t - \widehat{\varepsilon}_{t-1} = -2 - 2 = -4.$$

Thus,

$$(\widehat{\varepsilon}_t - \widehat{\varepsilon}_{t-1})^2 = 16.$$

The numerator of the Durbin–Watson statistic becomes relatively large, leading to

$$\boxed{DW > 2.}$$

Thus,

$$\boxed{\text{negative residual autocorrelation} \quad \Longrightarrow \quad DW > 2.}$$

Connection with the Lag-1 Autocorrelation

The Durbin–Watson statistic has a useful approximate relationship with the first-order sample autocorrelation of the residuals.

Let

$$\widehat{\rho}_1 = \operatorname{Corr}\left(\widehat{\varepsilon}_t, \widehat{\varepsilon}_{t-1}\right).$$

Then approximately,

$$\boxed{DW \approx 2\left(1 - \widehat{\rho}_1\right).}$$

Equivalently,

$$\boxed{\widehat{\rho}_1 \approx 1 - \frac{DW}{2}.}$$

This relationship makes the interpretation of the Durbin–Watson statistic particularly intuitive.

No Autocorrelation If

$$\widehat{\rho}_1 \approx 0,$$

then

$$DW \approx 2(1 - 0) = 2.$$

Hence,

$$\boxed{DW \approx 2 \quad \Longleftrightarrow \quad \widehat{\rho}_1 \approx 0.}$$

Strong Positive Autocorrelation If

$$\widehat{\rho}_1 \approx 1,$$

then

$$DW \approx 2(1-1) = 0.$$

Hence,

$$\boxed{DW \approx 0 \quad \Longleftrightarrow \quad \text{very strong positive autocorrelation.}}$$

Strong Negative Autocorrelation If

$$\widehat{\rho}_1 \approx -1,$$

then

$$DW \approx 2(1+1) = 4.$$

Hence,

$$\boxed{DW \approx 4 \quad \Longleftrightarrow \quad \text{very strong negative autocorrelation.}}$$

Application to the Regression Output

Suppose that the regression output reports

$$\boxed{DW = 1.124.}$$

Since

$$1.124 < 2,$$

the statistic suggests **positive first-order serial correlation** in the residuals.

Using the approximate relationship

$$DW \approx 2(1 - \widehat{\rho}_1),$$

we obtain

$$\widehat{\rho}_1 \approx 1 - \frac{DW}{2}.$$

Therefore,

$$\widehat{\rho}_1 \approx 1 - \frac{1.124}{2} = 1 - 0.562 = 0.438.$$

Thus,

$$\boxed{\widehat{\rho}_1 \approx 0.438.}$$

This provides an intuitive indication of substantial positive first-order serial dependence in the residuals.

However, this calculation is only an approximation and should not be interpreted as a formal hypothesis test.

The Durbin–Watson Number Is Not a p-Value

An important point is that the value reported by `statsmodels`,

```
Durbin-Watson: 1.124
```

is the Durbin–Watson **test statistic**, not a p-value.

The classical Durbin–Watson test uses special lower and upper critical values, conventionally denoted

$$d_L \qquad \text{and} \qquad d_U,$$

which depend on factors such as

$$T, \qquad k, \qquad \text{and the significance level.}$$

Therefore, the informal rule

$$DW \approx 2$$

is useful for diagnostics, but a formal Durbin–Watson hypothesis test requires the appropriate critical bounds.

Comparison with Other Residual Diagnostics

The Durbin–Watson statistic should not be confused with normality or heteroskedasticity tests.

These diagnostics investigate different properties of the residuals:

Omnibus / Jarque–Bera	$\longrightarrow$	normality of residuals,
Durbin–Watson	$\longrightarrow$	first-order serial correlation,
Ljung–Box	$\longrightarrow$	serial correlation across one or several lags,
ARCH test	$\longrightarrow$	serial dependence in squared residuals / conditional variance.

In particular,

$$\boxed{\mathrm{Corr}(\varepsilon_t, \varepsilon_{t-1})}$$

concerns serial dependence in the residuals themselves, whereas

$$\boxed{\mathrm{Corr}(\varepsilon_t^2, \varepsilon_{t-1}^2)}$$

concerns serial dependence in their magnitude and is related to ARCH effects.

The Condition Number and Singular Value Decomposition

The **condition number** reported in an OLS regression summary is a diagnostic of the numerical conditioning of the regression design matrix.

Consider the multiple linear regression model

$$\boxed{\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon},}$$

where

$$\mathbf{y} \in \mathbb{R}^n, \qquad X \in \mathbb{R}^{n\times k}, \qquad \boldsymbol{\beta} \in \mathbb{R}^k, \qquad \boldsymbol{\varepsilon} \in \mathbb{R}^n.$$

Here,

$$n = \text{number of observations}$$

and

$$k = \text{number of estimated regression coefficients,}$$

including the intercept if it is included as a column of X.

The OLS estimator is

$$\boxed{\widehat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y},}$$

provided that X has full column rank.

If some columns of X are exactly or nearly linear combinations of other columns, the matrix becomes singular or nearly singular.

This creates numerical instability in the estimation of $\boldsymbol{\beta}$.

The condition number provides a way of quantifying this problem.

Singular Value Decomposition

To understand the condition number, we first need to introduce the **Singular Value Decomposition**, or SVD.

Any real matrix

$$X \in \mathbb{R}^{n \times k}$$

can be decomposed as

$$\boxed{X = U \Sigma V^{\top}.}$$

This is called the Singular Value Decomposition of X.

For the reduced SVD, assuming $n \geq k$,

$$U \in \mathbb{R}^{n \times k}, \qquad \Sigma \in \mathbb{R}^{k \times k}, \qquad V \in \mathbb{R}^{k \times k}.$$

The matrices U and V have orthonormal columns:

$$\boxed{U^{\top} U = I_k, \qquad V^{\top} V = I_k.}$$

The matrix Σ is diagonal:

$$\boxed{\Sigma = \begin{pmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_k \end{pmatrix},}$$

where

$$\boxed{\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k \geq 0.}$$

The quantities

$$\boxed{\sigma_1, \sigma_2, \ldots, \sigma_k}$$

are called the **singular values** of X.

What Are Singular Values?

Singular values measure how strongly the matrix X acts along particular directions of the input space.

Consider a vector

$$\mathbf{v} \in \mathbb{R}^k.$$

The matrix X maps this vector into

$$X\mathbf{v} \in \mathbb{R}^n.$$

The right singular vectors

$$\mathbf{v}_1, \ldots, \mathbf{v}_k$$

are special orthogonal directions in the input space.

For each right singular vector,

$$\boxed{X\mathbf{v}_j = \sigma_j \mathbf{u}_j,}$$

where

$$\mathbf{u}_j$$

is the corresponding left singular vector.

Since

$$\|\mathbf{u}_j\|_2 = \|\mathbf{v}_j\|_2 = 1,$$

we obtain

$$\boxed{\|X\mathbf{v}_j\|_2 = \sigma_j.}$$

Thus, σ_j tells us how much the matrix stretches the direction $\mathbf{v}_j$.

A large singular value means that the corresponding direction is strongly represented by the matrix.

A very small singular value means that there exists a direction in the regressor space that the matrix almost collapses to zero.

Geometric Interpretation of the SVD

The SVD

$$X = U\Sigma V^\top$$

can be interpreted as three successive transformations:

$$\boxed{\mathbf{x} \xrightarrow{V^\top} V^\top\mathbf{x} \xrightarrow{\Sigma} \Sigma V^\top\mathbf{x} \xrightarrow{U} U\Sigma V^\top\mathbf{x}.}$$

The matrix

$$V^\top$$

first rotates or reflects the coordinate system.

The diagonal matrix

$$\Sigma$$

then stretches or compresses the different directions by the singular values

$$\sigma_1, \ldots, \sigma_k.$$

Finally,

$$U$$

rotates or reflects the resulting vectors into the output space.

Thus, conceptually,

$$\boxed{X = \underbrace{U}_{\text{rotation}} \quad \underbrace{\Sigma}_{\text{stretching}} \quad \underbrace{V^\top}_{\text{rotation}}.}$$

The singular values therefore quantify the amount of stretching or compression performed by X along its principal directions.

Connection Between Singular Values and Eigenvalues

The singular values of X are closely related to the eigenvalues of

$$X^\top X.$$

Starting from

$$X = U\Sigma V^\top,$$

we have

$$X^\top = V\Sigma^\top U^\top.$$

Therefore,

$$\begin{aligned} X^\top X &= V\Sigma^\top U^\top U\Sigma V^\top \\ &= V\Sigma^\top \Sigma V^\top, \end{aligned}$$

because

$$U^\top U = I.$$

For the reduced SVD,

$$\Sigma^\top \Sigma = \Sigma^2.$$

Hence,

$$\boxed{X^\top X = V \Sigma^2 V^\top.}$$

This is precisely an eigendecomposition of $X^\top X$.

Therefore,

$$\boxed{\lambda_j(X^\top X) = \sigma_j^2(X).}$$

Equivalently,

$$\boxed{\sigma_j(X) = \sqrt{\lambda_j(X^\top X)}.}$$

Thus, singular values can be interpreted as the square roots of the eigenvalues of $X^\top X$.

Singular Values and Linear Dependence

Suppose that the columns of X are exactly linearly dependent.

Then there exists a non-zero vector

$$\mathbf{a} \neq \mathbf{0}$$

such that

$$\boxed{X\mathbf{a} = \mathbf{0}.}$$

This means that X maps a non-zero direction of the parameter space exactly to zero.

Consequently, at least one singular value must be zero:

$$\boxed{\sigma_{\min}(X) = 0.}$$

The matrix does not have full column rank, and

$$X^\top X$$

is singular.

Therefore,

$$(X^\top X)^{-1}$$

does not exist.

This is the case of **perfect multicollinearity**.

Near Linear Dependence More commonly, the columns are not exactly linearly dependent but are **nearly** linearly dependent.

Then there exists some vector

$$\mathbf{a}$$

with

$$\|\mathbf{a}\|_2 = 1$$

such that

$$X\mathbf{a} \approx \mathbf{0}.$$

For example, we might have

$$X_3 \approx 2X_1 + 3X_2.$$

Equivalently,

$$2X_1 + 3X_2 - X_3 \approx 0.$$

Hence, for

$$\mathbf{a} = \begin{pmatrix} 2 \\ 3 \\ -1 \end{pmatrix},$$

after normalization if necessary,

$$X\mathbf{a} \approx 0.$$

This implies that the smallest singular value is close to zero:

$$\boxed{\sigma_{\min}(X) \approx 0.}$$

This is precisely the situation in which the regression design matrix is poorly conditioned.

Definition of the Condition Number

Using the Euclidean matrix norm, the condition number of X is

$$\boxed{\kappa_2(X) = \frac{\sigma_{\max}(X)}{\sigma_{\min}(X)}.}$$

Since the singular values are ordered as

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k,$$

we can equivalently write

$$\boxed{\kappa_2(X) = \frac{\sigma_1}{\sigma_k}.}$$

The condition number therefore compares the direction that is most strongly represented in the design matrix with the direction that is most weakly represented.

Interpretation of the Condition Number

If all singular values are of similar magnitude, then

$$\sigma_{\max} \approx \sigma_{\min},$$

and therefore

$$\kappa(X) \approx 1.$$

This corresponds to a well-conditioned matrix.

By contrast, if

$$\sigma_{\min} \ll \sigma_{\max},$$

then

$$\boxed{\kappa(X) \gg 1.}$$

This means that the matrix is ill-conditioned.

In the extreme case of exact linear dependence,

$$\sigma_{\min} = 0,$$

so

$$\boxed{\kappa(X) = \infty.}$$

We can summarize the intuition as

$\sigma_{\min}$ large relative to $\sigma_{\max}$	$\Longrightarrow$	well-conditioned matrix,
$\sigma_{\min} \approx 0$	$\Longrightarrow$	ill-conditioned matrix,
$\sigma_{\min} = 0$	$\Longrightarrow$	singular matrix.

Why Does This Matter for OLS?

Recall that

$$\widehat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}.$$

Using the SVD,

$$X = U \Sigma V^\top,$$

the OLS estimator can be expressed, when X has full column rank, as

$$\boxed{\widehat{\boldsymbol{\beta}} = V \Sigma^{-1} U^\top \mathbf{y}.}$$

Since

$$\Sigma^{-1} = \begin{pmatrix} 1/\sigma_1 & 0 & \cdots & 0 \\ 0 & 1/\sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1/\sigma_k \end{pmatrix},$$

a very small singular value

$$\sigma_k \approx 0$$

produces a very large inverse singular value:

$$\boxed{\frac{1}{\sigma_k} \gg 1.}$$

Therefore, small perturbations in the data can be strongly amplified when estimating the coefficients.

This explains why an ill-conditioned design matrix can produce unstable coefficient estimates.

Connection with the Variance of OLS Coefficients

Under the classical homoskedastic OLS assumptions,

$$\operatorname{Var}\left(\widehat{\boldsymbol{\beta}} \mid X\right) = \sigma_\varepsilon^2 (X^\top X)^{-1}.$$

Using

$$X^\top X = V\Sigma^2 V^\top,$$

we obtain

$$(X^\top X)^{-1} = \left(V\Sigma^2 V^\top\right)^{-1}.$$

Recall that, for three invertible matrices A, B, and C,

$$\boxed{(ABC)^{-1} = C^{-1}B^{-1}A^{-1}.}$$

Therefore,

$$\begin{aligned}(X^\top X)^{-1} &= \left(V\Sigma^2 V^\top\right)^{-1} \\ &= (V^\top)^{-1}(\Sigma^2)^{-1}V^{-1}.\end{aligned}$$

Since V is an orthogonal matrix,

$$V^\top V = VV^\top = I,$$

which implies

$$\boxed{V^{-1} = V^\top}$$

and consequently

$$\boxed{(V^\top)^{-1} = V.}$$

Moreover, since Σ is diagonal,

$$\Sigma = \begin{pmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_k \end{pmatrix},$$

we have

$$\Sigma^2 = \begin{pmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_k^2 \end{pmatrix}.$$

Assuming that X has full column rank, all singular values satisfy

$$\sigma_j > 0, \qquad j = 1, \ldots, k,$$

so Σ^2 is invertible. Its inverse is obtained simply by taking the reciprocal of each diagonal element:

$$(\Sigma^2)^{-1} = \begin{pmatrix} 1/\sigma_1^2 & 0 & \cdots & 0 \\ 0 & 1/\sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1/\sigma_k^2 \end{pmatrix}.$$

We denote this matrix by

$$\boxed{(\Sigma^2)^{-1} = \Sigma^{-2}.}$$

Putting these results together,

$$\begin{aligned}(X^\top X)^{-1} &= (V^\top)^{-1}(\Sigma^2)^{-1}V^{-1} \\ &= V\Sigma^{-2}V^\top.\end{aligned}$$

Hence,

$$\boxed{(X^\top X)^{-1} = V\Sigma^{-2}V^\top,}$$

where

$$\boxed{\Sigma^{-2} = \begin{pmatrix} 1/\sigma_1^2 & 0 & \cdots & 0 \\ 0 & 1/\sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1/\sigma_k^2 \end{pmatrix}.}$$

This result is particularly important in OLS because a very small singular value,

$$\sigma_j \approx 0,$$

generates a very large inverse squared singular value,

$$\boxed{\frac{1}{\sigma_j^2} \gg 1.}$$

Since

$$\mathrm{Var}\left(\widehat{\boldsymbol{\beta}} \mid X\right) = \sigma_\varepsilon^2 (X^\top X)^{-1} = \sigma_\varepsilon^2 V\Sigma^{-2}V^\top,$$

small singular values can therefore strongly amplify the variance of some linear combinations of the estimated OLS coefficients.

Therefore,

$$\boxed{\mathrm{Var}\left(\widehat{\boldsymbol{\beta}} \mid X\right) = \sigma_\varepsilon^2 V\Sigma^{-2}V^\top.}$$

If one singular value is very small,

$$\sigma_k \approx 0,$$

then

$$\frac{1}{\sigma_k^2}$$

becomes very large.

Hence, near-linear dependence between regressors can generate large variances and standard errors for some combinations of the estimated coefficients.

Connection with Multicollinearity

Suppose, for example, that

$$X_3 \approx 2X_1 + 3X_2.$$

Then the three regressors contain almost redundant information.

The regression may therefore have difficulty distinguishing the individual effects of

$$X_1, \qquad X_2, \qquad X_3.$$

This can lead to

unstable coefficient estimates, large standard errors, small individual t-statistics, coefficients sensitive to small changes in the data.

At the same time, the variables may still be jointly useful for explaining Y.

Therefore, it is possible to observe

a significant global F-test

while some individual coefficients have insignificant t-tests.

A Large Condition Number Does Not Necessarily Mean Multicollinearity

A crucial point is that a large condition number does **not** automatically imply strong multicollinearity.

The condition number is also sensitive to the **scale of the regressors**.

Suppose, for example, that

$$X_1 \sim 10^{-2}$$

while

$$X_2 \sim 10^4.$$

Even if X_1 and X_2 are not strongly correlated, their very different scales can produce singular values of very different magnitudes.

Consequently,

$$\frac{\sigma_{\max}}{\sigma_{\min}}$$

may become very large.

Thus,

$$\boxed{\text{large condition number} \not\Longrightarrow \text{multicollinearity necessarily.}}$$

Instead, a large condition number means more generally that

$$\boxed{\text{the design matrix is ill-conditioned.}}$$

The source of the ill-conditioning must then be investigated.

Checking the Effect of Variable Scaling

One useful diagnostic is to standardize the explanatory variables.

For regressor j,

$$\boxed{Z_{ij} = \frac{X_{ij} - \overline{X}_j}{s_j},}$$

where

$$\overline{X}_j$$

is the sample mean and

$$s_j$$

is the sample standard deviation of regressor j.

After standardization,

$$\mathbb{E}[Z_j] \approx 0$$

and

$$\mathrm{Var}(Z_j) \approx 1.$$

The regressors are therefore placed on comparable scales.

If the condition number falls dramatically after standardization, then a large part of the original condition number was caused by differences in scale.

If it remains very large, this provides stronger evidence of near-linear dependencies among the regressors.

Condition Number versus Variance Inflation Factor

Another useful diagnostic for multicollinearity is the **Variance Inflation Factor**.

For each explanatory variable X_j, regress it on all the other explanatory variables and compute

$$R_j^2.$$

The VIF of variable j is

$$\boxed{VIF_j = \frac{1}{1 - R_j^2}.}$$

If

$$R_j^2 \approx 1,$$

then X_j can be almost perfectly explained by the other regressors.

Consequently,

$$1 - R_j^2 \approx 0$$

and

$$\boxed{VIF_j \gg 1.}$$

The condition number and VIF therefore provide related but different information:

Condition number	$\longrightarrow$	overall conditioning of the design matrix,
VIF_j	$\longrightarrow$	multicollinearity associated with a particular regressor X_j.

Application to the Regression Output

The regression output reports

$$\boxed{\text{Cond. No.} = 1.53 \times 10^4.}$$

That is,

$$\boxed{\kappa(X) \approx 15{,}300.}$$

This is a large condition number.

For this reason, `statsmodels` reports the warning:

> The condition number is large. This might indicate that there are strong multicollinearity or other numerical problems.

The word **might** is important.

The large condition number indicates that the design matrix is poorly conditioned, but this can result from

strong multicollinearity, very different scales across regressors, or a combination of both.

Therefore, one should not conclude directly from

$$\kappa(X) = 15{,}300$$

that the regression necessarily suffers from severe multicollinearity.

Instead, useful next steps are

inspect the scales of the explanatory variables, standardize the regressors and recompute the condition number, inspect pairwise correlations, compute the VIFs.

Correct Specification of the Error Covariance Matrix

The OLS regression summary produced by `statsmodels` may contain the following note:

> `[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.`

This note concerns the validity of the **standard errors** and, consequently, the validity of the t-statistics, p-values, and confidence intervals reported by the regression.

It does not, by itself, mean that `statsmodels` has detected a problem. Rather, it reminds us that statistical inference depends on the covariance structure assumed for the regression errors.

The Classical OLS Error Covariance Assumption

Consider the linear regression model

$$\boxed{\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon},}$$

where

$$\mathbf{y} \in \mathbb{R}^n, \qquad X \in \mathbb{R}^{n \times k}, \qquad \boldsymbol{\beta} \in \mathbb{R}^k, \qquad \boldsymbol{\varepsilon} \in \mathbb{R}^n.$$

The error vector is

$$\boldsymbol{\varepsilon} = \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}.$$

With the default

`Covariance Type: nonrobust`

the classical OLS covariance assumption is

$$\boxed{\mathrm{Var}\left(\boldsymbol{\varepsilon} \mid X\right) = \sigma^2 I_n.}$$

Explicitly,

$$\sigma^2 I_n = \begin{pmatrix} \sigma^2 & 0 & \cdots & 0 \\ 0 & \sigma^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma^2 \end{pmatrix}.$$

This assumption contains two important components.

1. Homoskedasticity All observations are assumed to have the same conditional error variance:

$$\boxed{\mathrm{Var}\left(\varepsilon_i \mid X\right) = \sigma^2 \qquad \forall i.}$$

Thus, the magnitude of the error variance does not systematically change across observations.

2. No Correlation Between Errors For two different observations,

$$i \neq j,$$

the errors are assumed to be conditionally uncorrelated:

$$\boxed{\mathrm{Cov}\left(\varepsilon_i, \varepsilon_j \mid X\right) = 0.}$$

For time-series data, this assumption implies, in particular, the absence of serial correlation such as

$$\operatorname{Cov}(\varepsilon_t, \varepsilon_{t-1} \mid X) = 0.$$

Therefore,

$$\boxed{\operatorname{Var}(\boldsymbol{\varepsilon} \mid X) = \sigma^2 I_n}$$

combines constant error variance and zero covariance between different errors.

OLS Estimator

The OLS estimator is

$$\boxed{\widehat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}.}$$

Substituting

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon},$$

we obtain

$$\begin{aligned}
\widehat{\boldsymbol{\beta}} &= (X^\top X)^{-1} X^\top (X\boldsymbol{\beta} + \boldsymbol{\varepsilon}) \\
&= (X^\top X)^{-1} X^\top X\boldsymbol{\beta} + (X^\top X)^{-1} X^\top \boldsymbol{\varepsilon} \\
&= \boldsymbol{\beta} + (X^\top X)^{-1} X^\top \boldsymbol{\varepsilon}.
\end{aligned}$$

Hence,

$$\boxed{\widehat{\boldsymbol{\beta}} - \boldsymbol{\beta} = (X^\top X)^{-1} X^\top \boldsymbol{\varepsilon}.}$$

Covariance Matrix of the OLS Estimator

Conditional on X,

$$\operatorname{Var}\left(\widehat{\boldsymbol{\beta}} \mid X\right) = \operatorname{Var}\left[(X^\top X)^{-1} X^\top \boldsymbol{\varepsilon} \mid X\right].$$

Using the general rule

$$\operatorname{Var}(A\mathbf{x}) = A \operatorname{Var}(\mathbf{x}) A^\top,$$

we obtain

$$\boxed{\operatorname{Var}\left(\widehat{\boldsymbol{\beta}} \mid X\right) = (X^\top X)^{-1} X^\top \operatorname{Var}(\boldsymbol{\varepsilon} \mid X) X (X^\top X)^{-1}.}$$

This is the **general expression** for the covariance matrix of the OLS estimator.

Classical Homoskedastic Case

Under the classical assumption,

$$\mathrm{Var}(\boldsymbol{\varepsilon} \mid X) = \sigma^2 I_n.$$

Substituting into the general formula gives

$$\begin{aligned}\mathrm{Var}\left(\widehat{\boldsymbol{\beta}} \mid X\right) &= (X^\top X)^{-1} X^\top (\sigma^2 I_n) X (X^\top X)^{-1} \\ &= \sigma^2 (X^\top X)^{-1} X^\top X (X^\top X)^{-1} \\ &= \boxed{\sigma^2 (X^\top X)^{-1}.}\end{aligned}$$

Since the true error variance σ^2 is unknown, it must be estimated.

The usual estimator is

$$\boxed{\widehat{\sigma}^2 = \frac{\sum_{i=1}^{n} \widehat{\varepsilon}_i^2}{n-k}.}$$

The classical estimated covariance matrix is therefore

$$\boxed{\widehat{\mathrm{Var}}\left(\widehat{\boldsymbol{\beta}}\right) = \widehat{\sigma}^2 (X^\top X)^{-1}.}$$

From the Covariance Matrix to Standard Errors

The diagonal elements of

$$\widehat{\mathrm{Var}}\left(\widehat{\boldsymbol{\beta}}\right)$$

are the estimated variances of the individual coefficient estimators.

For coefficient j,

$$\boxed{\widehat{\mathrm{Var}}(\widehat{\beta}_j) = \left[\widehat{\mathrm{Var}}(\widehat{\boldsymbol{\beta}})\right]_{jj}.}$$

The corresponding standard error is

$$\boxed{SE(\widehat{\beta}_j) = \sqrt{\left[\widehat{\mathrm{Var}}(\widehat{\boldsymbol{\beta}})\right]_{jj}}.}$$

The standard error is then used to construct the t-statistic:

$$\boxed{t_j = \frac{\widehat{\beta}_j - \beta_{j,0}}{SE(\widehat{\beta}_j)}.}$$

For the usual null hypothesis

$$H_0 : \beta_j = 0,$$

this becomes

$$\boxed{t_j = \frac{\widehat{\beta}_j}{SE(\widehat{\beta}_j)}.}$$

The t-statistic is subsequently used to compute the p-value and confidence interval.

Therefore, there is a direct chain:

$$\boxed{\text{error covariance assumption} \longrightarrow \widehat{\text{Var}}(\widehat{\boldsymbol{\beta}}) \longrightarrow SE(\widehat{\beta}_j) \longrightarrow t_j \longrightarrow p\text{-value and confidence interval.}}$$

What Happens if the Error Covariance Is Misspecified?

Suppose that the classical assumption

$$\text{Var}(\boldsymbol{\varepsilon} \mid X) = \sigma^2 I_n$$

is incorrect.

More generally, suppose

$$\boxed{\text{Var}(\boldsymbol{\varepsilon} \mid X) = \Omega,}$$

where

$$\Omega \neq \sigma^2 I_n.$$

Then the correct covariance matrix of the OLS estimator is

$$\boxed{\text{Var}\left(\widehat{\boldsymbol{\beta}} \mid X\right) = (X^\top X)^{-1} X^\top \Omega X (X^\top X)^{-1}.}$$

Consequently, the classical formula

$$\widehat{\sigma}^2 (X^\top X)^{-1}$$

is generally no longer an appropriate estimator of the covariance matrix.

Example: Heteroskedasticity Suppose that

$$\text{Var}(\varepsilon_i \mid X) = \sigma_i^2,$$

so that the error variance differs across observations.

Then

$$\Omega = \begin{pmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^2 \end{pmatrix}.$$

Unless

$$\sigma_1^2 = \sigma_2^2 = \cdots = \sigma_n^2,$$

we have

$$\Omega \neq \sigma^2 I_n.$$

Therefore, the conventional nonrobust OLS standard errors may be incorrect.

Example: Serial Correlation For time-series data, errors may also be correlated over time:

$$\mathrm{Cov}(\varepsilon_t, \varepsilon_{t-h} \mid X) \neq 0.$$

In that case, Ω contains non-zero off-diagonal elements:

$$\Omega = \begin{pmatrix} \sigma_1^2 & \mathrm{Cov}(\varepsilon_1, \varepsilon_2) & \cdots \\ \mathrm{Cov}(\varepsilon_2, \varepsilon_1) & \sigma_2^2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}.$$

Again,

$$\Omega \neq \sigma^2 I_n,$$

so the classical covariance estimator may be inappropriate.

Coefficient Estimation versus Statistical Inference

A crucial distinction must be made between the OLS coefficient estimates and their estimated standard errors.

Under the exogeneity assumption

$$\boxed{\mathbb{E}[\boldsymbol{\varepsilon} \mid X] = \mathbf{0},}$$

we have

$$\begin{aligned} \mathbb{E}[\widehat{\boldsymbol{\beta}} \mid X] &= \boldsymbol{\beta} + (X^\top X)^{-1} X^\top \mathbb{E}[\boldsymbol{\varepsilon} \mid X] \\ &= \boldsymbol{\beta}. \end{aligned}$$

Therefore,

$$\boxed{\mathbb{E}[\widehat{\boldsymbol{\beta}} \mid X] = \boldsymbol{\beta}.}$$

Thus, heteroskedasticity or serial correlation does not, by itself, necessarily make the OLS coefficient estimator biased when the relevant exogeneity conditions hold.

However, the conventional covariance estimator may be incorrect.

Therefore,

incorrect covariance specification
$\Downarrow$
incorrect estimated coefficient variances
$\Downarrow$
incorrect standard errors
$\Downarrow$
incorrect t-statistics
$\Downarrow$
potentially incorrect p-values and confidence intervals.

For example, suppose

$$\widehat{\beta}_j = 0.10.$$

With a classical standard error of

$$SE_{\text{classical}}(\widehat{\beta}_j) = 0.04,$$

we obtain

$$t = \frac{0.10}{0.04} = 2.5.$$

This might suggest statistical significance.

However, suppose a robust covariance estimator gives

$$SE_{\text{robust}}(\widehat{\beta}_j) = 0.07.$$

Then

$$t_{\text{robust}} = \frac{0.10}{0.07} \approx 1.43.$$

The estimated coefficient is unchanged:

$$\boxed{\widehat{\beta}_j = 0.10,}$$

but the statistical inference changes because the estimated uncertainty around the coefficient has changed.

Heteroskedasticity-Robust Standard Errors

If heteroskedasticity is suspected, one can use a heteroskedasticity-consistent covariance estimator.

For example, in `statsmodels`,

```
model = sm.OLS(y, X).fit(cov_type='HC3')
```

uses HC3 robust standard errors.

The OLS coefficient estimates remain the same:

$$\boxed{\widehat{\boldsymbol{\beta}}_{\mathrm{HC3}} = \widehat{\boldsymbol{\beta}}_{\mathrm{OLS}},}$$

but the estimated covariance matrix changes:

$$\boxed{\widehat{\mathrm{Var}}_{\mathrm{HC3}}(\widehat{\boldsymbol{\beta}}) \neq \widehat{\sigma}^2 (X^\top X)^{-1}.}$$

Consequently, the standard errors, t-statistics, p-values, and confidence intervals may change.

HAC / Newey–West Standard Errors

For time-series regressions, the residuals may exhibit both

$$\boxed{\text{heteroskedasticity}}$$

and

$$\boxed{\text{serial correlation.}}$$

In this situation, a Heteroskedasticity and Autocorrelation Consistent (HAC) covariance estimator, such as Newey–West, can be used.

For example,

```
model = sm.OLS(y, X).fit(
    cov_type='HAC',
    cov_kwds={'maxlags': 5}
)
```

allows the estimated covariance matrix to be robust to both heteroskedasticity and autocorrelation up to the chosen lag structure.

Again, the OLS coefficient estimates are unchanged, but their estimated covariance matrix changes.

Alternative Least-Squares Estimators

In this section, we focus primarily on the estimation of linear regression models using **Ordinary Least Squares (OLS)**. However, other least-squares estimators can be used when the classical assumptions on the error covariance matrix are not appropriate.

Ordinary Least Squares (OLS) OLS assumes, in its classical covariance specification,

$$\mathrm{Var}(\boldsymbol{\varepsilon} \mid X) = \sigma^2 I.$$

It estimates the coefficients by minimizing the unweighted sum of squared residuals:

$$\boxed{\widehat{\boldsymbol{\beta}}_{\mathrm{OLS}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} \left(y_i - \mathbf{x}_i^\top \boldsymbol{\beta}\right)^2.}$$

Weighted Least Squares (WLS) WLS is useful when observations have different error variances (**heteroskedasticity**). Observations are assigned different weights:

$$\boxed{\widehat{\boldsymbol{\beta}}_{\mathrm{WLS}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} w_i \left(y_i - \mathbf{x}_i^\top \boldsymbol{\beta}\right)^2.}$$

Typically, observations with higher error variance receive lower weights:

$$w_i \propto \frac{1}{\sigma_i^2}.$$

Generalized Least Squares (GLS) GLS is the more general case in which

$$\boxed{\mathrm{Var}(\boldsymbol{\varepsilon} \mid X) = \Omega,}$$

where Ω may contain both unequal variances and correlations between errors.

GLS estimates

$$\boxed{\widehat{\boldsymbol{\beta}}_{\mathrm{GLS}} = (X^\top \Omega^{-1} X)^{-1} X^\top \Omega^{-1} \mathbf{y}.}$$

WLS can therefore be viewed as a special case of GLS in which Ω is diagonal:

$$\boxed{\Omega = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_n^2).}$$

Hence,

$$\boxed{\mathrm{OLS} \subset \mathrm{WLS} \subset \mathrm{GLS},}$$

in the sense that OLS corresponds to equal error variances, WLS allows observation-specific variances, and GLS additionally allows a general covariance structure between errors.

Quantiles and Value at Risk

Value at Risk (VaR) is a risk measure based on the quantiles of the return or loss distribution. We first define the quantile function before presenting the main approaches used to estimate VaR.

Quantile Function

Let X be a random variable with cumulative distribution function (CDF)

$$\boxed{F_X(x) = \mathbb{P}(X \leq x).}$$

For a probability level

$$\alpha \in (0, 1),$$

the α-quantile of X is defined as

$$\boxed{q_\alpha(X) = F_X^{-1}(\alpha) = \inf\left\{x \in \mathbb{R} : F_X(x) \geq \alpha\right\}.}$$

In words, q_α is the value below which approximately $100\alpha\%$ of the distribution lies.

For a continuous and strictly increasing CDF,

$$\boxed{F_X(q_\alpha) = \alpha.}$$

Equivalently,

$$\boxed{\mathbb{P}(X \leq q_\alpha) = \alpha.}$$

For example, if

$$q_{0.05}(R) = -0.0225,$$

then

$$\mathbb{P}\left(R \leq -0.0225\right) = 0.05.$$

Thus, approximately 5% of returns are below -2.25%.

Empirical Quantile Given a sample

$$X_1, \ldots, X_T,$$

sort the observations:

$$X_{(1)} \leq X_{(2)} \leq \cdots \leq X_{(T)}.$$

A simple empirical estimator of the α-quantile is approximately

$$\boxed{\widehat{q}_\alpha = X_{(\lceil \alpha T \rceil)}.}$$

In practice, statistical software may interpolate between adjacent observations, so the exact sample quantile can depend slightly on the interpolation convention.

For example, in Python,

```
alpha = 0.05
q = returns.quantile(alpha)
```

computes the empirical 5% quantile of the returns.

Definition of Value at Risk

Let R_{t+1} denote the portfolio return over the chosen risk horizon.

If VaR is reported as a **positive loss number**, the VaR at confidence level $1 - \alpha$ is

$$\boxed{\mathrm{VaR}_{1-\alpha} = -q_\alpha(R_{t+1}).}$$

Equivalently,

$$\boxed{\mathrm{VaR}_{1-\alpha} = -F_R^{-1}(\alpha).}$$

For example, for a 95% VaR,

$$\alpha = 0.05,$$

so

$$\boxed{\mathrm{VaR}_{95\%} = -q_{0.05}(R).}$$

If

$$q_{0.05}(R) = -2.25\%,$$

then

$$\boxed{\mathrm{VaR}_{95\%} = 2.25\%.}$$

This means that, according to the estimated return distribution, there is a 5% probability of observing a return below -2.25%:

$$\boxed{\mathbb{P}\left(R < -2.25\%\right) \approx 5\%.}$$

Equivalently, with approximately 95% probability, the loss does not exceed 2.25% over the specified horizon.

Loss Convention Instead of working with returns, define the loss variable

$$\boxed{L = -R.}$$

Large positive values of L correspond to large losses.

The VaR can then be written directly as the upper quantile of the loss distribution:

$$\boxed{\mathrm{VaR}_{1-\alpha} = q_{1-\alpha}(L) = F_L^{-1}(1-\alpha).}$$

Thus,

$$\boxed{-q_\alpha(R) = q_{1-\alpha}(-R).}$$

This explains why a 95% VaR corresponds either to

$$\boxed{\text{the 5\% lower-tail quantile of returns}}$$

or equivalently to

$$\boxed{\text{the 95\% upper-tail quantile of losses.}}$$

Main Methods for Estimating VaR

There are several approaches to estimating the quantile required for VaR. The main methods are:

1. Historical VaR;
2. Parametric VaR;
3. Monte Carlo VaR;
4. Conditional or volatility-model-based VaR.

1. Historical VaR

The historical approach makes no explicit parametric assumption about the distribution of returns.

Given historical returns

$$R_1, \ldots, R_T,$$

we estimate the lower-tail quantile directly from the empirical distribution:

$$\boxed{\widehat{\mathrm{VaR}}^{\mathrm{Hist}}_{1-\alpha} = -\widehat{q}_\alpha(R).}$$

For example, for a 95% VaR,

$$\boxed{\widehat{\mathrm{VaR}}_{95\%}^{\mathrm{Hist}} = -\widehat{q}_{0.05}(R).}$$

In Python,

```
alpha = 0.05
var_95 = -returns.quantile(alpha)
```

The historical approach therefore lets the empirical observations determine the shape of the distribution.

$$\boxed{\text{Historical VaR} \quad \Longrightarrow \quad \text{empirical quantile of past returns.}}$$

Its main advantage is that it does not impose a Gaussian distribution.

However, it assumes that the historical sample is representative of the future risk distribution, and tail estimation can be imprecise when few extreme observations are available.

2. Parametric VaR

The parametric approach assumes a particular probability distribution for returns.

Suppose, for example, that

$$\boxed{R \sim \mathcal{N}(\mu, \sigma^2).}$$

We can write

$$R = \mu + \sigma Z, \qquad Z \sim \mathcal{N}(0,1).$$

Let

$$z_\alpha = \Phi^{-1}(\alpha)$$

denote the α-quantile of the standard normal distribution.

Then

$$\boxed{q_\alpha(R) = \mu + \sigma z_\alpha.}$$

Therefore,

$$\boxed{\mathrm{VaR}_{1-\alpha}^{\mathrm{Normal}} = -\left(\mu + \sigma z_\alpha\right).}$$

Since

$$z_{0.05} \approx -1.645,$$

the 95% Gaussian VaR is

$$\boxed{\mathrm{VaR}_{95\%}^{\mathrm{Normal}} = -\mu + 1.645\sigma.}$$

If the expected return over the horizon is small relative to volatility, one often uses the approximation

$$\boxed{\mathrm{VaR}_{95\%} \approx 1.645\sigma.}$$

Similarly,

$$z_{0.01} \approx -2.326,$$

so

$$\boxed{\mathrm{VaR}_{99\%}^{\mathrm{Normal}} = -\mu + 2.326\sigma.}$$

The main limitation is that financial returns frequently exhibit skewness and heavy tails, so a Gaussian assumption may underestimate extreme risks.

Derivation of the Gaussian Quantile Suppose that

$$\boxed{R \sim \mathcal{N}(\mu, \sigma^2).}$$

We can standardize R by defining

$$\boxed{Z = \frac{R - \mu}{\sigma},}$$

where

$$Z \sim \mathcal{N}(0, 1).$$

By definition, the α-quantile of R, denoted $q_\alpha(R)$, satisfies

$$\boxed{\mathbb{P}\left(R \leq q_\alpha(R)\right) = \alpha.}$$

Since $\sigma > 0$, we can subtract μ and divide both sides of the inequality by σ without changing the direction of the inequality:

$$\mathbb{P}\left(\frac{R - \mu}{\sigma} \leq \frac{q_\alpha(R) - \mu}{\sigma}\right) = \alpha.$$

Using

$$Z = \frac{R - \mu}{\sigma},$$

this becomes

$$\mathbb{P}\left(Z \leq \frac{q_\alpha(R) - \mu}{\sigma}\right) = \alpha.$$

Let

$$\Phi(z) = \mathbb{P}(Z \leq z)$$

denote the cumulative distribution function of the standard normal distribution.

Therefore,

$$\Phi\left(\frac{q_\alpha(R) - \mu}{\sigma}\right) = \alpha.$$

Applying the inverse cumulative distribution function Φ^{-1} to both sides gives

$$\frac{q_\alpha(R) - \mu}{\sigma} = \Phi^{-1}(\alpha).$$

Define

$$\boxed{z_\alpha = \Phi^{-1}(\alpha),}$$

where z_α is the α-quantile of the standard normal distribution.

Hence,

$$\frac{q_\alpha(R) - \mu}{\sigma} = z_\alpha.$$

Multiplying by σ,

$$q_\alpha(R) - \mu = \sigma z_\alpha,$$

and adding μ gives

$$\boxed{q_\alpha(R) = \mu + \sigma z_\alpha.}$$

Other Parametric Distributions Parametric VaR does not necessarily require a Gaussian distribution.

If

$$R \sim F(\theta),$$

where F is some distribution with parameters θ, then

$$\boxed{\text{VaR}_{1-\alpha} = -F^{-1}(\alpha; \theta).}$$

For example, one may use a Student-t distribution to allow for heavier tails than under the Gaussian assumption.

3. Monte Carlo VaR

Monte Carlo VaR is based on simulating many possible future scenarios from a chosen model.

Suppose we simulate

$$R_{t+1}^{(1)}, R_{t+1}^{(2)}, \ldots, R_{t+1}^{(M)}.$$

These simulated returns form an empirical approximation to the future return distribution.

The VaR is then estimated from their empirical quantile:

$$\boxed{\widehat{\mathrm{VaR}}_{1-\alpha}^{\mathrm{MC}} = -\widehat{q}_\alpha \left(R_{t+1}^{(1)}, \ldots, R_{t+1}^{(M)} \right).}$$

The procedure is therefore

$$\boxed{\text{Model} \rightarrow \text{Simulate scenarios} \rightarrow \text{Construct empirical distribution} \rightarrow \text{Take tail quantile.}}$$

Monte Carlo simulation is particularly useful when portfolio returns depend non-linearly on several risk factors, for example for portfolios containing options.

4. Conditional VaR Using Volatility Models

The previous approaches can be extended by allowing the return distribution to change over time.

Suppose

$$\boxed{R_{t+1} = \mu_{t+1|t} + \sigma_{t+1|t} Z_{t+1},}$$

where

$$\mu_{t+1|t} = \mathbb{E}_t[R_{t+1}]$$

is the conditional expected return and

$$\sigma_{t+1|t}^2 = \mathrm{Var}_t(R_{t+1})$$

is the conditional variance.

If

$$Z_{t+1} \sim \mathcal{N}(0,1),$$

then

$$q_{\alpha,t} = \mu_{t+1|t} + \sigma_{t+1|t} z_\alpha.$$

Therefore,

$$\boxed{\mathrm{VaR}_{1-\alpha,t} = -\left(\mu_{t+1|t} + \sigma_{t+1|t} z_\alpha\right).}$$

For a 95% confidence level,

$$\boxed{\mathrm{VaR}_{95\%,t} = -\mu_{t+1|t} + 1.645\sigma_{t+1|t}.}$$

The conditional volatility

$$\sigma_{t+1|t}$$

can, for example, be estimated using a GARCH model.

For a GARCH(1,1),

$$\boxed{\sigma^2_{t+1|t} = \omega + \alpha\varepsilon_t^2 + \beta\sigma_t^2.}$$

Thus, unlike a VaR based on a constant unconditional volatility, the GARCH VaR changes through time:

$$\boxed{\text{high predicted volatility} \Longrightarrow \text{higher VaR.}}$$

Predicted VaR Using GARCH and Standardized Residuals

The predicted VaR considered here combines two components:

1. a GARCH model to estimate the time-varying conditional volatility;
2. a quantile of the standardized residual distribution to characterize the tail of the return distribution.

This approach can be understood from the general conditional return model.

Conditional Return Model

Suppose that the return at time $t+1$ can be written as

$$\boxed{R_{t+1} = \mu_{t+1|t} + \sigma_{t+1|t} Z_{t+1},}$$

where

$$\mu_{t+1|t} = \mathbb{E}_t[R_{t+1}]$$

is the conditional expected return,

$$\sigma^2_{t+1|t} = \mathrm{Var}_t(R_{t+1})$$

is the conditional variance, and

$$Z_{t+1}$$

is a standardized innovation satisfying

$$\boxed{\mathbb{E}[Z_{t+1}] = 0, \qquad \mathrm{Var}(Z_{t+1}) = 1.}$$

The GARCH model is used to forecast

$$\boxed{\sigma_{t+1|t}.}$$

For example, under a GARCH(1,1) model,

$$\boxed{\sigma^2_{t+1|t} = \omega + \alpha\varepsilon_t^2 + \beta\sigma_t^2.}$$

Conditional Quantile of Returns

Let

$$q_\alpha^Z$$

denote the α-quantile of the standardized innovation Z:

$$\boxed{q_\alpha^Z = F_Z^{-1}(\alpha).}$$

By definition,

$$\mathbb{P}\left(Z_{t+1} \le q_\alpha^Z\right) = \alpha.$$

Starting from

$$R_{t+1} = \mu_{t+1|t} + \sigma_{t+1|t} Z_{t+1},$$

we want to find the conditional α-quantile of R_{t+1}, denoted

$$q_{\alpha,t}(R_{t+1}).$$

By definition,

$$\mathbb{P}_t\left(R_{t+1} \le q_{\alpha,t}(R_{t+1})\right) = \alpha.$$

Substituting the conditional return model,

$$\mathbb{P}_t\left(\mu_{t+1|t} + \sigma_{t+1|t} Z_{t+1} \le q_{\alpha,t}(R_{t+1})\right) = \alpha.$$

Since

$$\sigma_{t+1|t} > 0,$$

we can rearrange the inequality:

$$\mathbb{P}_t\left(Z_{t+1} \leq \frac{q_{\alpha,t}(R_{t+1}) - \mu_{t+1|t}}{\sigma_{t+1|t}}\right) = \alpha.$$

Therefore,

$$\frac{q_{\alpha,t}(R_{t+1}) - \mu_{t+1|t}}{\sigma_{t+1|t}} = q_\alpha^Z.$$

Hence,

$$q_{\alpha,t}(R_{t+1}) - \mu_{t+1|t} = \sigma_{t+1|t} q_\alpha^Z,$$

and finally,

$$\boxed{q_{\alpha,t}(R_{t+1}) = \mu_{t+1|t} + \sigma_{t+1|t} q_\alpha^Z.}$$

This is the general conditional quantile formula.

Gaussian GARCH VaR

Suppose first that the standardized innovations are assumed to follow a standard normal distribution:

$$\boxed{Z_{t+1} \sim \mathcal{N}(0,1).}$$

Let

$$\Phi$$

denote the standard normal CDF.

Then

$$q_\alpha^Z = \Phi^{-1}(\alpha).$$

Defining

$$\boxed{z_\alpha = \Phi^{-1}(\alpha),}$$

the conditional return quantile becomes

$$\boxed{q_{\alpha,t}(R_{t+1}) = \mu_{t+1|t} + \sigma_{t+1|t} z_\alpha.}$$

GARCH with an Empirical Standardized-Residual Quantile

Instead of assuming

$$Z_t \sim \mathcal{N}(0,1),$$

we can estimate the distribution of Z_t directly from the standardized residuals of the GARCH model.

After estimating the model, define

$$\boxed{\widehat{z}_t = \frac{\widehat{\varepsilon}_t}{\widehat{\sigma}_t}.}$$

Here,

$$\widehat{\varepsilon}_t$$

is the estimated innovation and

$$\widehat{\sigma}_t$$

is the estimated conditional volatility.

If the conditional mean is assumed to be zero,

$$\boxed{\mu_t = 0,}$$

then the return model

$$R_t = \mu_t + \varepsilon_t$$

simplifies to

$$R_t = \varepsilon_t.$$

Indeed, the innovation is defined as the difference between the observed return and its conditional expected value:

$$\boxed{\varepsilon_t = R_t - \mu_t.}$$

Therefore, under the zero-mean assumption,

$$\varepsilon_t = R_t - 0 = R_t.$$

After estimating the model, this gives

$$\boxed{\widehat{\varepsilon}_t = R_t.}$$

In other words, when the model assumes that there is no predictable component in the conditional mean of returns, the entire observed return is interpreted as an innovation (or shock).

Consequently, the standardized residual is simply

$$\boxed{\widehat{z}_t = \frac{\widehat{\varepsilon}_t}{\widehat{\sigma}_t} = \frac{R_t}{\widehat{\sigma}_t}.}$$

More generally, if the conditional mean is not zero, then

$$\boxed{\widehat{\varepsilon}_t = R_t - \widehat{\mu}_t,}$$

and therefore

$$\boxed{\widehat{z}_t = \frac{R_t - \widehat{\mu}_t}{\widehat{\sigma}_t}.}$$

so that

$$\boxed{\widehat{z}_t = \frac{R_t}{\widehat{\sigma}_t}.}$$

The empirical α-quantile of the standardized residuals is then

$$\boxed{\widehat{q}_\alpha^Z = \widehat{F}_Z^{-1}(\alpha),}$$

where $\widehat{F}_Z$ is the empirical CDF of

$$\widehat{z}_1, \widehat{z}_2, \ldots, \widehat{z}_t.$$

Equivalently,

$$\boxed{\widehat{q}_\alpha^Z = \text{Quantile}_\alpha\left(\widehat{z}_1, \ldots, \widehat{z}_t\right).}$$

Thus, unlike the Gaussian approach, we do not impose

$$q_\alpha^Z = \Phi^{-1}(\alpha).$$

Instead, the tail quantile is estimated directly from the historical standardized innovations.

Predicted Return Quantile

Once we have

$$\widehat{\sigma}_{t+1|t}$$

from the GARCH model and

$$\widehat{q}_{\alpha}^{Z}$$

from the empirical standardized-residual distribution, the predicted conditional return quantile is

$$\boxed{\widehat{q}_{\alpha,t}(R_{t+1}) = \widehat{\mu}_{t+1|t} + \widehat{\sigma}_{t+1|t}\widehat{q}_{\alpha}^{Z}.}$$

If a zero-mean GARCH model is used, then

$$\widehat{\mu}_{t+1|t} = 0,$$

and the expression simplifies to

$$\boxed{\widehat{q}_{\alpha,t}(R_{t+1}) = \widehat{\sigma}_{t+1|t}\widehat{q}_{\alpha}^{Z}.}$$

This corresponds to the formula used in the notebook:

$$\boxed{\widehat{\mathrm{VaR}}_{\alpha,t+1|t} = \widehat{\sigma}_{t+1|t}\, q_{\alpha,t},}$$

where $q_{\alpha,t}$ denotes the empirical α-quantile of the standardized residuals.

Notice that, under this convention, the quantity called VaR is the **signed lower return quantile**, and is therefore typically negative for small values of α.

Numerical Example

Suppose the GARCH model predicts

$$\widehat{\sigma}_{t+1|t} = 2\% = 0.02.$$

Suppose also that the empirical 5% quantile of the standardized residuals is

$$\widehat{q}_{0.05}^{Z} = -1.92.$$

Then, under a zero-mean specification,

$$\begin{aligned}\widehat{q}_{0.05,t}(R_{t+1}) &= \widehat{\sigma}_{t+1|t}\widehat{q}_{0.05}^{Z} \\ &= 0.02 \times (-1.92) \\ &= -0.0384.\end{aligned}$$

Therefore,

$$\boxed{\widehat{q}_{0.05,t}(R_{t+1}) = -3.84\%.}$$

This means that the estimated conditional 5% return quantile for the next period is -3.84%.

Signed Return Quantile versus Positive-Loss VaR

Two conventions are commonly encountered.

Under the **signed return-quantile convention**,

$$\boxed{\mathrm{VaR}^{\mathrm{return}}_{\alpha,t+1|t} = q_{\alpha,t}(R_{t+1}),}$$

which is typically negative for a lower-tail quantile.

Thus, in the previous example,

$$\mathrm{VaR}^{\mathrm{return}}_{0.05,t+1|t} = -3.84\%.$$

Alternatively, VaR is often reported as a **positive loss number**:

$$\boxed{\mathrm{VaR}^{\mathrm{loss}}_{1-\alpha,t+1|t} = -q_{\alpha,t}(R_{t+1}).}$$

Therefore,

$$\boxed{\mathrm{VaR}^{\mathrm{loss}}_{95\%,t+1|t} = 3.84\%.}$$

The two numbers represent the same tail threshold but use different sign conventions.

Gaussian versus Empirical GARCH VaR

The two approaches have the same general structure:

$$\boxed{\text{Conditional quantile} = \text{conditional mean} + \text{conditional volatility} \times \text{standardized innovation quantile.}}$$

The difference concerns how the standardized innovation quantile is obtained.

For a Gaussian GARCH model,

$$\boxed{q^{Z}_{\alpha} = \Phi^{-1}(\alpha),}$$

so that

$$\boxed{q^{\mathrm{Gaussian}}_{\alpha,t} = \mu_{t+1|t} + \sigma_{t+1|t}\Phi^{-1}(\alpha).}$$

For the empirical standardized-residual approach,

$$\boxed{q^{Z}_{\alpha} = \widehat{F}^{-1}_{Z}(\alpha),}$$

so that

$$\boxed{q^{\mathrm{Empirical}}_{\alpha,t} = \mu_{t+1|t} + \sigma_{t+1|t}\widehat{F}^{-1}_{Z}(\alpha).}$$

Hence,

$$\boxed{\begin{array}{ll} \textbf{Gaussian GARCH} & q_\alpha^Z = \Phi^{-1}(\alpha) \quad \text{parametric tail shape,} \\ \textbf{Empirical GARCH} & q_\alpha^Z = \widehat{F}_Z^{-1}(\alpha) \quad \text{empirical tail shape.} \end{array}}$$

The empirical approach therefore separates the two components of the conditional return distribution:

$$\boxed{\underbrace{\widehat{\sigma}_{t+1|t}}_{\text{time-varying scale from GARCH}} + \underbrace{\widehat{F}_Z^{-1}(\alpha)}_{\text{tail shape from standardized residuals}}.}$$

More precisely, under a zero-mean model these components combine multiplicatively:

$$\boxed{\widehat{q}_{\alpha,t}(R_{t+1}) = \underbrace{\widehat{\sigma}_{t+1|t}}_{\text{predicted conditional volatility}} \times \underbrace{\widehat{F}_Z^{-1}(\alpha)}_{\text{empirical standardized-residual quantile}}.}$$

Thus, GARCH determines how the **scale of risk changes over time**, while the empirical standardized residuals determine the **shape and tail behavior** used to convert that volatility forecast into a conditional VaR threshold.

VaR in Monetary Units

If V_t denotes the current value of the portfolio and the VaR is expressed as a return, the corresponding monetary VaR is approximately

$$\boxed{\mathrm{VaR}_{1-\alpha}^{\$} = V_t \, \mathrm{VaR}_{1-\alpha}^{R}.}$$

For example, if

$$V_t = 1{,}000{,}000$$

and

$$\mathrm{VaR}_{95\%}^{R} = 2.5\%,$$

then

$$\boxed{\mathrm{VaR}_{95\%} = 1{,}000{,}000 \times 0.025 = 25{,}000.}$$

Summary

The central object behind VaR is the quantile function:

$$\boxed{q_\alpha(X) = F_X^{-1}(\alpha) = \inf\{x : F_X(x) \geq \alpha\}.}$$

When working with returns,

$$\boxed{\text{VaR}_{1-\alpha} = -q_\alpha(R).}$$

The main estimation approaches can be summarized as follows:

Method	**Distribution**	**VaR**
Historical	Empirical	$-\widehat{q}_\alpha(R)$
Gaussian parametric	$R \sim \mathcal{N}(\mu, \sigma^2)$	$-\mu - \sigma z_\alpha$
General parametric	$R \sim F(\theta)$	$-F^{-1}(\alpha; \theta)$
Monte Carlo	Simulated	$-\widehat{q}_\alpha(R^{(1)}, \ldots, R^{(M)})$
Conditional / GARCH	$R_{t+1} = \mu_{t+1\|t} + \sigma_{t+1\|t} Z_{t+1}$	$-\mu_{t+1\|t} - \sigma_{t+1\|t} z_\alpha$

VaR Backtesting: Coverage and Independence Tests

Once a Value at Risk model has been estimated, it is important to verify whether its predicted VaR levels are consistent with the losses actually observed.

Let

$$I_t = \begin{cases} 1, & \text{if a VaR exception occurs at time } t, \\ 0, & \text{otherwise.} \end{cases}$$

A VaR exception occurs when the realized return falls below the predicted lower-tail VaR threshold.

For example, for a 95% VaR,

$$p = 1 - 0.95 = 0.05,$$

so a correctly specified VaR model should generate approximately 5% exceptions.

Two properties are particularly important:

1. the **frequency** of exceptions should be correct;
2. the exceptions should be **independent over time**.

The first property is tested using the Kupiec Unconditional Coverage test, while the second is tested using the Christoffersen Independence test.

Kupiec Unconditional Coverage Test

Kupiec (1995) introduced a likelihood-ratio test, also called the **Proportion of Failures (POF) test**, to determine whether the observed frequency of VaR exceptions is consistent with the theoretical probability implied by the VaR confidence level.

Suppose that

$$N = \text{number of VaR forecasts}$$

and

$$x = \sum_{t=1}^{N} I_t = \text{number of VaR exceptions.}$$

The empirical exception frequency is therefore

$$\boxed{\widehat{p} = \frac{x}{N}.}$$

If the VaR model is correctly calibrated, the theoretical exception probability p should be equal to the observed exception frequency.

The hypotheses are

$$\boxed{H_0 : \mathbb{P}(I_t = 1) = p}$$

against

$$\boxed{H_1 : \mathbb{P}(I_t = 1) \neq p.}$$

Equivalently,

$$H_0 : \widehat{p} \text{ is statistically consistent with } p.$$

Likelihood Under the Null Hypothesis Under H_0, each exception indicator follows a Bernoulli distribution:

$$I_t \sim \text{Bernoulli}(p).$$

Therefore, observing x exceptions and $N - x$ non-exceptions gives the likelihood

$$\boxed{L_0 = (1-p)^{N-x} p^x.}$$

Unrestricted Likelihood Under the alternative hypothesis, the exception probability is not restricted to be equal to p.

The maximum-likelihood estimator of the exception probability is simply

$$\widehat{p} = \frac{x}{N}.$$

Hence, the maximized unrestricted likelihood is

$$\boxed{L_1 = \left(1 - \frac{x}{N}\right)^{N-x} \left(\frac{x}{N}\right)^{x}.}$$

Kupiec Likelihood-Ratio Statistic The likelihood-ratio statistic compares the restricted VaR model with the unrestricted model:

$$LR_{\text{POF}} = -2 \ln \left(\frac{L_0}{L_1} \right).$$

Therefore,

$$\boxed{LR_{\text{POF}} = -2 \ln \left[\frac{(1-p)^{N-x} p^x}{\left(1 - \frac{x}{N}\right)^{N-x} \left(\frac{x}{N}\right)^{x}} \right].}$$

Under the null hypothesis,

$$\boxed{LR_{\text{POF}} \overset{a}{\sim} \chi^2_1.}$$

Thus, at significance level α_{test}, we reject the VaR model if

$$LR_{\text{POF}} > \chi^2_{1,1-\alpha_{\text{test}}}.$$

Equivalently,

$$\boxed{p\text{-value} < \alpha_{\text{test}} \quad \Longrightarrow \quad \text{reject correct unconditional coverage.}}$$

Limitation of the Kupiec Test The Kupiec test only examines the **total number of exceptions**.

It does not examine when those exceptions occur.

For example, consider two models producing the same number of exceptions:

$$0, 0, 1, 0, 0, 0, 1, 0, 0, 1, \ldots$$

and

$$0, 0, 0, 0, 1, 1, 1, 1, 0, 0, \ldots$$

The Kupiec test may treat these models similarly because they have the same total number of exceptions.

However, the second sequence exhibits **clustering of VaR failures**, which suggests that the VaR model may fail to adapt correctly to changing risk conditions.

This motivates the Christoffersen Independence test.

Christoffersen Independence Test

Christoffersen (1998) proposed a test to determine whether VaR exceptions occur independently over time.

The null hypothesis is

$$\boxed{H_0 : I_t \text{ is independent of } I_{t-1}.}$$

The alternative is

$$\boxed{H_1 : I_t \text{ depends on } I_{t-1}.}$$

In particular, the test determines whether observing an exception today changes the probability of observing an exception tomorrow.

Transition Counts The test considers the four possible transitions between two consecutive periods:

$$\boxed{\begin{aligned} n_{00} &= \text{number of transitions } 0 \to 0, \\ n_{01} &= \text{number of transitions } 0 \to 1, \\ n_{10} &= \text{number of transitions } 1 \to 0, \\ n_{11} &= \text{number of transitions } 1 \to 1. \end{aligned}}$$

For example,

$$I_{t-1} = 0, \qquad I_t = 1$$

corresponds to a $0 \to 1$ transition and therefore contributes to

$$n_{01}.$$

The four transition counts can be summarized by the transition table

	$I_t = 0$	$I_t = 1$
$I_{t-1} = 0$	n_{00}	n_{01}
$I_{t-1} = 1$	n_{10}	n_{11}

Conditional Exception Probabilities Define

$$\boxed{\pi_0 = \frac{n_{01}}{n_{00} + n_{01}}}$$

as the probability of an exception at time t, conditional on no exception at time $t-1$:

$$\pi_0 = \mathbb{P}\left(I_t = 1 \mid I_{t-1} = 0\right).$$

Similarly,

$$\boxed{\pi_1 = \frac{n_{11}}{n_{10} + n_{11}}}$$

is the probability of an exception at time t, conditional on an exception at time $t-1$:

$$\pi_1 = \mathbb{P}\left(I_t = 1 \mid I_{t-1} = 1\right).$$

Finally, the unconditional empirical probability of an exception is

$$\boxed{\pi = \frac{n_{01} + n_{11}}{n_{00} + n_{01} + n_{10} + n_{11}}.}$$

Intuition If exceptions are independent, knowing whether an exception occurred yesterday should provide no information about the probability of an exception today.

Therefore, under independence,

$$\boxed{\pi_0 = \pi_1 = \pi.}$$

In particular, if

$$\pi_1 \gg \pi_0,$$

an exception is much more likely after another exception.

This provides evidence of

$$\boxed{\text{clustering of VaR exceptions.}}$$

Likelihood Under Independence Under the null hypothesis of independence, the probability of an exception does not depend on the previous state.

The likelihood is therefore

$$\boxed{L_{\text{ind},0} = (1-\pi)^{n_{00}+n_{10}} \pi^{n_{01}+n_{11}}.}$$

Under the alternative hypothesis, the probability of an exception depends on the previous state:

$$\boxed{L_{\text{ind},1} = (1-\pi_0)^{n_{00}} \pi_0^{n_{01}} (1-\pi_1)^{n_{10}} \pi_1^{n_{11}}.}$$

Christoffersen Independence Statistic The likelihood-ratio statistic is

$$LR_{\text{CCI}} = -2\ln\left(\frac{L_{\text{ind},0}}{L_{\text{ind},1}}\right).$$

Hence,

$$\boxed{LR_{\text{CCI}} = -2\ln\left[\frac{(1-\pi)^{n_{00}+n_{10}}\pi^{n_{01}+n_{11}}}{(1-\pi_0)^{n_{00}}\pi_0^{n_{01}}(1-\pi_1)^{n_{10}}\pi_1^{n_{11}}}\right].}$$

Under the null hypothesis,

$$\boxed{LR_{\text{CCI}} \overset{a}{\sim} \chi^2_1.}$$

Therefore,

$$\boxed{p\text{-value} < \alpha_{\text{test}} \quad \Longrightarrow \quad \text{reject independence of VaR exceptions.}}$$

A rejection indicates that VaR exceptions exhibit temporal dependence, such as clustering.

Christoffersen Conditional Coverage Test

A good VaR model should satisfy **both**:

$$\boxed{\begin{array}{ll} \text{Correct coverage:} & \mathbb{P}(I_t = 1) = p, \\ \text{Independence:} & I_t \text{ is independent over time.} \end{array}}$$

Christoffersen's Conditional Coverage (CC) test jointly tests these two properties.

The test statistic is the sum of the unconditional coverage and independence statistics:

$$\boxed{LR_{\text{CC}} = LR_{\text{POF}} + LR_{\text{CCI}}.}$$

Since each component contributes one restriction, under the joint null hypothesis,

$$\boxed{LR_{\text{CC}} \overset{a}{\sim} \chi^2_2.}$$

The joint null hypothesis is therefore

$$\boxed{H_0 : \begin{cases} \mathbb{P}(I_t = 1) = p, \\ I_t \text{ is independent over time.} \end{cases}}$$

At significance level α_{test},

$$\boxed{p\text{-value} < \alpha_{\text{test}} \quad \Longrightarrow \quad \text{reject the VaR model's conditional coverage.}}$$

Summary of VaR Backtesting Tests

Test	Question	Statistic	Distribution
Kupiec POF	Correct exception frequency?	LR_{POF}	χ^2_1
Christoffersen IND	Are exceptions independent?	LR_{CCI}	χ^2_1
Christoffersen CC	Both properties jointly?	$LR_{\text{CC}} = LR_{\text{POF}} + LR_{\text{CCI}}$	χ^2_2

Hence, a well-calibrated VaR model should generate

$$\boxed{\text{the correct number of exceptions}}$$

and those exceptions should

$$\boxed{\text{not exhibit systematic clustering over time.}}$$

CoVaR and Systemic Risk

Value at Risk (VaR) measures the tail risk of a single institution or portfolio considered in isolation. However, in a financial system, an important question is whether the distress of one institution is associated with distress in the rest of the financial system.

The **CoVaR** framework considers the tail risk of the financial system *conditional on the state of a particular institution.*

Let

$$r_{m,t}$$

denote the return of the market or financial system at time t, and let

$$r_{i,t}$$

denote the return of institution i.

Individual Value at Risk

The α-VaR of institution i is defined as the lower α-quantile of its return distribution:

$$\boxed{\mathbb{P}\left(r_{i,t} \leq VaR_{i,t}(\alpha)\right) = \alpha.}$$

For example, if

$$\alpha = 0.05,$$

then

$$VaR_{i,t}(0.05)$$

is the 5% lower-tail quantile of the institution's return.

Thus,

$$\boxed{\mathbb{P}\left(r_{i,t} \leq VaR_{i,t}(0.05)\right) = 0.05.}$$

Here, VaR is written using the **signed return convention**, so the lower-tail VaR is typically negative.

Definition of CoVaR

CoVaR measures the VaR of the financial system conditional on a particular state of institution i.

More generally, let $C(r_{i,t})$ denote some condition imposed on institution i. Then CoVaR is defined through

$$\boxed{\mathbb{P}\left(r_{m,t} \leq CoVaR_t^{m|C(r_i)}(\alpha) \;\middle|\; C(r_{i,t})\right) = \alpha.}$$

The notation

$$CoVaR_t^{m|C(r_i)}(\alpha)$$

means:

> the α-quantile of the market return conditional on institution i being in state C.

Institution i in Distress The distress state of institution i is usually represented by setting its return equal to its own VaR:

$$\boxed{r_{i,t} = VaR_{i,t}(\alpha).}$$

The corresponding CoVaR is therefore defined by

$$\boxed{\mathbb{P}\left(r_{m,t} \leq CoVaR_t^{m|r_i=VaR_i(\alpha)} \;\middle|\; r_{i,t} = VaR_{i,t}(\alpha)\right) = \alpha.}$$

Thus,

$$\boxed{CoVaR_t^{m|r_i=VaR_i(\alpha)}}$$

measures the lower-tail risk of the market when institution i is itself in a tail-risk state.

Conceptually,

$$\boxed{\text{VaR}_i = \text{tail risk of institution } i,}$$

whereas

$$\boxed{\text{CoVaR}^{m|i} = \text{tail risk of the market conditional on the state of } i.}$$

Why Do We Need a Benchmark State?

A low market CoVaR when institution i is distressed does not, by itself, tell us how much institution i contributes to systemic risk.

The market may already be risky even when institution i is in a normal state.

We therefore compare the market CoVaR under two states of institution i:

$$\boxed{\begin{array}{ll} \text{Distress state:} & r_{i,t} = VaR_{i,t}(\alpha), \\ \text{Normal state:} & r_{i,t} = \text{Median}(r_{i,t}). \end{array}}$$

The median is used as a benchmark representing a "normal" state of the institution.

$\Delta CoVaR$

The contribution of institution i to systemic risk is measured by the difference between the market CoVaR when institution i is distressed and the market CoVaR when institution i is in its median state:

$$\boxed{\Delta CoVaR_{i,t}(\alpha) = CoVaR_t^{m|r_i=VaR_i(\alpha)} - CoVaR_t^{m|r_i=\text{Median}(r_i)}.}$$

Therefore,

$$\boxed{\Delta CoVaR_i = \text{market tail risk when } i \text{ is distressed} - \text{market tail risk when } i \text{ is normal.}}$$

If returns are used and lower-tail VaR is negative, a strongly negative

$$\Delta CoVaR_i$$

means that the market's lower-tail outcome becomes substantially worse when institution i moves from its normal state to its distress state.

Thus, under this signed-return convention,

$$\boxed{\text{more negative } \Delta CoVaR_i \quad \Longrightarrow \quad \text{larger adverse systemic-risk contribution.}}$$

CoVaR under a Gaussian Linear Model

To understand the formula used in the notebook, suppose that the market and institution returns are jointly Gaussian:

$$\boxed{\begin{pmatrix} R_{m,t+1} \\ R_{i,t+1} \end{pmatrix} \sim \mathcal{N}\left[\begin{pmatrix} \mu_{m,t+1} \\ \mu_{i,t+1} \end{pmatrix}, \begin{pmatrix} \sigma^2_{m,t+1} & \sigma_{mi,t+1} \\ \sigma_{mi,t+1} & \sigma^2_{i,t+1} \end{pmatrix} \right].}$$

Define the conditional correlation between the market and institution i as

$$\boxed{\rho_{m,i,t+1} = \frac{\sigma_{mi,t+1}}{\sigma_{m,t+1}\sigma_{i,t+1}}.}$$

Equivalently,

$$\sigma_{mi,t+1} = \rho_{m,i,t+1}\sigma_{m,t+1}\sigma_{i,t+1}.$$

Conditional Distribution of the Market

For two jointly Gaussian random variables, the conditional distribution of the market return given

$$R_{i,t+1} = r_i$$

is Gaussian:

Derivation of the Conditional Gaussian Distribution Consider the market return $R_{m,t+1}$ and the return of institution i, $R_{i,t+1}$. Assume that they are jointly Gaussian:

$$\boxed{\begin{pmatrix} R_{m,t+1} \\ R_{i,t+1} \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_{m,t+1} \\ \mu_{i,t+1} \end{pmatrix}, \begin{pmatrix} \sigma^2_{m,t+1} & \sigma_{mi,t+1} \\ \sigma_{mi,t+1} & \sigma^2_{i,t+1} \end{pmatrix} \right).}$$

Here,

$$\mu_{m,t+1} = \mathbb{E}[R_{m,t+1}], \qquad \mu_{i,t+1} = \mathbb{E}[R_{i,t+1}],$$

$$\sigma^2_{m,t+1} = \mathrm{Var}(R_{m,t+1}), \qquad \sigma^2_{i,t+1} = \mathrm{Var}(R_{i,t+1}),$$

and

$$\boxed{\sigma_{mi,t+1} = \mathrm{Cov}\left(R_{m,t+1}, R_{i,t+1}\right).}$$

We want to determine the distribution of the market return conditional on observing a particular realization of the institution's return:

$$R_{i,t+1} = r_i.$$

General Conditional Gaussian Result Consider two jointly Gaussian random variables

$$\begin{pmatrix} X \\ Y \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_X \\ \mu_Y \end{pmatrix}, \begin{pmatrix} \sigma^2_X & \sigma_{XY} \\ \sigma_{XY} & \sigma^2_Y \end{pmatrix} \right).$$

Then the conditional distribution of X given $Y = y$ is also Gaussian:

$$\boxed{X \mid Y = y \sim \mathcal{N}\left(\mu_X + \frac{\sigma_{XY}}{\sigma_Y^2}(y - \mu_Y),\ \sigma_X^2 - \frac{\sigma_{XY}^2}{\sigma_Y^2}\right).}$$

Thus, conditioning modifies both the mean and the variance.

Derivation of the Conditional Mean Because X and Y are jointly Gaussian, we can decompose X into a component that is linearly related to Y and an orthogonal residual:

$$\boxed{X = \mu_X + \beta(Y - \mu_Y) + \eta,}$$

where η is chosen such that

$$\mathrm{Cov}(\eta, Y) = 0.$$

This decomposition can be interpreted as the population linear regression of X on Y.

To determine β, write

$$\eta = X - \mu_X - \beta(Y - \mu_Y).$$

The orthogonality condition gives

$$\mathrm{Cov}(\eta, Y) = 0.$$

Therefore,

$$\mathrm{Cov}\left(X - \mu_X - \beta(Y - \mu_Y), Y\right) = 0.$$

Using linearity of covariance,

$$\mathrm{Cov}(X, Y) - \beta\,\mathrm{Cov}(Y, Y) = 0.$$

Since

$$\mathrm{Cov}(X, Y) = \sigma_{XY}$$

and

$$\mathrm{Cov}(Y, Y) = \mathrm{Var}(Y) = \sigma_Y^2,$$

we obtain

$$\sigma_{XY} - \beta\sigma_Y^2 = 0.$$

Hence,

$$\boxed{\beta = \frac{\sigma_{XY}}{\sigma_Y^2}.}$$

The decomposition therefore becomes

$$\boxed{X = \mu_X + \frac{\sigma_{XY}}{\sigma_Y^2}(Y - \mu_Y) + \eta.}$$

Conditioning on $Y = y$ Suppose now that we observe

$$Y = y.$$

Then

$$X \mid Y = y = \mu_X + \frac{\sigma_{XY}}{\sigma_Y^2}(y - \mu_Y) + \eta.$$

Taking the conditional expectation gives

$$\mathbb{E}[X \mid Y = y] = \mu_X + \frac{\sigma_{XY}}{\sigma_Y^2}(y - \mu_Y) + \mathbb{E}[\eta \mid Y = y].$$

Because (X, Y) is jointly Gaussian, η and Y are also jointly Gaussian.
Moreover, we constructed η such that

$$\mathrm{Cov}(\eta, Y) = 0.$$

For jointly Gaussian variables, zero covariance implies independence. Therefore,

$$\eta \perp Y,$$

and consequently

$$\mathbb{E}[\eta \mid Y = y] = \mathbb{E}[\eta] = 0.$$

Hence,

$$\boxed{\mathbb{E}[X \mid Y = y] = \mu_X + \frac{\sigma_{XY}}{\sigma_Y^2}(y - \mu_Y).}$$

Application to Market and Institution Returns We now identify

$$X = R_{m,t+1}$$

and

$$Y = R_{i,t+1}.$$

Therefore,

$$\mu_X = \mu_{m,t+1}, \qquad \mu_Y = \mu_{i,t+1},$$

and

$$\sigma_{XY} = \sigma_{mi,t+1}.$$

Conditioning on

$$R_{i,t+1} = r_i,$$

we obtain

$$\boxed{\mu_{m|i} = \mathbb{E}\left[R_{m,t+1} \mid R_{i,t+1} = r_i\right] = \mu_{m,t+1} + \frac{\sigma_{mi,t+1}}{\sigma^2_{i,t+1}}\left(r_i - \mu_{i,t+1}\right).}$$

Interpretation The conditional mean consists of two components:

$$\boxed{\mu_{m|i} = \underbrace{\mu_{m,t+1}}_{\text{unconditional market mean}} + \underbrace{\frac{\sigma_{mi,t+1}}{\sigma^2_{i,t+1}}}_{\text{sensitivity of market to institution } i} \underbrace{\left(r_i - \mu_{i,t+1}\right)}_{\text{institution's deviation from its mean}}.}$$

The coefficient

$$\boxed{\beta_{m|i} = \frac{\sigma_{mi,t+1}}{\sigma^2_{i,t+1}}}$$

is exactly the population OLS slope coefficient obtained when regressing the market return on the institution's return:

$$R_{m,t+1} = a + \beta_{m|i} R_{i,t+1} + \eta_{t+1}.$$

Indeed,

$$\boxed{\beta_{m|i} = \frac{\mathrm{Cov}(R_m, R_i)}{\mathrm{Var}(R_i)}.}$$

Therefore,

$$\boxed{\mu_{m|i} = \mu_m + \beta_{m|i}(r_i - \mu_i).}$$

If

$$r_i = \mu_i,$$

then

$$\mu_{m|i} = \mu_m.$$

Thus, observing institution i exactly at its mean does not shift the conditional expected market return.

If instead

$$r_i < \mu_i$$

and

$$\sigma_{mi} > 0,$$

then

$$\mu_{m|i} < \mu_m.$$

Hence, a negative shock to an institution that is positively related to the market shifts the conditional distribution of the market to the left.

Expression in Terms of Correlation Recall that

$$\boxed{\rho_{m,i} = \frac{\sigma_{mi}}{\sigma_m \sigma_i}.}$$

Therefore,

$$\sigma_{mi} = \rho_{m,i}\sigma_m\sigma_i.$$

Substituting into the regression coefficient gives

$$\begin{aligned} \frac{\sigma_{mi}}{\sigma_i^2} &= \frac{\rho_{m,i}\sigma_m\sigma_i}{\sigma_i^2} \\ &= \boxed{\rho_{m,i}\frac{\sigma_m}{\sigma_i}.} \end{aligned}$$

Hence, the conditional mean can equivalently be written as

$$\boxed{\mu_{m|i} = \mu_m + \rho_{m,i}\frac{\sigma_m}{\sigma_i}(r_i - \mu_i).}$$

This form makes clear that the response of the market to the state of institution i depends on

$$\boxed{\text{correlation} \times \text{relative volatility}.}$$

Conditional Variance The residual in the previous decomposition is

$$\eta = X - \mu_X - \frac{\sigma_{XY}}{\sigma_Y^2}(Y - \mu_Y).$$

Its variance is

$$\mathrm{Var}(\eta) = \mathrm{Var}\left(X - \frac{\sigma_{XY}}{\sigma_Y^2}Y\right).$$

Using

$$\mathrm{Var}(aX + bY) = a^2\,\mathrm{Var}(X) + b^2\,\mathrm{Var}(Y) + 2ab\,\mathrm{Cov}(X, Y),$$

we obtain

$$\begin{aligned}
\mathrm{Var}(\eta) &= \sigma_X^2 + \left(\frac{\sigma_{XY}}{\sigma_Y^2}\right)^2 \sigma_Y^2 - 2\frac{\sigma_{XY}}{\sigma_Y^2}\sigma_{XY} \\
&= \sigma_X^2 + \frac{\sigma_{XY}^2}{\sigma_Y^2} - 2\frac{\sigma_{XY}^2}{\sigma_Y^2} \\
&= \boxed{\sigma_X^2 - \frac{\sigma_{XY}^2}{\sigma_Y^2}.}
\end{aligned}$$

Since η is independent of Y under joint Gaussianity, this is also the conditional variance:

$$\boxed{\mathrm{Var}(X \mid Y = y) = \sigma_X^2 - \frac{\sigma_{XY}^2}{\sigma_Y^2}.}$$

Applied to market and institution returns,

$$\boxed{\sigma_{m|i}^2 = \sigma_m^2 - \frac{\sigma_{mi}^2}{\sigma_i^2}.}$$

Using

$$\sigma_{mi} = \rho_{m,i}\sigma_m\sigma_i,$$

we obtain

$$\begin{aligned}
\sigma_{m|i}^2 &= \sigma_m^2 - \frac{\rho_{m,i}^2\sigma_m^2\sigma_i^2}{\sigma_i^2} \\
&= \boxed{\sigma_m^2(1 - \rho_{m,i}^2).}
\end{aligned}$$

Therefore,

$$\boxed{R_{m,t+1} \mid R_{i,t+1} = r_i \sim \mathcal{N}\left(\mu_{m,t+1} + \rho_{m,i,t+1}\frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}(r_i - \mu_{i,t+1}),\ \sigma_{m,t+1}^2(1 - \rho_{m,i,t+1}^2)\right).}$$

This conditional Gaussian result is the key mathematical step used to derive the Gaussian linear CoVaR and $\Delta CoVaR$ formulas.

$$R_{m,t+1} \mid R_{i,t+1} = r_i \sim \mathcal{N}\left(\mu_{m|i}, \sigma^2_{m|i}\right),$$

where the conditional mean is

$$\boxed{\mu_{m|i} = \mu_{m,t+1} + \frac{\sigma_{mi,t+1}}{\sigma^2_{i,t+1}}\left(r_i - \mu_{i,t+1}\right).}$$

Using

$$\sigma_{mi,t+1} = \rho_{m,i,t+1}\sigma_{m,t+1}\sigma_{i,t+1},$$

we obtain

$$\frac{\sigma_{mi,t+1}}{\sigma^2_{i,t+1}} = \rho_{m,i,t+1}\frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}.$$

Hence,

$$\boxed{\mu_{m|i} = \mu_{m,t+1} + \rho_{m,i,t+1}\frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}\left(r_i - \mu_{i,t+1}\right).}$$

The conditional variance is

$$\boxed{\sigma^2_{m|i} = \sigma^2_{m,t+1}\left(1 - \rho^2_{m,i,t+1}\right).}$$

Conditional Quantile and CoVaR

Since the conditional distribution is Gaussian, its α-quantile is

$$CoVaR_t^{m|r_i}(\alpha) = \mu_{m|i} + z_\alpha \sigma_{m|i},$$

where

$$z_\alpha = \Phi^{-1}(\alpha).$$

Therefore,

$$\boxed{CoVaR_t^{m|r_i}(\alpha) = \mu_{m,t+1} + \rho_{m,i,t+1}\frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}\left(r_i - \mu_{i,t+1}\right) + z_\alpha \sigma_{m,t+1}\sqrt{1 - \rho^2_{m,i,t+1}}.}$$

Derivation of $\Delta CoVaR$

We now evaluate the conditional market quantile under two different values of the institution's return.

First, under the distress state,

$$r_i = VaR^{\alpha}_{i,t+1},$$

we obtain

$$\begin{aligned} CoVaR_t^{m|r_i=VaR_i^{\alpha}} &= \mu_{m,t+1} + \rho_{m,i,t+1}\frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}\left(VaR^{\alpha}_{i,t+1} - \mu_{i,t+1}\right) \\ &\quad + z_{\alpha}\sigma_{m,t+1}\sqrt{1-\rho^2_{m,i,t+1}}. \end{aligned}$$

Second, under the median state,

$$r_i = \text{Median}(R_{i,t+1}),$$

we obtain

$$\begin{aligned} CoVaR_t^{m|r_i=\text{Median}(R_i)} &= \mu_{m,t+1} + \rho_{m,i,t+1}\frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}\left(\text{Median}(R_{i,t+1}) - \mu_{i,t+1}\right) \\ &\quad + z_{\alpha}\sigma_{m,t+1}\sqrt{1-\rho^2_{m,i,t+1}}. \end{aligned}$$

Recall that

$$\Delta CoVaR^{\alpha}_{i,t} = CoVaR_t^{m|r_i=VaR_i^{\alpha}} - CoVaR_t^{m|r_i=\text{Median}(R_i)}.$$

When we subtract the two expressions, the terms

$$\mu_{m,t+1}$$

and

$$z_{\alpha}\sigma_{m,t+1}\sqrt{1-\rho^2_{m,i,t+1}}$$

cancel because they are identical under the two conditioning states.

Similarly, the terms involving

$$\mu_{i,t+1}$$

cancel.

We are therefore left with

$$\boxed{\Delta CoVaR^{\alpha}_{i,t} = \rho_{m,i,t+1}\frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}\left[VaR^{\alpha}_{i,t+1} - \text{Median}(R_{i,t+1})\right].}$$

Interpretation as a Regression Coefficient

Notice that

$$\boxed{\beta_{m|i,t+1} = \frac{\mathrm{Cov}_t(R_{m,t+1}, R_{i,t+1})}{\mathrm{Var}_t(R_{i,t+1})}}$$

can be written as

$$\begin{aligned}\beta_{m|i,t+1} &= \frac{\rho_{m,i,t+1}\sigma_{m,t+1}\sigma_{i,t+1}}{\sigma^2_{i,t+1}} \\ &= \boxed{\rho_{m,i,t+1}\frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}.}\end{aligned}$$

Therefore, the Gaussian linear $\Delta CoVaR$ formula can also be written as

$$\boxed{\Delta CoVaR^{\alpha}_{i,t} = \beta_{m|i,t+1}\left[VaR^{\alpha}_{i,t+1} - \mathrm{Median}(R_{i,t+1})\right].}$$

This decomposition has a useful interpretation:

$$\boxed{\Delta CoVaR = \underbrace{\beta_{m|i}}_{\substack{\text{sensitivity of the market} \\ \text{to institution } i}} \times \underbrace{(VaR_i - \mathrm{Median}(R_i))}_{\substack{\text{movement of institution } i \\ \text{from normal to distress}}}.}$$

Economic Interpretation

The formula

$$\Delta CoVaR^{\alpha}_{i,t} = \rho_{m,i,t+1}\frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}\left[VaR^{\alpha}_{i,t+1} - \mathrm{Median}(R_{i,t+1})\right]$$

shows that the systemic-risk contribution depends on two main components.

First,

$$\boxed{\rho_{m,i,t+1}\frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}}$$

measures how strongly movements in institution i are associated with movements in the market.

Second,

$$\boxed{VaR^{\alpha}_{i,t+1} - \mathrm{Median}(R_{i,t+1})}$$

measures how far institution i moves when going from its normal state to its distress state.

For a lower-tail VaR,

$$VaR^{\alpha}_{i,t+1} < \mathrm{Median}(R_{i,t+1}),$$

so this difference is generally negative.

If the institution is positively related to the market,

$$\rho_{m,i,t+1} > 0,$$

then

$$\Delta CoVaR^{\alpha}_{i,t} < 0.$$

Thus, the distress of institution i shifts the market's lower-tail quantile downward.

Gaussian Median Under a Gaussian distribution,

$$\boxed{\text{Median}(R_{i,t+1}) = \mu_{i,t+1}.}$$

Moreover,

$$VaR^{\alpha}_{i,t+1} = \mu_{i,t+1} + \sigma_{i,t+1} z_{\alpha}.$$

Therefore,

$$VaR^{\alpha}_{i,t+1} - \text{Median}(R_{i,t+1}) = \sigma_{i,t+1} z_{\alpha}.$$

Substituting into the previous expression gives

$$\begin{aligned} \Delta CoVaR^{\alpha}_{i,t} &= \rho_{m,i,t+1} \frac{\sigma_{m,t+1}}{\sigma_{i,t+1}} (\sigma_{i,t+1} z_{\alpha}) \\ &= \boxed{\rho_{m,i,t+1} \sigma_{m,t+1} z_{\alpha}.} \end{aligned}$$

Since

$$z_{\alpha} < 0$$

for lower-tail probabilities such as $\alpha = 0.05$, positive correlation implies a negative $\Delta CoVaR$.

Summary

The three concepts can be summarized as

$$\boxed{VaR^{\alpha}_{i} = \text{tail risk of institution } i,}$$

$$\boxed{CoVaR^{m|i,\alpha} = \text{tail risk of the market conditional on the state of } i,}$$

and

$$\boxed{\Delta CoVaR_i^{\alpha} = \text{change in market tail risk when } i \text{ moves from its median state to distress.}}$$

Under the Gaussian linear assumption,

$$\boxed{\Delta CoVaR_{i,t}^{\alpha} = \rho_{m,i,t+1} \frac{\sigma_{m,t+1}}{\sigma_{i,t+1}} \left[VaR_{i,t+1}^{\alpha} - \text{Median}(R_{i,t+1}) \right].}$$

Equivalently,

$$\boxed{\Delta CoVaR_{i,t}^{\alpha} = \beta_{m|i,t+1} \left[VaR_{i,t+1}^{\alpha} - \text{Median}(R_{i,t+1}) \right],}$$

with

$$\boxed{\beta_{m|i,t+1} = \rho_{m,i,t+1} \frac{\sigma_{m,t+1}}{\sigma_{i,t+1}}.}$$

Hence, $\Delta CoVaR$ measures how much the market's tail-risk threshold changes when institution i moves from a normal state to a distress state.